



Trabajo de Fin de Máster

"Máster Universitario en Microelectrónica: Diseño y Aplicaciones de Sistemas Micro/Nanométricos"

Diseño y caracterización de aleatoriedad de un TRNG basado en osciladores en anillos

Design and randomness assessment of a Ring Oscillator-based TRNG

Autor / Author: Mario García Jiménez

Tutores / Advisors

Fecha / Date: Junio de 2027 / June 2027

Resumen

Las claves criptográficas valen lo que vale la entropía que hay detrás, y sin embargo la mayoría de los generadores embebidos se evalúan como cajas negras: se pasan baterías de tests estadísticos sobre su salida y un aprobado se toma como prueba de aleatoriedad. Este TFM muestra primero, sobre el generador hardware del ESP32, por qué no basta: su salida pasa NIST SP 800-22 y puntúa como ruido en los estimadores de SP 800-90B incluso cuando, como revela una prueba de reinicios, el generador no recibe ningún ruido y solo continúa una secuencia pseudoaleatoria sembrada al arrancar — y el propio NIST ha rechazado después la primera batería para evaluar generadores criptográficos. Después toma el camino contrario: un motor de generación de claves de caja blanca cuya entropía se justifica desde la física del dispositivo. Se diseña a nivel de transistor una fuente de entropía por oscilador de anillo, se extrae su jitter por simulación y se convierte, mediante un modelo estocástico validado, en una cota inferior de min-entropía por bit y en los parámetros de diseño del muestreador. A su alrededor, un núcleo digital totalmente portable — tests de salud SP 800-90B, acondicionado y CTR_DRBG con función de derivación basados en AES, autenticación AES-CMAC e interfaz de registros I2C — se escribe una sola vez y se verifica de tres maneras: contra los vectores oficiales, en simulación sobre una FPGA Basys 3 que hace de vehículo de verificación, y frente a un ESP32 que recalcula cada resultado con una implementación independiente (mbedtls). El mismo VHDL se sintetiza por último sobre una biblioteca abierta de celdas estándar de 130 nm para dar el área en puertas equivalentes y el camino hacia un circuito integrado dedicado. Todo el código, modelos, vectores y datos se publican.

Palabras clave generador de números verdaderamente aleatorios, oscilador en anillo, jitter, min-entropía, modelo estocástico, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, elemento seguro, FPGA, ASIC, ESP32.

Abstract

Cryptographic keys are only as good as the entropy behind them, yet most embedded key generators are evaluated as black boxes: statistical test suites are run on their output and a pass is taken as evidence of randomness. This thesis first shows, on the hardware random number generator of the ESP32, why that is not enough: its output passes NIST SP 800-22 and scores like noise under the SP 800-90B estimators even when, as a restart test reveals, the generator receives no noise at all and only continues a pseudo-random sequence seeded at boot — and NIST itself has since rejected the first suite for assessing cryptographic generators. It then takes the opposite route: a white-box key-generation engine whose entropy is justified from the physics of the device. A ring-oscillator entropy source is designed at transistor level and its jitter is extracted from simulation and turned, through a validated stochastic model, into a lower bound on min-entropy per bit and into the design parameters of the sampler. Around it, a fully portable digital core — SP 800-90B health tests, an AES-based conditioning and CTR_DRBG with derivation function, AES-CMAC authentication and an I2C register interface — is written once and verified three ways: against the official test vectors, in simulation on a Basys 3 FPGA acting as the verification vehicle, and against an ESP32 that recomputes every result with an independent implementation (mbedtls). The same VHDL is finally synthesized on an open 130 nm standard-cell library to give the area in gate equivalents and the path to a dedicated integrated circuit. All code, models, vectors and data are released.

Keywords true random number generator, ring oscillator, jitter, min-entropy, stochastic model, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, secure element, FPGA, ASIC, ESP32.

Agradecimientos

A mi familia, a mis compañeros de Airzone y al profesorado del Máster en Microelectrónica del IMSE-CNM, por su apoyo durante este trabajo.

Índice general

Resumen **1**

Abstract **2**

Agradecimientos **3**

Índice de figuras **7**

Índice de tablas **9**

Lista de acrónimos **12**

1. Introducción **14**

1.1. Motivación: una clave vale lo que vale su entropía 14
1.2. De un estudio de caja negra del ESP32 a un motor de caja blanca 15
1.3. Objetivos 16
1.4. Alcance y exclusiones 16
1.5. Metodología y convenciones de reproducibilidad 17
1.6. Estructura del documento 17

2. Estado del arte **18**

2.1. Arquitecturas de TRNG basadas en osciladores en anillo 18
2.2. Modelos estocásticos y cotas de entropía 19
2.3. Medida embebida del jitter 20
2.4. Normas: AIS 20/31 v3.0 y NIST SP 800-90B 21
2.5. Generadores de los microcontroladores de uso común 22
2.6. Segundas fuentes y recolección de entropía 24
2.7. Núcleos simétricos para hardware con recursos limitados 27
2.8. Elementos seguros comerciales 28
2.9. De la FPGA al ASIC: qué cambia en la fuente de entropía 28
2.10. Hueco que aborda este trabajo 29

3. El generador del ESP32 como caja negra **31**
3.1. El periférico y lo que documenta Espressif 31
3.2. Diseño experimental y captura 32
3.3. La batería SP 800-22 35
3.4. Entropía de Shannon, tasa y temperatura 38
3.5. Un recolector de claves multifuente 40
3.6. La referencia del ESP32 frente a SP 800-90B 45
3.7. Lo que una caja negra puede decir y lo que no 50

4. Fuente de entropía: del transistor a la min-entropía **53**
4.1. El oscilador en anillo elemental 53
4.2. Modelo estocástico y factor de calidad 54
4.3. Entropía condicional exacta frente a la cota asintótica 54
4.3.1. Objetivo de diseño 55
4.4. Estimador embebido del jitter y sus dos correcciones 55
4.5. Banco a nivel de transistor y su resolución 57
4.6. Objetivo de diseño y selección del divisor 58

5. Motor digital **59**
5.1. Arquitectura y la decisión de una única primitiva 59
5.2. Núcleo de cifrado AES-128 60
5.3. AES-CMAC 61
5.4. CTR_DRBG con función de derivación 61
5.5. Tests de salud 61
5.6. Ruta de captura y cruces de dominios de reloj 63
5.7. Interfaz de registros I2C y secuenciador de órdenes 63

6. Verificación **65**
6.1. Estrategia: vectores, simulación, verificador independiente 65
6.2. Modelos de referencia en Python 66
6.3. Bancos de pruebas y resultados 66
6.4. Firmware verificador del ESP32 69
6.5. El motor en un producto: lecturas firmadas 70
6.6. Puesta en marcha en la FPGA 72

7. Camino hacia un circuito integrado **73**
7.1. Flujo de síntesis con celdas estándar 73
7.2. Área por bloque en puertas equivalentes 74
7.3. Optimizaciones y sus ahorros esperados 75
7.4. Lo que necesita el oscilador en anillo en silicio 76

7.5. Coste y plazos de una lanzadera académica 77

8. Resultados y comparación **78**
8.1. Estado frente a los objetivos 78
8.2. Ruido del anillo a nivel de transistor 78
8.3. El anillo con los transistores de un PDK abierto 80
8.4. Punto de funcionamiento: divisor y tasa de bits 83
8.5. Verificación funcional consolidada 83
8.6. Comparación con elementos seguros comerciales 84
8.7. La referencia del ESP32 86
8.8. Balance de incertidumbre 86
8.9. La campaña pendiente 88

9. Conclusiones y trabajo futuro **89**
9.1. Conclusiones 89
9.2. Aportaciones 89
9.3. Trabajo futuro 89

Bibliografía **90**

A. Mapa de registros **103**

B. Vectores de prueba oficiales utilizados **107**

C. Guía de reproducción **109**

Índice de figuras

3.1. Tareas y núcleos del firmware de captura. La tarea de campaña lee el generador a RAM mientras se mantiene la condición; la tarea del ADC (condiciones A y C) y el controlador de Wi-Fi (A y B) funcionan en el otro núcleo. Las flechas discontinuas son los caminos por los que las fuentes documentadas pueden llegar al generador. La tarea del ADC se detiene antes de transmitir el búfer. 33

3.2. Valores p de SP 800-22 de las cuatro secuencias de la primera campaña, uno por condición y prueba, de 10^6 bits cada una [MEDIDO]. Una prueba falla por debajo de 0,01; ninguna lo hace. Las pruebas con varios valores p muestran su mínimo. 37

3.3. Autocorrelación y periodograma de las cuatro secuencias de la primera campaña [MEDIDO], frente al ruido blanco (discontinua). (a) Distribución de $r_k\sqrt{n}$ para retardos k de 1 a 1000, $n = 10^6$; para ruido blanco es normal estándar. (b) Distribución de los intervalos del periodograma de los primeros 65 536 bits, divididos por su media, en escala logarítmica; para ruido blanco es exponencial, y por eso la planitud espectral de un único periodograma tiende a $e^{-\gamma} \approx 0{,}561$ y no a 1. 39

3.4. Entropía de bytes de las cuatro secuencias de la primera campaña frente a la lectura del sensor del chip [MEDIDO], en su propia unidad, un entero sin calibrar en grados Fahrenheit; una barra abarca las lecturas anterior y posterior a la secuencia. La banda es una fuente perfecta de 125 000 bytes, media y una desviación típica (calculado). Toda la campaña es un paso del sensor, así que la correlación de $r = 0{,}795$ calculada sobre estos puntos no dice nada de la temperatura. 40

3.5. Rendimiento del corrector de Von Neumann sobre bits de entrada independientes (calculado). Incluso una entrada sin sesgo pierde tres cuartas partes de sus bits; la salida carece de sesgo para todo p de $(0, 1)$ solo si los bits de entrada son independientes. 41

3.6. Validación fuera de línea de la construcción del recolector (calculado, sobre entradas medidas). (a) Distancia de Hamming entre dos de las 1000 semillas derivadas de un patrón de SRAM y de la secuencia de la condición D, sobre 20 000 pares aleatorios, frente a la distribución binomial de semillas uniformes independientes. (b) Valores p de las siete pruebas de SP 800-22 aplicables a los 256 000 bits del flujo de semillas. Ambas se pasan por construcción (véase el texto) y no dicen nada de la entropía de las semillas. 43

5.1. Ruta de datos del motor. Solo los osciladores en anillo dependen de la tecnología; todos los demás bloques son RTL portable. 59

Índice de tablas

2.1. Núcleos TRNG compatibles con AIS en Xilinx Spartan-6 (45 nm), según Petura *et al.* [8], tabla II. La entropía es la estimación T8; la "viabilidad" es la clasificación de los autores (cuanto más alta, más fácil). 19
2.2. Evaluaciones publicadas de los generadores hardware de componentes de clase IoT. "Condiciones" indica si se varió el estado de funcionamiento. . . 25

3.1. Las cuatro campañas de captura [MEDIDO]. Cada ciclo es un arranque en frío seguido de las cuatro condiciones en el orden D, C, B, A, de 10^6 bits cada una. 34
3.2. Métricas estadísticas de la primera campaña, una secuencia de 10^6 bits por condición [MEDIDO]: entropía de Shannon por byte y por bit, mayor autocorrelación $|r_k|$ para retardos k de 1 a 1000, planitud espectral de un periodograma de los primeros 65 536 bits, razón de compresión de zlib y χ^2 de bytes. La última fila es lo que devuelve una fuente perfecta con esta longitud (calculado, véase el texto). 38
3.3. El generador del ESP32 frente a los estimadores de SP 800-90B [MEDIDO]. Condiciones A–D: nueve repeticiones de 10^6 bits cada una, tres en cada uno de los tres chips, media y rango sobre las repeticiones; E y F: capturas únicas de $3{,}5 \times 10^6$ y $8{,}4 \times 10^6$ bits. El techo es el valor más alto que el estimador del valor más común (MCV) puede devolver para una fuente perfectamente uniforme de esa longitud con un 99 % de confianza; la cifra no IID es el mínimo de los diez estimadores de la Sección 6.3 de la norma, y la última columna nombra el estimador que la fijó (veces de nueve). Los veredictos de la vía IID se discuten en el texto. 46

3.4. Prueba de reinicios del generador del ESP32 [MEDIDO]: 1000 reinicios en frío por variante, un chip. H_r y H_c son el mínimo de los estimadores no IID sobre el conjunto de filas y el conjunto de columnas de la norma (1000 reinicios × 1000 bits); la prueba falla cuando cualquiera de los dos queda por debajo de la mitad de la estimación inicial H_I, la cifra no IID más baja de este chip en la misma condición (0,838 bit por bit en D, 0,859 en B). El formato de bytes da los mismos veredictos. Los solapes son reinicios cuya salida aparece desplazada dentro de la de otro, todos idénticos palabra por palabra. Las lecturas tardías caen entre 352 y 412 ms después de que arranque la aplicación, la mayoría a 352 ms. 48
3.5. Qué fuente inyecta ruido [MEDIDO], tercer chip, sin el ruido de arranque: una primera lectura al arrancar, una fuente y una segunda lectura en un instante fijo de 800 ms. Una fuente inyecta ruido si las segundas lecturas dejan de ser ventanas de la secuencia fija (solapes) y no se repiten nunca. 300 reinicios por variante, 1000 para el ADC y la fuente de arranque, que además pasan la prueba de reinicios de la norma (H_r / H_c 0,864 / 0,877 y 0,840 / 0,842). 50

4.1. Entropía de Shannon condicional por bit: cálculo exacto con la gaussiana envuelta frente a desarrollos truncados [SIMULADO]. 55
4.2. Recuperación de un Q_{bruto} conocido mediante el estimador embebido corregido ($K = 20\,000$ bloques de $N = 100$ pares) [SIMULADO]. 56

6.1. Bancos de pruebas, qué comprueban y su resultado (GHDL 6.0, VHDL-2008). 67

7.1. Síntesis lógica sobre IHP SG13G2, aplanada, sin rutado. 1 GE = 7,2576 μm^2. Cota inferior del área [SIMULADO]. 74

8.1. Objetivos y evidencia que respalda cada uno. 79
8.2. Jitter por periodo del anillo de cinco etapas frente a la densidad de ruido inyectada [SIMULADO]. $\sigma_{\text{rel}} = \sigma/T$. 79
8.3. Anillo de cinco etapas en IHP SG13G2 [SIMULADO]. La densidad de ruido no se supone: es el suelo blanco del propio modelo de ruido de los transistores en el punto de conmutación. El punto térmico es la media de ocho realizaciones independientes, ± una desviación típica de la media; las demás filas son realizaciones únicas. La potencia es la media sobre periodos completos. 81
8.4. Divisor y tasa de bits resultante por anillo para $Q \geq 0{,}2286$ [ESTIMADO]. . 83
8.5. Totales de la verificación [SIMULADO]. 84

8.6. El motor frente a elementos seguros comerciales. Datos de certificación tomados de las hojas de datos de los fabricantes y de los registros de validación del NIST [7, 102, 98, 99]. 85

A.1. Registros. R: lectura, W: escritura. 104
A.2. STATUS (0x02). 104
A.3. DIAG (0x0E). 105
A.4. CONTROL (0x03). Una orden por escritura. 105

B.1. Valores de referencia y su origen. Las cuentas son las comprobaciones superadas en la última ejecución (`results/tb_motor_top.log`, 177 en total, con 11 de 11 aserciones PSL y 6 de 6 coberturas). 108

C.1. Lo que reproduce el flujo de integración continua. 109

Lista de acrónimos

AES	Advanced Encryption Standard, estándar de cifrado avanzado
AIS	Anwendungshinweise und Interpretationen zum Schema (BSI)
APT	Adaptive Proportion Test, prueba de proporción adaptativa
ASIC	circuito integrado de aplicación específica
BSI	Bundesamt für Sicherheit in der Informationstechnik
CAVP	Cryptographic Algorithm Validation Program del NIST
CDC	cruce de dominios de reloj
CMAC	código de autenticación de mensajes basado en cifrado
CTR_DRBG	generador determinista de bits aleatorios en modo contador
DRBG	generador determinista de bits aleatorios
ERO	oscilador en anillo elemental
FPGA	matriz de puertas programable en campo
GE	puerta equivalente
I2C	bus entre circuitos integrados
KAT	prueba de respuesta conocida
LUT	tabla de consulta
MEMS	sistema microelectromecánico
MPW	oblea multiproyecto
NIST	National Institute of Standards and Technology
PDK	kit de diseño del proceso

PLL	lazo de enganche de fase
PUF	función físicamente no clonable
PSL	Property Specification Language
RCT	Repetition Count Test, prueba de recuento de repeticiones
RO	oscilador en anillo
RTL	nivel de transferencia de registros
VHDL	lenguaje de descripción de hardware VHSIC
SPI	interfaz periférica serie
TRNG	generador de números verdaderamente aleatorios
XADC	convertidor analógico-digital de Xilinx

Capítulo 1

Introducción

1.1. Motivación: una clave vale lo que vale su entropía

Toda conexión segura que abre un dispositivo embebido, toda imagen de firmware que autentica y toda credencial que almacena descansan sobre una clave, y toda clave descansa sobre una fuente de aleatoriedad. Si esa fuente es más débil de lo supuesto, ninguna sofisticación criptográfica posterior recupera lo que se perdió al principio: una clave de 128 bits extraída de un generador con 40 bits efectivos de entropía es una clave de 40 bits. No es una preocupación teórica. La inyección de frecuencia en la línea de alimentación de un microcontrolador seguro comercial redujo su espacio de claves de 2^{64} a unas 3300 [1]; un generador certificado de tarjeta inteligente resultó estar sesgado pese a superar las baterías estadísticas estándar [2]; y el generador hardware de uno de los microcontroladores IoT más extendidos, el ESP32, devuelve en silencio valores pseudoaleatorios cuando su radio está apagada sin dejar de superar todos los tests de caja negra [3], algo que este trabajo mide directamente con la prueba de reinicios de SP 800-90B (Sección 3.6).

El hilo común es que la aleatoriedad se ha juzgado mirando la salida. Las baterías estadísticas como NIST SP 800-22 comprueban si un flujo de bits *parece* uniforme e independiente; no pueden distinguir una fuente de ruido física de un generador determinista cuya semilla es una constante, y el NIST lo dijo formalmente en 2022 al anunciar la revisión de esa publicación, "rechazando su uso para evaluar generadores criptográficos de números aleatorios" [4]. Las normas actuales — AIS 20/31 en Europa [5] y SP 800-90B en Estados Unidos [6] — piden algo distinto: un modelo estocástico de la fuente de ruido, una estimación de su min-entropía derivada de ese modelo y tests de salud que vigilen la propia fuente en lugar de su salida. Es un problema de microelectrónica antes que de criptografía, y es el problema que aborda este TFM.

1.2. De un estudio de caja negra del ESP32 a un motor de caja blanca

Este trabajo tiene dos partes, y la primera es la justificación de la segunda. La primera caracteriza como caja negra el generador hardware de números aleatorios del ESP32: se ejercita el mismo periférico en cuatro condiciones de funcionamiento, su salida se pasa por una implementación autovalidada de los quince tests de SP 800-22 y por los estimadores de SP 800-90B, se construye un esquema de recolección de claves que combina varias fuentes internas del chip, y una prueba de reinicios con y sin las fuentes de ruido documentadas abre la caja todo lo que se puede abrir desde fuera (Capítulo 3). Su hallazgo central es metodológico: los tests se superan en condiciones en las que la entropía no puede ser la que los tests dan a entender. Con la radio y el ADC apagados el generador no recibe ningún ruido: solo continúa una secuencia pseudoaleatoria sembrada por el bootloader, y si se quita ese ruido de arranque su salida es una secuencia fija leída desde un punto algo distinto en cada arranque. Aun así, todos los tests estadísticos puntúan la salida como ruido — que es exactamente el punto ciego descrito antes. Dos cifras de la primera campaña también tuvieron que revisarse durante el trabajo — se leyó una correlación con la temperatura en un sensor cuya resolución era un solo escalón en todo el rango observado, y se infirió una cifra de estabilidad donde podía haberse medido — y ambas lecciones se convierten en reglas en la Sección 1.5.

La segunda parte construye el generador en lugar de medir el de otro: un motor cuya fuente de entropía esté diseñada, cuyo jitter se mida dentro del dispositivo, cuya min-entropía se deduzca de un modelo y cuyas claves produzcan y usen primitivas verificadas frente a las normas. El ESP32 cambia de papel: de objeto de estudio pasa a juez. Tiene sus propios aceleradores AES y SHA y una biblioteca criptográfica madura, de modo que puede recalcular todo lo que produce el motor con una implementación que no comparte nada con él, a través de un bus real.

El motor está diseñado para silicio. Los elementos seguros — pequeños circuitos integrados que generan, almacenan y usan claves en nombre de un microcontrolador anfitrión a través de I2C — son una categoría de producto madura; la *transparencia* de su entropía no lo es. De tres componentes de uso extendido, uno se vende con una validación NIST de su fuente de entropía y dos no mencionan SP 800-90 en ningún lugar de su documentación [7]. La FPGA de este trabajo es, por tanto, el vehículo de verificación, no el objetivo: permite ejecutar millones de vectores de prueba contra el diseño en minutos, algo que ningún simulador a nivel de transistor puede hacer, antes de que la misma descripción se sintetice sobre celdas estándar.

1.3. Objetivos

O1 Diseñar a nivel de transistor una fuente de entropía de oscilador en anillo, con habilitación y muestreador, y simularla con ruido, declarando la resolución numérica del banco antes de leer de él ninguna cifra.

O2 Extraer el jitter de las formas de onda simuladas con un estimador que separe las componentes de cuantificación, térmica y de parpadeo, validado previamente sobre datos sintéticos de jitter conocido.

O3 Convertir el jitter en una cota de min-entropía mediante un modelo estocástico contrastado con un cálculo exacto, y deducir el divisor del muestreador que cumple con margen el objetivo de AIS 20/31.

O4 Escribir el motor digital — tests de salud, acondicionamiento basado en AES y CTR_DRBG, AES-CMAC y una interfaz de registros I2C — como RTL independiente de la tecnología, verificado frente a vectores oficiales.

O5 Verificar funcionalmente el motor en una FPGA con el ESP32 como oráculo independiente: pruebas de respuesta conocida, interoperabilidad cruzada y reto-respuesta.

O6 Realizar una campaña de entropía sobre silicio real (los anillos de la FPGA) con los estimadores de SP 800-90B y comparar con el generador del ESP32 bajo la misma metodología.

O7 Sintetizar el motor sobre una biblioteca de celdas estándar abierta e informar de su área en puertas equivalentes, con los factores de crecimiento hasta el área de núcleo.

O8 Comparar el diseño con el generador del ESP32, con elementos seguros comerciales y con circuitos publicados, con incertidumbres declaradas.

El resultado mínimo defendible es O1–O3 junto con O5: una fuente de entropía diseñada desde el transistor, con su jitter extraído y convertido en una cota, y un motor digital verificado frente a una implementación independiente. Un resultado negativo bien medido cierra el trabajo igual que uno positivo: si el exponente de acumulación en hardware es dos en lugar de uno, domina el ruido de parpadeo y el divisor no puede crecer indefinidamente, y eso es un hallazgo.

1.4. Alcance y exclusiones

Cuatro cosas quedan deliberadamente fuera del alcance, cada una por una razón explícita. *Fabricación*: una lanzadera académica es asequible y el motor cabe en el bloque más pequeño que se ofrece, pero el plazo desde el tape-out hasta tener muestras encapsuladas

y caracterizadas excede lo que cabe en un TFM; el coste y el camino se documentan, no se ejecutan. *Criptografía asimétrica*: un multiplicador de curva elíptica es un proyecto en sí mismo y consumiría todo el presupuesto de área; el motor proporciona claves simétricas y autenticación. *Resistencia a canales laterales*: el modelo de amenaza es un adversario lógico y remoto; las fugas de temporización se evitan por construcción, pero no se intenta ningún enmascaramiento ni evaluación basada en trazas, y se señala el propio laboratorio del instituto para ese tipo de trabajo como la continuación natural. *Certificación formal*: los criterios de AIS 20/31 y SP 800-90B y los vectores oficiales se usan en todo el trabajo, y no se reivindica ninguna validación.

1.5. Metodología y convenciones de reproducibilidad

Cuatro convenciones se aplican a toda cifra de este documento. Primera, cada número lleva la etiqueta [MEDIDO], [SIMULADO] o [ESTIMADO], y va acompañado de su incertidumbre o de la fuente de la que procede. Segunda, **la resolución del instrumento se declara antes de interpretar cualquier tendencia**, sea el instrumento un sensor, un estimador o un simulador — el suelo de ruido numérico del banco SPICE y la cuantificación de un sensor de temperatura se tratan igual. Tercera, las magnitudes se miden en lugar de inferirse siempre que exista una medida directa. Cuarta, la novedad se expresa como "no encontrado en la literatura consultada", nunca como "no existe"; una afirmación de este tipo en un borrador temprano de este trabajo tuvo que corregirse al encontrarse un artículo de 2024 que medía exactamente la magnitud en cuestión, y la corrección se deja visible en las notas de trabajo.

Todo se publica: el VHDL y sus bancos de pruebas, los modelos y el análisis en Python, el firmware, el flujo de síntesis, los vectores de prueba oficiales utilizados y los datos brutos de cada campaña, con las órdenes que reproducen cada tabla.

1.6. Estructura del documento

El Capítulo 2 revisa los generadores basados en osciladores en anillo, sus modelos estocásticos, las normas y el panorama comercial. El Capítulo 3 es la primera parte del trabajo: caracteriza el generador del ESP32 como caja negra y muestra lo que un estudio así puede y no puede establecer. El Capítulo 4 construye la cadena desde el transistor hasta la min-entropía y valida cada eslabón. El Capítulo 5 describe el motor digital y el Capítulo 6, su verificación. El Capítulo 7 sintetiza el motor sobre celdas estándar y expone el camino y el coste de un circuito integrado. El Capítulo 8 presenta la campaña en hardware y las comparaciones, y el Capítulo 9 concluye.

Capítulo 2

Estado del arte

La revisión se organiza siguiendo la cadena que construye el TFM: cómo se convierten los osciladores en anillo en bits aleatorios, cómo se modela y se acota su entropía, cómo se mide su jitter desde dentro del dispositivo, qué exigen las dos normas que rigen el campo, qué se sabe de los generadores de los microcontroladores de uso común — sobre todo del ESP32 — y de las segundas fuentes y los recolectores construidos a su alrededor, qué primitivas simétricas encajan en un motor con recursos limitados, qué ofrecen los componentes comerciales y qué cambia cuando el diseño sale de la FPGA hacia el silicio. La última sección expone el hueco que aborda este trabajo, en la forma prudente que exigen las reglas de trabajo.

2.1. Arquitecturas de TRNG basadas en osciladores en anillo

Un oscilador en anillo es una cadena impar de inversores cerrada sobre sí misma; su periodo fluctúa porque cada transición se retrasa una cantidad aleatoria, y esa fluctuación — el jitter — es la fuente de ruido de toda una familia de generadores. Lo que distingue a las arquitecturas es cómo se acumula y se muestrea el jitter [8, 9].

El **oscilador en anillo elemental** (ERO) muestrea un anillo con un flip-flop cuyo reloj es un segundo anillo dividido por K_D; el divisor fija el tiempo de acumulación. Tiene el modelo estocástico más maduro [10, 11, 12] y una medida embebida del jitter [13], a costa de una tasa de salida muy baja (K_D de 10^4 a 10^5: 80 000 en Spartan-6 y 135 000 en Cyclone V en la implementación de Petura, unos 430 000 en el diseño de Fischer y Lubicz para 0,997 bit de entropía). El generador **multianillo** de Sunar, Martin y Stinson [14] combina mediante XOR muchos anillos muestreados por un reloj de referencia; Wold y Tan [15] añadieron un flip-flop por anillo antes de la XOR y publicaron 100 Mbit/s con menos de 100 elementos lógicos. Su debilidad la pusieron de manifiesto Bochard *et al.* [16]: parte de la aleatoriedad aparente es *pseudo*aleatoriedad debida a la interacción

entre anillos de frecuencias distintas, y la independencia que supone la demostración no se cumple en una FPGA. El **muestreo coherente** (COSO) usa dos anillos de periodo casi igual y cuenta el batido [17, 18]; su modelo da la min-entropía directamente a partir de la distribución de la cuenta [19], y la calibración dinámica de anillos reconfigurables lo hace portable entre dispositivos y emplazamientos (3,30 Mbit/s en Spartan-6, 4,65 Mbit/s en Spartan-7) [20, 19]. El **anillo de efecto de transición** (TERO) cuenta cuántas veces oscila una estructura biestable antes de estabilizarse [21], con un modelo completo que va de lo físico a lo estocástico [22]; el **anillo autotemporizado** (STR) hace circular varios eventos por un anillo asíncrono para obtener fases separadas del orden del jitter [23], y alcanza 154 Mbit/s; y el **ES-TRNG** de Yang *et al.* usa la cadena de acarreo como convertidor de tiempo a digital para codificar la posición del flanco con 10 LUT y 5 flip-flops [24, 25].

La Tabla 2.1 reproduce la comparación que hicieron Petura *et al.* de los núcleos conformes a AIS sobre un mismo dispositivo [8]. La columna de entropía es una estimación estadística (el test T8 de AIS 31), no una cota del modelo; la columna de viabilidad es la valoración cualitativa de los autores, en la que COSO y TERO puntúan peor porque necesitan un ajuste manual por dispositivo. No existe una tabla comparable para Artix-7; el proyecto abierto OpenTRNG implementa ERO, multianillo y COSO en una Arty A7 con el mismo dado XC7A35T que la Basys 3, pero no publica cifras ni de entropía ni de tasa de bits [26].

Tabla 2.1: Núcleos TRNG compatibles con AIS en Xilinx Spartan-6 (45 nm), según Petura *et al.* [8], tabla II. La entropía es la estimación T8; la "viabilidad" es la clasificación de los autores (cuanto más alta, más fácil).

Núcleo	LUT/reg.	Potencia (mW)	Tasa (Mbit/s)	Entropía/bit	Viab.
ERO	46/19	2,16	0,0042	0,999	5
COSO	18/3	1,22	0,54	0,999	1
MURO	521/131	54,72	2,57	0,999	4
PLL	34/14	10,6	0,44	0,981	3
TERO	39/12	3,31	0,625	0,999	1
STR	346/256	65,9	154	0,998	2

2.2. Modelos estocásticos y cotas de entropía

La exigencia de que un generador físico vaya acompañado de un *modelo estocástico* — una descripción matemática de la fuente de ruido a partir de la cual puede derivarse la entropía de la salida — la introdujeron Killmann y Schindler [11, 27] y es obligatoria en AIS 20/31. Para los osciladores en anillo, el modelo de referencia es el de Baudet *et al.* [10]: la fase del anillo muestreado es un proceso de Wiener con deriva μ y volatilidad

σ^2, el bit muestreado es el semiperiodo en el que cae la fase, y todo depende del factor de calidad $Q = \sigma^2 \Delta t$ (varianza de fase acumulada entre muestras) y de la razón de frecuencias $\nu = \mu \Delta t$. El modelo da la probabilidad del bit condicionada a la fase anterior, la probabilidad de cualquier vector de salida y una cota inferior de la tasa de entropía de Shannon, $H \geq 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q}$, cuyo alcance exacto se examina en el Capítulo 4. Petura *et al.* la reescriben en magnitudes de diseño, $H = 1 - \frac{4}{\pi^2 \ln 2} \exp(-\pi^2 \sigma_{th}^2 K T_2 / T_1^3)$, y Fischer y Lubicz la invierten para obtener el divisor K_D a partir de un jitter medido [13]. Ma *et al.* proponen un modelo alternativo que usa el grupo de Lovaina [12]; Haddad *et al.* cuestionan la independencia mutua de las realizaciones del jitter en la que se apoyan estas cancelaciones [28]; y Saarinen advierte de que la cota de Baudet "nunca es inferior a 0,415, ni siquiera cuando Q tiende a cero" y solo es segura bajo hipótesis adicionales, y propone una cota válida en todo el intervalo [29].

Hay dos matices que importan en el diseño. Primero, no todo el jitter es entropía. Fischer y Lubicz separan la varianza de fase acumulada en un término de paseo aleatorio, lineal en el tiempo, producido por el ruido térmico (blanco), y unos términos de parpadeo ($1/f^\beta$) cuya varianza crece cuadráticamente y que dominan con tiempos de acumulación largos [13]; el jitter determinista y global que procede de la alimentación y del sustrato se comporta como estos últimos y puede ser conocido o inducido por un adversario [30]. Solo puede computarse el término lineal, y por eso no se puede hacer el divisor arbitrariamente grande para compensar una fuente débil. Segundo, toda medida embebida es *diferencial*: dos anillos que comparten alimentación ven el mismo jitter global, que se cancela en su fase relativa y deja la suma de sus jitters locales. Lubicz y Skórski formalizan este "principio de transferencia del jitter" y muestran que con dos anillos solo se obtiene la suma, mientras que con tres anillos y tres medidas por pares se recuperan las varianzas individuales; su experimento muestra además que la hipótesis de varianza lineal con el periodo falla en un factor dos en uno de los casos [31].

2.3. Medida embebida del jitter

Los osciloscopios no pueden resolver los picosegundos de jitter de un anillo dentro de un chip sin un divisor externo [32], así que los métodos útiles miden desde dentro. El **método del contador** cuenta periodos de un anillo en una ventana fija del otro; la varianza de la cuenta crece con el jitter relativo acumulado [11, 33], con un suelo de cuantificación de una cuenta y con contaminación por ruido de parpadeo en ventanas largas. El método de **Fischer y Lubicz** [13] no necesita divisor: en $K \approx 10^4$ bloques de $N \approx 100$ bits muestreados con $K_D = 1$, la fracción de pares (b_j, b_{j+M}) que difieren sigue a la fase acumulada en M periodos, y su varianza entre bloques, representada frente a M entre 200 y 1600, da el jitter por periodo con un error inferior al 5 % en simulación; en una Cyclone III midieron $\sigma = 5{,}01$ ps por periodo de 7,81 ns, y mostraron que enfriar el

chip reduce σ — lo que convierte esa misma medida en un test en línea. El **muestreo coherente** proporciona el jitter acumulado a partir de la varianza de la cuenta de batido sin hardware adicional [19]. Más recientemente, Benea *et al.* caracterizan el jitter de los anillos con la **varianza de Allan**, ajustando $\sigma_t^2(t) = a_0 + a_1 t + a_2 t^2$ para separar los términos de cuantificación, térmico y de parpadeo, y lo hacen tanto en una Artix-7 — la familia de FPGA de este trabajo — como en un circuito FD-SOI de 28 nm, con los datos y los scripts publicados [32]. Las cifras publicadas de jitter por periodo se agrupan en 2–4 ps para periodos de 4–8 ns en Spartan-6 y Cyclone V [8], 5 ps en Cyclone III con medida embebida [13], y 2–2,5 ps en Cyclone III y Virtex-5 para anillos autotemporizados [23]; los coeficientes de Benea *et al.* para Artix-7 se publican en unidades del contador, no en segundos, así que queda por medir una cifra absoluta para la XC7A35T.

2.4. Normas: AIS 20/31 v3.0 y NIST SP 800-90B

Dos documentos rigen cómo se evalúa un generador físico. La norma alemana **AIS 20/31**, en su versión 3.0 de septiembre de 2024 [5], sustituye la exigencia de 2011 de 0,997 bit de entropía de Shannon por bit interno [34] por una alternativa en la clase PTG.2: entropía de Shannon $\geq$ 0,9998 y/o min-entropía $\geq$ 0,98, junto con $P(1) \in (0{,}493;\ 0{,}507)$; la política de transición del BSI solo permite nuevas evaluaciones con la versión antigua hasta marzo de 2026 [35]. Sus requisitos son tan estructurales como numéricos: un modelo estocástico verificable es obligatorio; el test en línea debe derivarse del modelo y fallar con alta probabilidad cuando el parámetro de la fuente sale del conjunto admisible; un test de fallo total debe detectar que la entropía cae a cero, y para ello se acepta explícitamente la prueba de recuento de repeticiones de SP 800-90B; y la batería de caja negra T_{irn} incluye ahora dos de los predictores del NIST (MultiMMC y LZ78Y), una armonización deliberada [5, 36].

La norma estadounidense **SP 800-90B** [6] estima la *min-entropía* — la magnitud que cuenta frente a un adversario, $-\log_2$ de la probabilidad del valor más probable — a partir de al menos un millón de muestras en bruto y de una matriz de reinicios de 1000×1000, con el estimador del valor más común si se demuestra que la fuente es i.i.d. y, en caso contrario, con el mínimo de diez estimadores, cuatro de ellos predictores que intentan adivinar la muestra siguiente. Aprueba dos tests de salud continuos y no exige ningún otro: la prueba de recuento de repeticiones, con umbral $C = 1 + \lceil -\log_2 \alpha / H \rceil$, y la prueba de proporción adaptativa sobre una ventana de 1024 muestras para fuentes binarias (512 para las no binarias), $C = 1 + \text{CRITBINOM}(W, 2^{-H}, 1-\alpha)$, con $\alpha = 2^{-20}$ como valor recomendado. Enumera los componentes de acondicionamiento cuya salida puede declarar entropía completa — HMAC, CMAC, CBC-MAC, las funciones hash aprobadas y las dos funciones de derivación de SP 800-90A — y da la fórmula de la entropía de una salida acondicionada en función de la longitud de la entrada, la entropía de la entrada y

la anchura interna. La implementación de referencia de los estimadores es pública [37].

Las dos normas coinciden en lo que no pueden hacer las baterías estadísticas más antiguas. SP 800-22 [38] comprueba la uniformidad y la independencia de un flujo de bits; un generador determinista con entropía nula la supera por construcción, y también una fuente física débil situada detrás de un postprocesado criptográfico [29]. La decisión del NIST de 2022 de revisar la publicación lo dice expresamente: "rechazando su uso para evaluar generadores criptográficos de números aleatorios" [4]. La crítica que llevó hasta ahí es anterior: Saarinen califica la batería de "claramente obsoleta, posiblemente dañina", porque la superan generadores sin secreto alguno [39]; Iwasaki muestra que los resultados de sus tests no son mutuamente independientes y que le falta un criterio global de aceptación con fundamento [40]; y los comentarios públicos recogidos en la propia revisión del NIST insisten en la redundancia de los tests y en el mal uso de la batería como certificación [41]. Los datos empíricos lo confirman: un generador certificado de tarjeta inteligente mostró sesgos persistentes aun superando la mayoría de los tests [2, 42], y el generador del ESP32 supera esas mismas baterías con la radio apagada, cuando la propia documentación de Espressif dice que devuelve valores pseudoaleatorios [43, 44, 3]. La literatura académica sobre ese generador solo aplica SP 800-22 o Dieharder [45, 46]; no se encontró ninguna caracterización suya según SP 800-90B (Sección 2.5).

2.5. Generadores de los microcontroladores de uso común

Por qué el generador de un microcontrolador merece una sección propia lo muestra lo que ocurre cuando uno falla sobre el terreno. Heninger *et al.* recorrieron los servidores TLS y SSH de todo el espacio IPv4 y encontraron que el 0,75 % de los certificados TLS compartía su clave con otro servidor; factorizaron las claves RSA de dispositivos reales siempre que dos claves compartían un primo, y atribuyeron la causa a equipos embebidos sin pantalla — encaminadores, cortafuegos — que generaban sus claves en el primer arranque, antes de que su reserva de entropía se llenara: el *agujero de entropía en el arranque* [47]. El RFC 4086, la buena práctica del IETF sobre aleatoriedad, enuncia el principio de que la seguridad de claves, nonces y cookies descansa en valores impredecibles y de que un proceso pseudoaleatorio mal sembrado solo da "pseudoseguridad" [48]; y Tillmanns *et al.*, al hacer ingeniería inversa del firmware de los chips Bluetooth de Broadcom y Cypress, encontraron que los generadores hardware dedicados que llevan algunos superan tests avanzados, mientras que muchos dispositivos populares no tienen ninguno y recurren a un generador pseudoaleatorio inseguro [49]. Un termostato o un equipo HVAC conectado que despierta del sueño profundo y abre una sesión TLS antes de que su radio se haya asociado es el mismo patrón en miniatura.

La cadena del ESP32. En el ESP32 el camino del periférico al protocolo es corto. El cargador de arranque de segunda etapa, y la aplicación cuando lo pide mediante `bootloader_random_enable()`, ponen el ADC SAR en un modo de muestreo de ruido (`bootloader_random.c` [50]); la interfaz de entropía de mbedTLS en ESP-IDF, `mbedtls_hardware_poll()` en `esp_hardware.c`, es una envoltura fina sobre `esp_fill_random()` [51]; y la pila TLS construida encima [52] toma de esa función los aleatorios de cliente y servidor, los escalares efímeros de ECDHE, los IV y los identificadores de sesión, sin acondicionamiento propio. Sea cual sea el régimen del generador en el momento en que corre un saludo TLS, las claves de sesión lo heredan. Al medir TLS 1.3 en el ESP32, Salles y Farias encuentran una suite con Curve25519 y RSA unas cuatro veces más rápida que P-256 con ECDSA a igual seguridad [53], de modo que el saludo es donde se va el tiempo, y un generador que se bloquea o un acondicionador software lento gravan el momento más caro de la conexión. El cargador de arranque es también lo que siembra el generador antes de que haya radio: a la pregunta de cuándo cuenta el subsistema de RF como "habilitado" para que `esp_random()` sea verdaderamente aleatorio — si basta con inicializar una pila Bluetooth, y qué casos saturan el ADC de alta velocidad con el Wi-Fi encendido —, un responsable de Espressif contestó que el arranque de la radio habilita el ADC de alta velocidad y que el cargador de arranque siembra antes la entropía del generador, y la documentación se reescribió a raíz de ello [54]. El cargador de arranque anuncia en su registro la conmutación de esa fuente ("Enabling/Disabling RNG early entropy source"), lo que también ha desconcertado a usuarios [55]. La frontera entre el funcionamiento verdaderamente aleatorio y el pseudoaleatorio es, en suma, sutil incluso para los propios usuarios del fabricante; la prueba de reinicios de la Sección 3.6 mide cuánto aporta el ruido del arranque.

El ESP32 original y sus sucesores. Las guías de los chips posteriores, ESP32-S2, -C3 y -H2, documentan una fuente de entropía secundaria que el ESP32 original no tiene: el muestreo de un oscilador interno asíncrono de 8 MHz, siempre habilitado y mezclado por hardware en el estado del generador, que por sí solo "was sufficient to pass the Dieharder random number test suite", aunque la salida verdaderamente aleatoria solo se garantiza con la fuente principal también habilitada [56]. La guía del ESP32 original no menciona ninguna fuente así [43]. Los resultados obtenidos en un chip posterior no se trasladan, por tanto, al original, y la advertencia de "pseudo-random only" pesa más sobre él.

Miradas anteriores al generador del ESP32. El trabajo revisado por pares más cercano, de Hidayatulloh *et al.*, usa `esp_random()` para los nonces de AES en modo contador sobre enlaces LoRa punto a punto e informa de que los números superan la batería del NIST, en una sola condición de funcionamiento y sin las proporciones de aprobados por test ni la uniformidad de los valores p [46]. Castillo *et al.* presentan un entorno web

automatizado para SP 800-22 y lo validan con 2 GB de un ESP32-C3, que lleva la fuente secundaria; la aportación es la herramienta, no el dispositivo [45]. La validación del propio Espressif es la única pasada de Dieharder en condición favorable de la Sección 3.1. El resto es informal: Cheich dibuja histogramas de hasta unos 300 000 valores con el Wi-Fi encendido y apagado y entre reinicios, y concluye que la salida parece aleatoria, sin batería formal [57], y el tutorial de AndroidCrypto reproduce la advertencia de Espressif y enseña a llamar a `bootloader_random_enable()`, sin caracterización alguna [58]. Blanco-Romero *et al.* señalan que la salida pseudoaleatoria con la radio apagada supera los cribados estadísticos [3]. No se encontró en la literatura consultada una caracterización del ESP32 original que varíe las condiciones de funcionamiento y estime la min-entropía de la salida.

Otros microcontroladores. La Tabla 2.2 sitúa el ESP32 junto a los generadores de otros componentes de la misma clase. STMicroelectronics documenta en la nota de aplicación AN4230 una validación del generador de los STM32 frente a SP 800-22 y SP 800-90B e informa de su evaluación frente a AIS-31 PTG.2, la caracterización pública de fabricante más completa que se encontró [59]; aun así, Ngo y Dubrova recuperan el peso de Hamming de sus palabras de salida a partir del consumo con un modelo de aprendizaje profundo, con más de un 60 % de acierto frente a un 14 % de referencia: la fuga por canal lateral es una amenaza distinta de la debilidad estadística [60]. Nordic documenta para el nRF52840 un generador basado en el ruido térmico interno con un corrector de sesgo hardware opcional [61]. El RP2040 expone un bit de un oscilador en anillo, `RANDOMBIT`, del que su hoja de datos dice que no cumple los requisitos de los sistemas de seguridad [62, 63]. Los procesadores de aplicación de Broadcom de la Raspberry Pi exponen un generador cerrado como `/dev/hwrng`, que las evaluaciones de la comunidad con ENT, `rngtest` y Dieharder dan por bueno a cerca de 1 Mbit/s [64]; con el diseño y cualquier filtrado sin publicar, no se puede construir un modelo estocástico y "supera los tests" no se puede convertir en una cifra de entropía. El ESP32 queda entre los diseños documentados en FPGA de la Sección 2.1 y estas cajas negras: sus fuentes están nombradas, no cuantificadas.

2.6. Segundas fuentes y recolección de entropía

Un nodo que mantiene la radio apagada necesita entropía de otra parte, y dos líneas de trabajo dicen dónde encontrarla y cómo combinarla.

El estado de arranque de la SRAM. Una celda SRAM de seis transistores es un par de inversores acoplados; al encenderse, ambos parten cerca de su punto metaestable y la celda se resuelve hacia un estado u otro. En las celdas cuyos transistores están muy desapareados, el resultado lo fija la fabricación y se repite en cada encendido: una

Tabla 2.2: Evaluaciones publicadas de los generadores hardware de componentes de clase IoT. “Condiciones” indica si se varió el estado de funcionamiento.

Plataforma	Fuente declarada	Evaluación	Condiciones	Resultado	Ref.
ESP32	Ruido de los ADC de RF y SAR, relojes asíncronos	Dieharder, 2 GB (fabricante)	Solo Wi-Fi encendido	Supera; datos no públicos	[44]
ESP32	`esp_random()`	SP 800-22	Una	Supera (nonces LoRa)	[46]
ESP32-C3	Como el ESP32, más la secundaria de 8 MHz	SP 800-22, 2 GB	Una	Supera (objeto: el entorno)	[45]
ESP32 (blog)	`esp_random()`	Histogramas	Wi-Fi sí/no, reinicios	“Parece aleatorio”	[57]
STM32	Ruido analógico, dedicado	SP 800-22, SP 800-90B; AIS-31 PTG.2	No se indica	Supera (fabricante)	[59]
nRF52840	Ruido térmico, corrector de sesgo	Ninguna publicada	—	—	[61]
RP2040	Bit de un oscilador en anillo	Análisis de la comunidad	—	No apto para criptografía (hoja de datos)	[62]
BCM2835	Cerrada	ENT, `rngtest`, Dieharder	No	Supera	[64]
Bluetooth de Broadcom/ Cypress	Generador en el chip, si lo hay	Baterías avanzadas, ingeniería inversa	Muchos dispositivos	Generador bueno; alternativa insegura	[49]

huella propia del dispositivo, la base de la PUF de SRAM. En las celdas muy apareadas decide cada vez el ruido térmico, y esas celdas son una fuente de entropía. El mismo patrón se lee así de dos maneras, las celdas estables como identidad y las inestables como aleatoriedad, como mostraron Holcomb, Burleson y Fu con FERNS, al extraer de 512 bytes de SRAM en el encendido 128 bits aleatorios que superan los tests de rachas, de entropía aproximada y de frecuencia por bloques del NIST [65]. Guajardo *et al.* dieron la primera PUF de SRAM intrínseca en FPGA y la metodología de distancias de Hamming intra e interdispositivo que hoy es la habitual para separar la huella del ruido [66], y van der Leest *et al.* cuantificaron la min-entropía del ruido de los patrones de arranque de SRAM comerciales y lo acondicionaron en semillas de 256 bits con entropía completa [67]. El patrón se describe por su uniformidad (fracción de unos, idealmente la mitad), su unicidad (distancia entre dispositivos, idealmente la mitad de la longitud) y su fiabilidad, cuyo complemento, la tasa de bits erróneos intradispositivo entre encendidos de un mismo chip, se sitúa en un intervalo de un 3 a un 8 % aproximadamente en condiciones nominales [65, 67]. El uso como identidad corrige esos errores con un extractor difuso y datos de ayuda públicos; el uso como entropía los busca, y debe recordar que la mayoría estable la conoce cualquiera que haya leído el patrón una vez. Para el ESP32 no se encontró ninguna estimación revisada por pares de la min-entropía de arranque. La biblioteca abierta `esp32_puflib` implementa una PUF de SRAM para la familia como componente de ESP-IDF, lee la memoria desde un stub de despertar del sueño profundo y guarda los datos de ayuda en la flash [68]; y un hilo del foro informa de que la ROM de arranque ya ha escrito los primeros 0x1320 bytes de SRAM1 y 0x6E0 bytes de SRAM2 cuando corre el código de la aplicación [69], de modo que quien lea debe usar regiones que la ROM no toca. Las dos fuentes de un chip son complementarias, no redundantes: el ruido del generador es continuo pero necesita sus inyectores alimentados, mientras que el patrón de la SRAM es de un solo uso, gratuito en el encendido, solo se renueva con un ciclo de alimentación y está dominado por su huella fija.

Acondicionamiento y recolección. El lema del hash residual de Impagliazzo, Levin y Luby acota cuánto se acerca a la uniformidad un hash 2-universal de una fuente de min-entropía conocida, pero necesita una semilla uniforme [70], que un dispositivo escaso de entropía no tiene. En la práctica se usa una función criptográfica fija y su solidez se argumenta en términos computacionales: Dodis *et al.* analizan cuándo las construcciones CBC-MAC, en cascada y HMAC actúan como extractores de aleatoriedad [71], SP 800-90B las admite como acondicionadores aprobados (Sección 2.4), y HKDF divide la derivación de claves en un paso de extracción, que destila una clave pseudoaleatoria a partir de una entrada no uniforme, y un paso de expansión, que la estira [72]. Fusionar varias fuentes débiles tiene precedentes en dispositivos con recursos limitados. Pawlowski, Jara y Ogorzalek recolectan los bits menos significativos de sensores de temperatura,

humedad y luz en nodos IoT, y los de luz dan unos 7,9 bits por byte tras el ajuste a razón de 1,3 ms por byte, aproximadamente [73]; Zhang y Wu construyen una reserva de entropía que fusiona circuitos caóticos y sucesos del entorno en redes multisensor [74]; Foreman, Yeung y Curchod muestran cómo los extractores (Toeplitz, Trevisan) elevan la calidad medida de generadores comerciales con baterías ampliadas [75]; y la revisión de Kietzmann, Schmidt y Wählisch recorre las fuentes de radio, sensores, SRAM y jitter, los generadores y las pautas de diseño para el IoT con recursos limitados, y trata allí la escasez de entropía como un problema estructural [76]. El `pico_rand` del SDK del RP2040 es un caso real en silicio: como el bit del oscilador en anillo está autocorrelado cuando se muestrea deprisa, mezcla muestras del anillo (a lo sumo unos 100 kbit/s, separadas 10 μs), el tiempo y contadores de bus mediante un hash [77]. A todos ellos se les aplican dos cautelas. La min-entropía se suma entre fuentes solo si son independientes, y fuentes que comparten dado, alimentación y temperatura no tienen por qué serlo; lo que garantiza la concatenación seguida de un hash sin esa hipótesis es solo la min-entropía de la mejor fuente. Y una salida resumida con un hash parece uniforme entre lo que entre, así que las baterías estadísticas sobre ella validan la construcción, no la entropía; el recolector de la Sección 3.5 se lee a esa luz.

2.7. Núcleos simétricos para hardware con recursos limitados

El motor necesita un cifrador de bloque y, si lo usa, una función hash. Para AES-128 (diez rondas [78]), el espacio de diseño va desde cauces segmentados totalmente desenrollados, orientados a la tasa de bits, hasta rutas de datos de 8 bits, orientadas al área: Good y Benaissa publican 124 slices y dos block RAM a 2,2 Mbit/s en una Spartan-II en un extremo y 25 Gbit/s en el otro [79]; el diseño compacto de Chodowiec y Gaj usa 222 slices y tres block RAM [80]; y las realizaciones ASIC más pequeñas llegan a 2,4 kGE con una ruta de datos de 8 bits a 226 ciclos por bloque [81], en la línea de las S-box en cuerpo compuesto de Satoh [82], Canright [83] y Boyar–Peralta [84]. Las construcciones en cuerpo compuesto optimizan el número de puertas en ASIC y pierden su ventaja en una LUT de FPGA, donde una tabla directa es a la vez más rápida y más sencilla. Existen diseños de referencia abiertos con cifras publicadas para Artix-7: un cifrador de una ronda por ciclo con 1104 LUT a 294 MHz [85], y un cifrador/descifrador iterativo de 32 bits con 2298 slices a 97 MHz [86]. Para un motor cuya salida sale por un enlace I2C, cualquiera de ellos es mucho más rápido que el enlace, y la elección la dictan la verificabilidad y el área, no la velocidad.

SHA-256 [87], en su forma iterativa, cuesta unos 750 slices a 108 MHz en Artix-7 [88], lo que es comparable a un AES compacto; su papel en un generador sería el

de acondicionador aprobado de SP 800-90B. Los generadores deterministas de SP 800-90A [89] — Hash_DRBG, HMAC_DRBG y CTR_DRBG — se diferencian sobre todo en el coste: CTR_DRBG con AES-128 mantiene un estado de 256 bits más un contador y produce 128 bits por cada cifrado de bloque, Hash_DRBG necesita sumas modulares de 440 bits, y HMAC_DRBG, dos compresiones de hash por bloque de salida. Sin función de derivación, CTR_DRBG requiere una entrada con entropía completa de exactamente la longitud de la semilla; con ella, que es la configuración habitual en la práctica, acepta entradas arbitrarias. El análisis de Woodage y Shumow advierte de que la flexibilidad de la norma permite elecciones que empeoran la fuga del estado, y recomienda actualizar tras cada operación de generación y resembrar muy por debajo del límite de 2^{48} [90]; SP 800-90C, definitiva desde 2025, establece el marco en el que se combinan una fuente física y un DRBG [91]. Para la autenticación, AES-CMAC (SP 800-38B [92], RFC 4493 [93]) cuesta un cifrado de bloque por cada bloque de mensaje y es en sí mismo un acondicionador aprobado. Los vectores oficiales de todos ellos son públicos [94, 95, 96, 97].

2.8. Elementos seguros comerciales

La categoría de producto a la que pertenece este motor existe. Un elemento seguro es un pequeño circuito integrado que genera y almacena claves y realiza operaciones simétricas y asimétricas para un microcontrolador anfitrión, casi siempre por I2C. Cuatro componentes dominan el segmento de bajo coste: el ATECC608B de Microchip, el SE050 de NXP, el OPTIGA Trust M de Infineon y el STSAFE-A110 de ST [7, 98, 99, 100]. El ATECC608B, a 0,90 $ por unidad suelta [101], ofrece AES-128, SHA-256/HMAC, ECDSA/ECDH sobre P-256, dieciséis ranuras de clave y una interfaz I2C, y su fuente de entropía cuenta con una validación del NIST según SP 800-90B, obtenida precisamente con osciladores en anillo [102]. Dos de sus competidores no hacen ninguna referencia a SP 800-90 en sus hojas de datos; el tercero afirma que su generador está “diseñado para cumplir” la norma. La lección para este trabajo es dónde puede competir un diseño académico y dónde no: no en precio, donde un componente certificado cuesta menos de un euro, sino en la transparencia de la entropía, que es justo lo que un componente de caja negra no puede ofrecer y lo que las normas exigen ahora.

2.9. De la FPGA al ASIC: qué cambia en la fuente de entropía

La lógica digital se traslada de una FPGA a celdas estándar con una penalización conocida — Kuon y Rose midieron 40× en área, 3,2× en retardo y 12× en potencia dinámica para diseños solo de lógica en 90 nm, y 27–49× en área para AES en concreto [103];

el oscilador en anillo no se traslada en absoluto, porque en silicio se construye con celdas inversoras cuyo ruido, variabilidad y acoplamiento difieren de los de una cadena de LUT. Tres resultados publicados definen lo que debe añadir una implementación en silicio. Benea *et al.* muestran que el ruido de parpadeo, cuyo peso crece en los nodos más recientes, cambia el carácter del jitter y, contra lo esperado, que tratarlo como fuente legítima puede cuadruplicar la tasa de bits a igual entropía [32]. Yang, Blaauw y Sylvester muestran que la variación de proceso — mayor en 40 nm que en 180 nm — debe absorberse con un anillo configurable y un lazo de ajuste para que el generador sobreviva entre -40 y 120 °C y entre 0,6 y 0,9 V [104]. Y dos ataques muestran por qué un dispositivo de seguridad no puede ignorar su entorno: una frecuencia inyectada en la alimentación engancha los anillos y eliminó el jitter de un microcontrolador seguro comercial, con lo que su espacio de claves pasó de 2^{64} a 3300 [1], y unos pocos cientos de microvatios de campo radiado sesgaron un generador de cincuenta anillos hasta que falló todos los tests [105]; la defensa publicada combina el filtrado de la alimentación con el mismo lazo de calibración [104], y el envejecimiento del dispositivo por inestabilidad por polarización y temperatura y por inyección de portadores calientes degrada con los años los generadores mal temporizados, mientras deja intactos los bien temporizados.

El acceso al silicio es más abierto de lo que parece. La Universidad de Sevilla y el IMSE-CNM son miembros de Europractice [106], cuya lista de precios académicos es pública: un bloque mini@sic cuesta 3 430 € en UMC 180 nm y 3 691 € en TSMC 65 nm, y el SG13G2 de IHP se ofrece a 5 110 €/mm^2, con una tirada gratuita para diseños de código abierto de menos de 2 mm^2 [107, 108, 109]. El área de seguridad hardware del instituto ya ha llevado primitivas de osciladores en anillo desde FPGA Xilinx de la serie 7, evaluadas con SP 800-22 y SP 800-90B, hasta silicio de 65 nm caracterizado [110, 111, 112, 113], y ha fabricado antes circuitos criptográficos [114]. Las bibliotecas abiertas de celdas estándar que hacen posible un capítulo de síntesis — IHP SG13G2, SkyWater SKY130, GlobalFoundries GF180MCU y la predictiva Nangate 45 nm — vienen con vistas liberty, LEF y GDS, y la literatura compara este tipo de diseños en puertas equivalentes normalizadas a la NAND2 mínima de la biblioteca [115, 116, 117, 118, 119], con la advertencia de que el área de síntesis lógica es una cota inferior que crece en un factor de 1,7 a 4,6 al pasar al área de núcleo tras el rutado.

2.10. Hueco que aborda este trabajo

No se reivindica una novedad absoluta. El ERO y su modelo, la medida embebida del jitter, el estudio comparativo de núcleos conformes a AIS, el COSO calibrado en dispositivos de la serie 7 y una implementación abierta sobre el mismo dado de la Basys 3 existen todos, y el propio grupo del instituto ha pasado de las FPGA de la serie 7 al silicio de 65 nm con primitivas de osciladores en anillo; existen también la entropía del

arranque de la SRAM y recolectores de varias fuentes con un acondicionador criptográfico, en el RP2040 entre otros. Lo que *no se encontró* en la literatura consultada, y lo que este trabajo se propone aportar de forma modesta pero verificable, es lo siguiente.

1. Una cifra de jitter por periodo para la Artix-7 XC7A35T en unidades absolutas, obtenida dentro del dispositivo con un método diferencial, con las componentes térmica y de parpadeo separadas y con su incertidumbre declarada, dado que los coeficientes publicados para Artix-7 están en unidades del contador.

2. La cadena completa *jitter medido* → *factor de calidad* → *cota de min-entropía* → *divisor* → *umbrales de los tests de salud*, aplicada a una plataforma y contrastada con los diez estimadores no i.i.d. de SP 800-90B y con los requisitos PTG.2 de AIS 20/31 *versión 3.0*, que la mayor parte de la literatura anterior no pudo usar.

3. Una caracterización según SP 800-90B del generador del ESP32 en sus estados de funcionamiento, y una comparación controlada, con la misma metodología, entre una fuente de caja blanca en FPGA y una comercial de caja negra.

4. El uso de un microcontrolador de uso común, con sus propios aceleradores criptográficos, como oráculo independiente de un motor en FPGA, sobre un bus real, con pruebas de respuesta conocida, de interoperabilidad y de reto-respuesta.

5. Un motor de generación de claves de una sola primitiva — AES para el acondicionamiento, la generación y la autenticación — escrito como RTL portable, verificado contra vectores oficiales, sintetizado sobre una biblioteca abierta de 130 nm con el resultado expresado como cota inferior, y presupuestado para una lanzadera académica.

Cada punto se formula como “no encontrado” y no como “no existe”. Una afirmación anterior de este tipo en las notas de trabajo — que no se había publicado ninguna medida del jitter en Artix-7 — tuvo que corregirse cuando se encontró el artículo de Benea *et al.*, y la corrección se mantiene visible allí.

Capítulo 3

El generador del ESP32 como caja negra

Este capítulo es la primera parte del trabajo. Trata el generador hardware de números aleatorios del ESP32 tal como se lo encuentra un integrador: un registro que devuelve palabras de 32 bits, detrás del cual el fabricante nombra dos fuentes de ruido y no cuantifica ninguna. El generador se hace funcionar en cuatro condiciones de operación en cuatro campañas de captura sobre tres chips; su salida pasa por una batería SP 800-22 validada con una autocomprobación, por métricas de Shannon y espectrales y por los estimadores de SP 800-90B; se construye a su alrededor un recolector de claves; y una prueba de reinicios abre la caja hasta donde puede abrirse desde fuera. Toda cifra está medida [MEDIDO] en silicio salvo que se diga que es calculada. El capítulo termina explicando por qué ninguna de estas medidas acota la entropía del generador, que es la razón del resto de este TFM.

3.1. El periférico y lo que documenta Espressif

El dispositivo es el ESP32 original, un sistema en chip Xtensa LX6 de dos núcleos; los tres chips medidos aquí son ESP32-D0WD-V3, revisión de silicio 3.1. Según el manual técnico de referencia [44], cada palabra de 32 bits leída de `RNG_DATA_REG` (dirección 0x3FF75144) es un número verdaderamente aleatorio, generado a partir del ruido térmico y del desajuste entre relojes asíncronos; el ruido procede del ADC de alta velocidad de la radio, del SAR ADC o de ambos, y entra en el generador a través de una puerta XOR. El SAR ADC aporta 2 bits por ciclo del `RC_FAST_CLK` de 8 MHz, así que el registro debería leerse a no más de 500 kHz; el ADC de alta velocidad aporta 2 bits por ciclo del reloj APB de 80 MHz, lo que permite 5 MHz. Con ninguno de los dos convertidores habilitado, según el manual, se devuelven números pseudoaleatorios. La guía de software [43] lo expresa en términos operativos: la salida es verdaderamente aleatoria mientras el

Wi-Fi o el Bluetooth están habilitados, mientras está activa la fuente SAR que enciende `bootloader_random_enable()` — que debe apagarse antes de que la aplicación use el ADC, el I2S o la radio — o mientras se ejecuta el bootloader de segunda etapa, y en otro caso "debe considerarse solo pseudoaleatoria". Las aplicaciones leen el generador mediante `esp_random()`, que hace una espera activa entre palabras para que pueda entrar ruido nuevo, o mediante `esp_fill_random()`, que llena un búfer con él. En el ESP32 la espera es de 16 ciclos de APB, un techo de 5 millones de palabras por segundo (`hw_random.c`, sin cambios en este punto desde ESP-IDF 5.2.1 hasta 5.5.2, entre las que queda la 5.3.2 usada aquí); los 15–75 kHz que cita la guía corresponden a las esperas más largas programadas para chips posteriores.

La única validación que publica Espressif es una muestra de 2 GB, leída a 5 MHz con solo el ADC de alta velocidad habilitado, que superó la batería Dieharder 3.31.1 [44]; los datos no son públicos, y la muestra cubre solo la condición favorable. No se publica nada sobre el estado interno con el que se mezcla el ruido, ni ningún modelo del ruido ni cifra de min-entropía, y la bibliografía consultada solo aplica al generador SP 800-22 o Dieharder (Sección 2.4). Para un integrador es una caja negra con dos entradas documentadas que pueden encenderse y apagarse, y así es como se mide aquí.

3.2. Diseño experimental y captura

Las dos fuentes documentadas se cruzan como dos factores binarios, lo que da cuatro condiciones: A, Wi-Fi y ADC; B, solo Wi-Fi; C, solo ADC; D, ninguna. "Wi-Fi encendido" significa la radio arrancada en modo estación con `esp_wifi_start()`, sin asociarse a ninguna red, ya que la documentación liga el ruido a que el subsistema de RF esté habilitado, no al tráfico. "ADC encendido" significa una tarea en segundo plano que mantiene el SAR ADC convirtiendo en el canal 0 del ADC1 (GPIO36, flotante) tan rápido como permite el controlador de conversión única. Esa no es la fuente SAR documentada, el modo de muestreo de ruido de `bootloader_random_enable()`, y Espressif no dice si las conversiones ordinarias llegan al generador; la estadística que sigue no puede decidirlo, y solo la prueba de reinicios de la Sección 3.6 muestra que sí llegan, sin decir cuánto aportan. Las condiciones se ejecutan en el orden D, C, B, A desde un arranque en frío: D primero, porque una vez que la radio ha funcionado, un estado con la radio apagada ya no es el de un dispositivo cuya radio nunca se ha encendido, y cada paso posterior añade una fuente. El firmware espera 500 ms tras cada cambio, y 200 ms más tras arrancar la radio, para que cada secuencia pertenezca a una única condición estacionaria.

Cada condición produce 125 000 bytes, 10^6 bits, la longitud mínima que SP 800-22 recomienda para su batería completa [38] y el número mínimo de muestras de SP 800-90B [6]. La secuencia se genera en RAM con `esp_fill_random()` mientras se mantiene la condición y solo entonces se transmite, con la tarea del ADC detenida, porque se

comprobó que la actividad del ADC durante ráfagas serie largas las corrompía; antes de ello, el firmware cronometra la generación de 64 KiB, y lee la temperatura del chip antes y después. El flujo es hexadecimal, en tramas que llevan la condición, la repetición, la temperatura, la tasa y un CRC-32 calculado en el dispositivo; el anfitrión solo acepta una secuencia si el CRC coincide y la longitud es exactamente 125 000 bytes. La consola funciona a 115 200 baudios, la única velocidad a la que la salida de la aplicación llegaba de forma fiable (unos 25 s por condición), con el registro de mensajes en nivel `ERROR` porque los mensajes del controlador de Wi-Fi rompían tramas. Antes del ciclo, `app_main` copia un búfer de 4 KiB de la sección sin inicializar `.noinit`, el patrón de SRAM de la Sección 3.5. El firmware está en `firmware/esp32_rng`, y las herramientas, en `analysis/esp32_rng` y `analysis/campana_rng.py`.

El firmware hace funcionar la CPU a 240 MHz, para que las tasas de la Sección 3.4 se refieran a un reloj fijo, con el tick de FreeRTOS a 1 kHz. Tras la copia de la SRAM, `app_main` imprime la dirección MAC y la versión de ESP-IDF y arranca una única tarea de campaña fijada al núcleo 1. La tarea del ADC de las condiciones A y C funciona en el núcleo 0, con menor prioridad, junto a la tarea del controlador de Wi-Fi, que ESP-IDF coloca en el núcleo 0 por defecto (Figura 3.1). La tarea de campaña ejecuta las cuatro condiciones, imprime 100 semillas del recolector y, tras 2 s, empieza el ciclo siguiente, indefinidamente. Todo artefacto sale del dispositivo como una trama entre una línea `BEGIN` y una `END` —`PUF` para el patrón de SRAM, `RNG` para una secuencia, `SEEDS` para las semillas y `CYCLE` alrededor de un ciclo entero—, con sus metadatos como pares clave–valor en la cabecera, una carga útil de 64 bytes por línea hexadecimal y el CRC-32 de los bytes en bruto, de `esp_rom_crc32_le`, en la línea de cierre. Una secuencia se transmite en trozos de 4 KiB, cediendo un tick del planificador entre trozos al perro guardián y al controlador de la UART, tras una pausa de 100 ms una vez detenida la tarea del ADC.

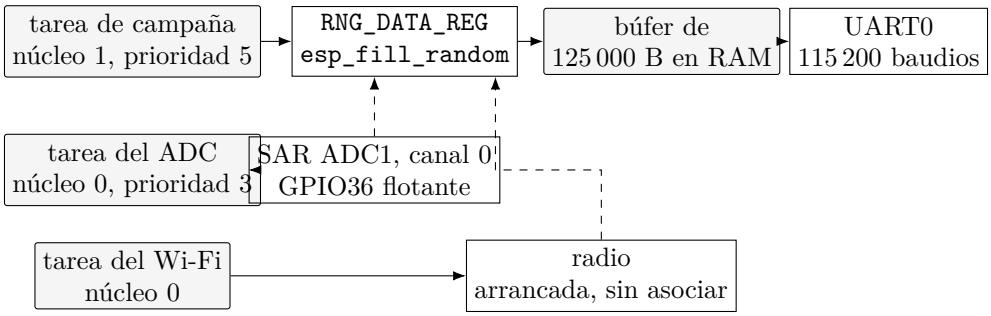


Figura 3.1: Tareas y núcleos del firmware de captura. La tarea de campaña lee el generador a RAM mientras se mantiene la condición; la tarea del ADC (condiciones A y C) y el controlador de Wi-Fi (A y B) funcionan en el otro núcleo. Las flechas discontinuas son los caminos por los que las fuentes documentadas pueden llegar al generador. La tarea del ADC se detiene antes de transmitir el búfer.

La velocidad serie no fue la primera elección. A 460 800 y 921 600 baudios seguía llegando el mensaje inicial del cargador de arranque de segunda etapa, a la velocidad por

defecto de la ROM, pero la salida de la aplicación no llegaba nunca; no se investigó la causa, y se mantuvieron 115 200 baudios. A esa velocidad los 250 000 caracteres hexadecimales de una secuencia necesitan unos 22 s (calculado, diez bits de línea por carácter, sin contar los saltos de línea ni las líneas de trama), frente a los unos 25 s observados por condición. En el anfitrión (`analysis/esp32_rng/capture_rng.py`), una pequeña máquina de estados recoge las líneas hexadecimales entre las marcas e ignora todo lo demás fuera de un bloque; antes de decodificar elimina todo carácter que no sea hexadecimal, como dejaría un fragmento suelto de un mensaje de registro, y descarta un medio byte final impar, de modo que una línea intercalada acaba en un rechazo del CRC y no en un fallo de decodificación. En el banco apareció un fallo más: ante picos de corriente, sobre todo con la radio y el ADC activos a la vez, la placa se reiniciaba y su puente USB–serie, un CP2102, volvía a enumerarse, lo que invalidaba el puerto abierto en mitad de una lectura. Se atribuyó a una caída de tensión (*brown-out*) y no se midió. La herramienta de captura descarta entonces el bloque en curso, reabre el puerto hasta que reaparece y sigue; y numera ella misma las repeticiones, por el número de secuencias válidas que ya tiene de cada condición, porque el contador del firmware vuelve a empezar con la placa. Una reconexión puede, por tanto, perder un bloque, pero nunca duplicarlo ni sobrescribirlo.

El firmware contiene además un camino opcional de telemetría, `mqtt_reporter`, implementado y nunca activado. Un menú de configuración (`Kconfig.projbuild`) recoge el nombre de la red, su contraseña y la URI del broker, para que ninguna credencial se compile desde el fuente; tras el primer ciclo, el firmware intenta asociarse y conectarse solo si el nombre de la red difiere de su valor de relleno, y si no, avisa de que MQTT está desactivado y sigue por el enlace serie. Activo, publicaría la condición, la temperatura del chip y la tasa en los temas `esp32/rng/condition`, `esp32/rng/temperature` y `esp32/rng/throughput`, y la última semilla de cada ciclo en `esp32/harvester/seed`. Ninguna campaña configuró las credenciales, así que la radio nunca se asoció, como exigen las condiciones. Activarlo habría cambiado los ciclos posteriores, con una radio asociada, y la URI del broker por defecto es `mqtt://` sin cifrar, así que la semilla habría cruzado la red en claro: tal como está escrito es un camino de telemetría para el banco, no para material de claves. Las secuencias nunca estuvieron pensadas para él, ya que las condiciones C y D necesitan la radio apagada.

Tabla 3.1: Las cuatro campañas de captura [MEDIDO]. Cada ciclo es un arranque en frío seguido de las cuatro condiciones en el orden D, C, B, A, de 10^6 bits cada una.

Campaña	Fecha	Chip (MAC)	Firmware compilado con	Ciclos
Primera	2026-08-04	ec:e3:34:9a:d1:18	ESP-IDF 5.3	1
Segunda	2026-09-19	ec:e3:34:9a:d1:18	ESP-IDF 5.3.2	2
Tercera	2026-10-12	a0:dd:6c:0e:fb:4c	ESP-IDF 5.3.2	3
Cuarta	2026-10-20/21	f8:b3:b7:2a:09:28	ESP-IDF 5.3.2	3 + 3

Se hicieron cuatro campañas (Tabla 3.1). La primera capturó un ciclo y los patrones

de SRAM en el chip de dirección MAC ec:e3:34:9a:d1:18, que el firmware notifica en el arranque y que se anotó con los resultados; su registro de captura no tiene columna de MAC. Después, el mismo firmware, recompilado sin cambios, ejecutó dos ciclos más en ese chip, tres en un segundo chip, a0:dd:6c:0e:fb:4c, y dos tandas de tres en un tercero, f8:b3:b7:2a:09:28, la primera con el mismo binario exacto usado en el segundo chip. En el tercer chip el enlace serie con el anfitrión estropeó de vez en cuando un bloque, del lado del anfitrión; el CRC rechazó los tres bloques así, dos se tomaron de nuevo del ciclo siguiente del mismo arranque, y un ciclo incompleto se apartó y se repitió. Todas las secuencias conservadas tienen la longitud y el CRC correctos. Los datos están en `results/esp32_rng_agosto` y en `results/esp32_campana_rng`, con un subdirectorio por chip, cada uno con un registro de las temperaturas, las tasas y los veredictos del CRC. La batería SP 800-22 de primer nivel, las métricas de Shannon y el recolector de las tres secciones siguientes usan la primera campaña; el análisis de segundo nivel de SP 800-22 usa las cuatro, y los estimadores de SP 800-90B de la Sección 3.6, todas menos la segunda tanda de la cuarta, que se reserva para una comprobación.

El firmware no se detiene tras un ciclo: vuelve a la condición D con la radio parada pero habiendo funcionado, y el primer diseño pretendía comparar esas D posteriores con la del arranque en frío, para ver si el estado residual de la radio cambia la salida. No se capturó ninguna. Todas las secuencias de la condición D conservadas, en las cuatro campañas, vienen del primer ciclo tras un arranque en frío; los dos bloques tomados de un segundo ciclo en el tercer chip eran de la condición A. La prueba de reinicios de la Sección 3.6 responde a la pregunta más directamente de lo que habría podido una D posterior: una vez que la radio ha funcionado, el generador ha recibido ruido (Tabla 3.5), así que una D posterior continúa desde un estado que ya no es el fijo, y la estadística que sigue, que no separa D de A, tampoco habría separado las dos D.

3.3. La batería SP 800-22

Las quince pruebas de SP 800-22 Rev. 1a [38] se implementaron a partir de las definiciones de la norma en lugar de tomarse de una biblioteca (`analysis/esp32_rng/nist_sp800_22.py`), tomando los bits del más significativo al menos significativo y decidiendo cada prueba con $\alpha = 0{,}01$. Las cinco pruebas que producen varios valores p — plantillas sin solape, serie, sumas acumuladas, excursiones aleatorias y su variante — conservan el mínimo, de modo que una prueba solo pasa si pasan todas sus partes; la prueba de plantillas sin solape usa las 8 primeras de las 148 plantillas de longitud 9 que enumera la norma. Una autocomprobación ejecuta la batería sobre tres entradas de 125 000 bytes: los datos de `os.urandom` deben pasar con un fallo como máximo, un búfer todo a ceros debe fallar, y el patrón alternado `0x55` debe ser rechazado por la prueba de rachas o por la de entropía aproximada. La supera: los ceros fallan las quince pruebas, y el patrón alternado

falla todas salvo las de frecuencia, frecuencia por bloques y sumas acumuladas, que una secuencia perfectamente equilibrada tiene que pasar.

La longitud de 10^6 bits es también la que permite que cada prueba funcione en el régimen que necesitan sus aproximaciones (calculado a partir de los parámetros de la norma). La prueba de rango de matrices usa matrices de 32×32, de 1024 bits, y necesita al menos 38 para su χ^2 de tres clases; 10^6 bits dan 976. La prueba universal de Maurer alcanza su configuración de $L = 7$, con $Q = 1280$ bloques de inicialización, solo a partir de 904 960 bits. Las dos pruebas de excursiones aleatorias solo se aplican cuando el paseo ± 1 de la secuencia vuelve a cero al menos $J = 500$ veces. Para un paseo simétrico de n pasos $E[J] \approx \sqrt{2n/\pi}$, así que el umbral pide en promedio unos $3{,}9 \times 10^5$ bits, y con 10^6 bits $E[J] \approx 800$. La dispersión es grande: $J/\sqrt{n}$ tiende al valor absoluto de una variable normal estándar, así que una fuente perfecta de 10^6 bits se queda por debajo de 500 retornos con probabilidad 0,38. La norma considera entonces no aplicables las pruebas de excursiones; la implementación de este trabajo registra el caso como un fallo con una nota, de modo que un paseo corto no puede pasar inadvertido, y las cuatro secuencias de la primera campaña tuvieron retornos suficientes. La misma dispersión explica que solo 34 de las 59 secuencias del análisis de segundo nivel de más abajo entren en esas pruebas.

Sobre las cuatro secuencias de la primera campaña la batería pasa las quince pruebas en las cuatro condiciones (Figura 3.2). La longitud limita lo que eso puede significar. Una fuente perfecta falla al menos una de quince pruebas con probabilidad $1 - 0{,}99^{15} \approx 0{,}14$ (calculado), así que solo un fallo que se reprodujera indicaría un defecto, y con una secuencia por condición el análisis de segundo nivel de la norma — la proporción de secuencias que pasan y la uniformidad de sus valores p — no es posible. El menor de los sesenta valores p, 0,017, corresponde a la prueba de plantillas en la condición A, y es esperable: el mínimo de k valores p uniformes tiene una mediana de $1 - 2^{-1/k}$, que es 0,083 para ocho plantillas y 0,038 para los dieciocho estados de la variante de excursiones (calculado), y la fila de plantillas (0,017 a 0,055) y la de la variante (0,020 a 0,176) se sitúan en torno a esas medianas. Nada separa las condiciones.

Con las campañas posteriores la batería puede leerse como pretende la norma, en el segundo nivel (Sección 4.2 de SP 800-22): cincuenta y nueve secuencias de 10^6 bits — las cuarenta y ocho de las condiciones A–D en los tres chips y once trozos de las dos capturas largas E y F de la Sección 3.6—, con cada valor p de las pruebas de varias partes tomado por separado, cuarenta y ocho estadísticos en total. Todos cumplen el criterio de proporción (al menos 0,951 de las secuencias pasan con $\alpha = 0{,}01$; 0,939 en las pruebas de excursiones, que se aplican a las 34 secuencias con ciclos suficientes), y los valores p son uniformes: el menor P-value$_T$ es 0,0075, frente a un umbral de 10^{-4} (orientativo en las pruebas de excursiones, por debajo de las 55 secuencias que pide la norma). Dieciocho de los 2182 valores p caen por debajo de 0,01, donde se esperan 21,8. El segundo nivel confirma lo que dijo el primero, con más potencia, y no añade nada sobre la predictibilidad.

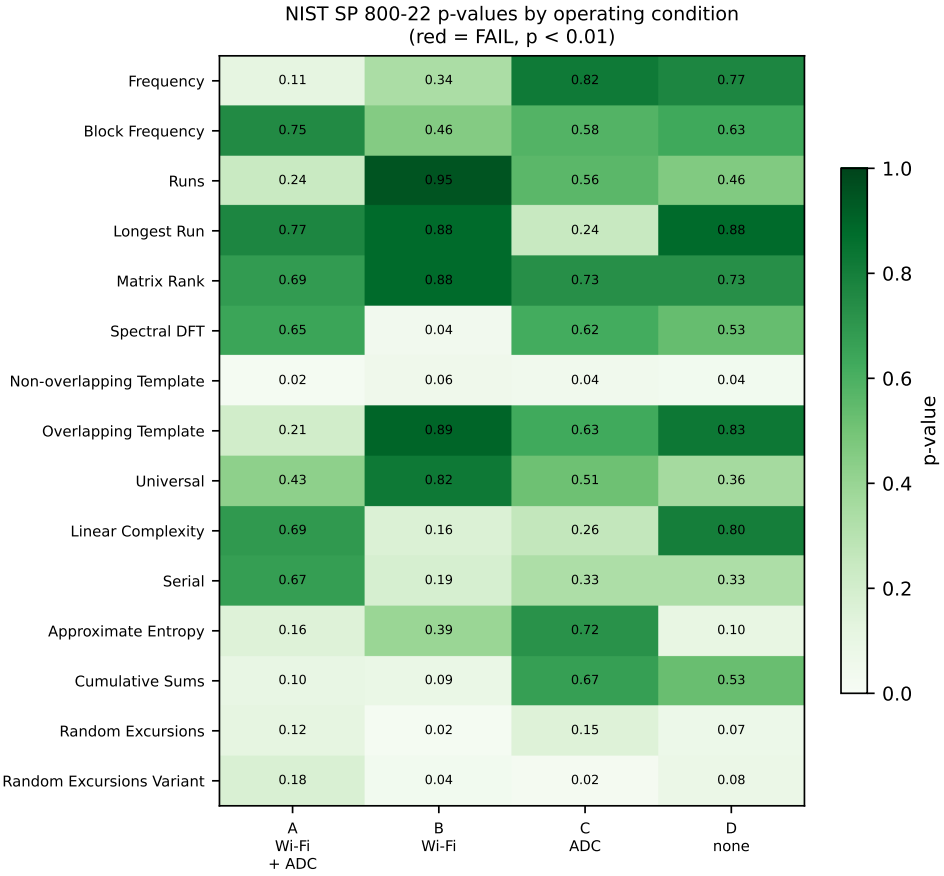


Figura 3.2: Valores p de SP 800-22 de las cuatro secuencias de la primera campaña, uno por condición y prueba, de 10^6 bits cada una [MEDIDO]. Una prueba falla por debajo de 0,01; ninguna lo hace. Las pruebas con varios valores p muestran su mínimo.

Pasar la batería dice que quince estadísticos fijos no distinguen estos 10^6 bits de bits independientes y sin sesgo al nivel del 1 %. No dice que sean impredecibles: un generador determinista con una buena función de salida pasa por diseño, y por eso el NIST decidió en 2022 revisar SP 800-22 y rechazó su uso para evaluar generadores criptográficos [4]. La condición D es justo ese caso. La batería la pasa, Espressif la llama pseudoaleatoria, y la prueba de reinicios de la Sección 3.6 muestra que sin el ruido de arranque es una secuencia fija. La batería sirve aquí como control de defectos groseros — sesgo, periodicidad, correlación de corto alcance — y no encuentra ninguno.

3.4. Entropía de Shannon, tasa y temperatura

Sobre las mismas cuatro secuencias se calcularon seis métricas más (`analysis/esp32_rng/entropy_analysis.py`). La Tabla 3.2 las da junto al valor que devuelve una fuente perfecta con la misma longitud, que es la referencia, en lugar del ideal de una secuencia infinita.

Tabla 3.2: Métricas estadísticas de la primera campaña, una secuencia de 10^6 bits por condición [MEDIDO]: entropía de Shannon por byte y por bit, mayor autocorrelación $|r_k|$ para retardos k de 1 a 1000, planitud espectral de un periodograma de los primeros 65 536 bits, razón de compresión de zlib y χ^2 de bytes. La última fila es lo que devuelve una fuente perfecta con esta longitud (calculado, véase el texto).

Condición	H (bit/byte)	H (bit/bit)	máx $\lvert r_k \rvert$	Planitud	zlib	p de χ^2
A	7,99852	0,9999982	0,00375	0,564	1,00037	0,473
B	7,99852	0,9999993	0,00345	0,564	1,00037	0,470
C	7,99877	1,0000000	0,00332	0,563	1,00037	0,972
D	7,99836	0,9999999	0,00314	0,561	1,00037	0,109
Fuente perfecta	$7{,}99853 \pm 0{,}00013$	$1 - 7 \times 10^{-7}$	0,0034	0,561	> 1	uniforme

Cada métrica tiene un suelo propio con 10^6 bits, y la lectura empieza ahí. La entropía empírica (plug-in) de 125 000 bytes sobre 256 valores se queda por debajo de 8 bits incluso para una fuente perfecta, en $\chi^2_{255}/(2N \ln 2)$, es decir, $0{,}00147 \pm 0{,}00013$ bit; los cuatro déficits medidos, de 0,0012 a 0,0016 bit, son ese sesgo, y la entropía de bytes y el χ^2 de bytes son el mismo estadístico en dos unidades. La entropía por bit reformula la prueba monobit. La autocorrelación en cada retardo tiene una desviación típica de $1/\sqrt{n} = 0{,}001$, y el mayor de mil valores así tiene una mediana de 0,0034 para ruido blanco. La planitud de un único periodograma sin promediar de ruido blanco no es 1 sino $e^{-\gamma} \approx 0{,}561$, siendo γ la constante de Euler, porque cada intervalo de frecuencia sigue una distribución exponencial. La Figura 3.3 muestra las dos distribuciones que hay detrás de esas columnas: en las cuatro condiciones las mil autocorrelaciones tipificadas siguen una distribución normal estándar y los intervalos del periodograma una exponencial, como en el ruido blanco. En todas las columnas las cuatro condiciones quedan donde quedaría una fuente perfecta.

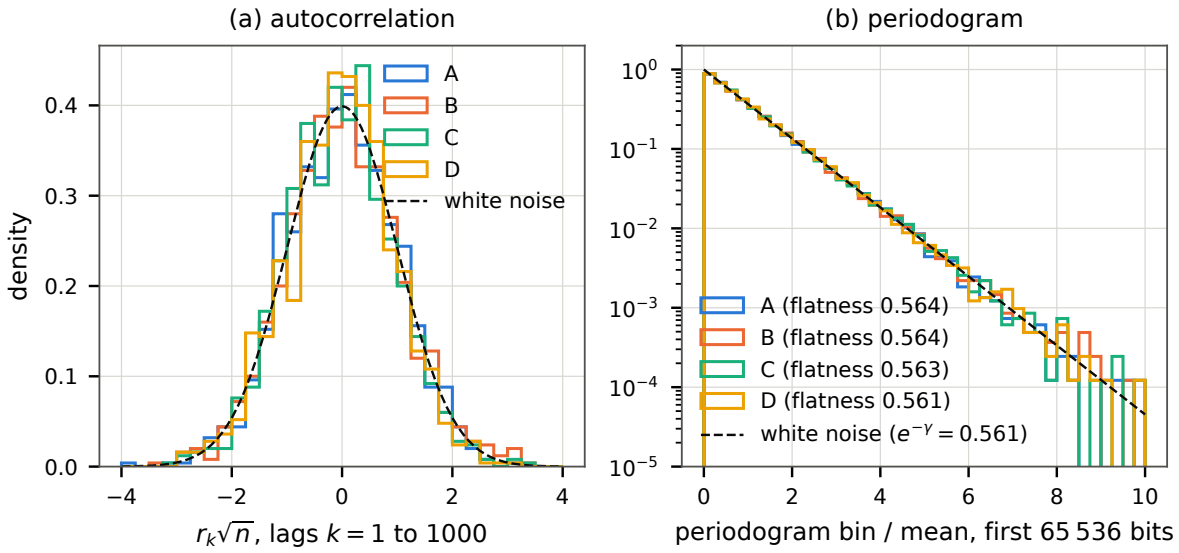


Figura 3.3: Autocorrelación y periodograma de las cuatro secuencias de la primera campaña [MEDIDO], frente al ruido blanco (discontinua). (a) Distribución de $r_k\sqrt{n}$ para retardos k de 1 a 1000, $n = 10^6$; para ruido blanco es normal estándar. (b) Distribución de los intervalos del periodograma de los primeros 65 536 bits, divididos por su media, en escala logarítmica; para ruido blanco es exponencial, y por eso la planitud espectral de un único periodograma tiende a $e^{-\gamma} \approx 0{,}561$ y no a 1.

Como SP 800-22, son promedios sobre la salida que una secuencia determinista puede alcanzar, y la entropía de Shannon no es la magnitud que necesita una clave: la generación de claves cuenta la min-entropía, el logaritmo negativo de la probabilidad del resultado más probable, que nunca es mayor y es el objeto de la Sección 3.6.

El firmware cronometra la tasa con el temporizador del sistema de 1 μs sobre 64 KiB, unos 12,6 ms, así que su resolución es mejor que el 0,01 % (calculado). En las cuatro campañas, las veinticuatro secuencias sin la tarea del ADC se generaron a entre 5,21 y 5,22 MB/s (10^6 bytes por segundo), y las veinticuatro con ella a 2,77 MB/s en las dos primeras campañas y a 2,70 MB/s en las dos últimas; la radio cambia la tasa en menos del 0,1 %. La tarea del ADC reduce la tasa a la mitad, por una causa que no se aisló. Frente al manual, 1,30 millones de palabras por segundo sin la tarea está por debajo de los 5 MHz del camino de la radio, y de 0,68 a 0,69 millones con ella está por encima de los 500 kHz del camino SAR: las condiciones A y C leen de 21,6 a 22,2 Mbit/s mientras que el camino SAR inyecta como mucho 16 Mbit/s, así que aunque las conversiones de la condición C llegaran al generador y cada bit inyectado fuera completamente aleatorio, detrás de cada bit de salida no podría haber más de 0,72 a 0,74 bit de ruido nuevo (calculado). El camino de la radio, a 160 Mbit/s, supera casi cuatro veces la mayor tasa de lectura, 41,7 Mbit/s.

La temperatura va al final porque su instrumento es el más tosco. El firmware lee el sensor mediante una función de la ROM no documentada que devuelve un entero de 8 bits sin calibrar en grados Fahrenheit, así que un paso es 5/9 °C, 0,556 °C. En la

primera campaña solo devolvió 52,2 y 52,8 °C, que son 126 y 127 °F: todo el rango de temperatura de la campaña es un paso del sensor. La correlación entre la entropía de bytes y la temperatura sobre las cuatro secuencias, $r = 0{,}795$ con $p = 0{,}21$, se calculó por tanto frente al bit menos significativo del sensor, y no aporta información; no se afirma ningún efecto de la temperatura, en ningún sentido. La Figura 3.4 muestra los datos en la unidad del propio sensor: una secuencia leyó 126 °F de principio a fin, dos leyeron 127 °F, una pasó del primer valor al segundo mientras se generaba, y las cuatro entropías quedan a menos de dos desviaciones típicas de una fuente perfecta de la misma longitud. Los tres chips identificados leyeron después de 49,4 a 51,7, de 61,7 a 64,4 y de 42,8 a 45,0 °C en la misma escala sin calibrar, así que chip y temperatura están confundidos. Un estudio de temperatura necesita un sensor calibrado y un barrido controlado, y no se hizo ninguno.

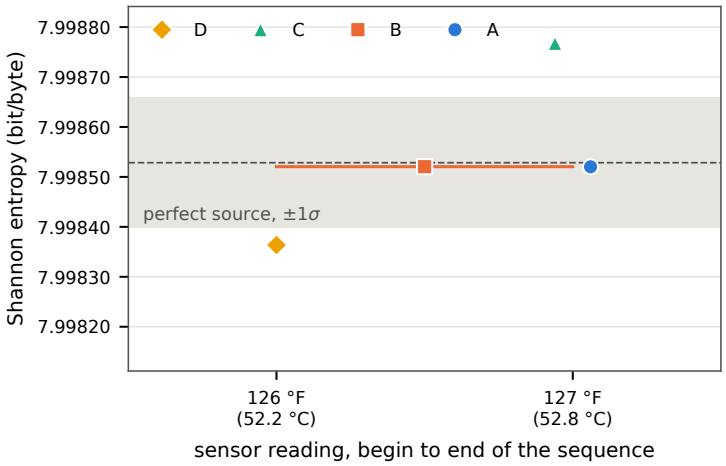


Figura 3.4: Entropía de bytes de las cuatro secuencias de la primera campaña frente a la lectura del sensor del chip [MEDIDO], en su propia unidad, un entero sin calibrar en grados Fahrenheit; una barra abarca las lecturas anterior y posterior a la secuencia. La banda es una fuente perfecta de 125 000 bytes, media y una desviación típica (calculado). Toda la campaña es un paso del sensor, así que la correlación de $r = 0{,}795$ calculada sobre estos puntos no dice nada de la temperatura.

3.5. Un recolector de claves multifuente

Si no se puede confiar en el generador con la radio apagada, un nodo que mantiene la radio apagada necesita otras fuentes. Esta parte del trabajo construyó un recolector (`firmware/esp32_rng/main/entropy_harvester.c`) que añade dos fuentes del mismo chip cuya disponibilidad complementa la del generador: el patrón de arranque de la SRAM, presente en cada encendido antes de que arranque ningún periférico [65], y los bits menos significativos del ADC leyendo una entrada flotante. Estas pasan por la corrección de Von Neumann [120], que descarta los pares 00 y 11 y convierte 01 en 0 y 10 en 1, de modo que bits independientes salen sin sesgo a costa de al menos tres cuartas partes de ellos.

Las aportaciones se acondicionan con SHA-256 en una reserva (*pool*) de 256 bits,

$$\text{pool}_0 = \text{SHA256}\big(r \,\|\, \text{SHA256}(S) \,\|\, a \,\|\, \texttt{"TFM0"}\big), \tag{3.1}$$

donde r son 32 bytes de `esp_fill_random()`, S el patrón de SRAM de 4 KiB, a 32 bytes de bits del ADC corregidos y el último campo una etiqueta que separa este uso del hash de los demás. Cada salida de 32 bytes aplica el hash a la reserva junto con un contador y una lectura nueva del generador, y después la reserva avanza como un trinquete (ratchet),

$$o_i = \text{SHA256}(\text{pool} \,\|\, i \,\|\, r_i), \qquad \text{pool} \leftarrow \text{SHA256}(\text{pool} \,\|\, o_i), \tag{3.2}$$

de modo que una reserva capturada tras la salida i no permite obtener las anteriores; un resembrado incorpora a la reserva, mediante el hash, datos nuevos del generador y del ADC. Concatenar antes de aplicar el hash da la única garantía que no necesita independencia entre fuentes que comparten chip, alimentación y temperatura: $H_\infty(X_1 \| X_2 \| X_3) \geq \max_i H_\infty(X_i)$.

La afirmación del corrector se apoya en una línea. Para bits independientes con una probabilidad desconocida p de valer uno, $\Pr[01] = (1-p)\,p = \Pr[10]$, así que un par conservado da 0 y 1 con probabilidad $1/2$ cada uno, sea cual sea p. Un par se conserva con probabilidad $2p(1-p)$ y da un bit por cada dos, así que el rendimiento es de $p(1-p)$ bits de salida por bit de entrada, como mucho $1/4$, con $p = 1/2$ (Figura 3.5). El corrector elimina el sesgo, no la correlación: para bits dependientes Pr[01] y Pr[10] no tienen por qué ser iguales. El firmware toma el bit menos significativo de cada conversión de 12 bits, empaqueta ocho por byte, corrige 2048 cada vez y repite hasta tener 32 bytes. No existe una captura aparte de estos bits, así que no se midieron ni su sesgo ni su correlación.

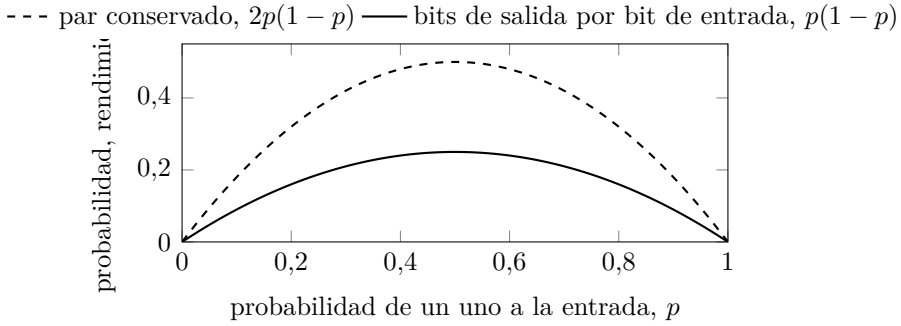


Figura 3.5: Rendimiento del corrector de Von Neumann sobre bits de entrada independientes (calculado). Incluso una entrada sin sesgo pierde tres cuartas partes de sus bits; la salida carece de sesgo para todo p de $(0, 1)$ solo si los bits de entrada son independientes.

Aplicar el hash a la concatenación se justifica, bajo hipótesis, con la teoría de los extractores de aleatoriedad. El lema del hash sobrante [70] establece que una función elegida al azar de una familia de hash universal, aplicada a una entrada de min-entropía

k, da una salida de ℓ bits a distancia estadística ε de la uniforme siempre que

$$k \geq \ell + 2\log_2(1/\varepsilon); \tag{3.3}$$

para $\ell = 256$ y $\varepsilon = 2^{-32}$ la entrada necesita 320 bits de min-entropía. SHA-256 es una función fija, no un miembro de una familia elegido al azar, así que el lema no se le aplica tal como está enunciado; su uso como extractor se apoya en modelos idealizados, o en análisis como el de Dodis et al. [71] para las construcciones HMAC y en cascada bajo hipótesis sobre la función de compresión, y SP 800-90B admite una función hash aprobada como componente de acondicionamiento verificado [6]. Lo que importa aquí es la condición más que la cota: la reserva solo está cerca de la uniforme si las entradas reúnen unos 320 bits de min-entropía, y los párrafos siguientes muestran que eso no se midió para ninguna de ellas. La cota de la concatenación se sigue de una desigualdad que vale para cualquier distribución conjunta, porque el trío más probable no es más probable que el valor más probable de cualquiera de sus componentes,

$$\max_{x_1,x_2,x_3} \Pr[X_1{=}x_1, X_2{=}x_2, X_3{=}x_3] \;\leq\; \min_{i} \max_{x_i} \Pr[X_i{=}x_i], \tag{3.4}$$

y tomando $-\log_2$ en los dos lados se obtiene. Combinar con XOR fuentes de la misma anchura también conserva la mejor min-entropía, pero solo si las fuentes son independientes, y no puede suponerse de tres fuentes en un mismo chip; la concatenación no necesita esa hipótesis, a costa de una entrada del hash más larga. Si el ADC no entrega datos, su campo se llena de ceros y la reserva se forma con las otras dos fuentes.

El trinquete de la Ecuación (3.2) da secreto hacia delante (*forward secrecy*): la reserva tras la salida i no revela las salidas anteriores mientras SHA-256 no pueda invertirse. El resembrado que sigue a las 100 semillas de cada ciclo incorpora con el hash 32 bytes nuevos del generador y 32 del ADC, pool $\leftarrow$ SHA256(pool $\|$ r $\|$ a), y pretende dar secreto hacia atrás (*backward secrecy*), la recuperación tras quedar expuesta la reserva; lo consigue solo en la medida en que r y a lleven entropía que el atacante no pueda predecir, que es justo lo que no se sabe con la radio apagada. La construcción se parece al Hash_DRBG de SP 800-90A [89] —un estado sometido a hash, salidas que aplican el hash al estado, resembrados desde una fuente viva—, pero apunta a algo más estrecho: cien semillas de 256 bits por ciclo, cada una para expandirse con una función de derivación de claves, en lugar de flujos largos de salida, con cada salida absorbiendo una lectura nueva del generador y todo el estado en una palabra de 256 bits. Es un acondicionador pequeño de absorción y exprimido, sin los contadores de resembrado, las pruebas de salud ni la validación de un generador de SP 800-90A.

El recolector se validó fuera de línea. Las semillas producidas en el dispositivo no se capturaron de forma fiable en la primera campaña, y los scripts de captura posteriores dejaban de escuchar en cuanto tenían las cuatro condiciones, antes de que se impriman

las semillas; en el tercer chip, donde la escucha siguió para repetir un bloque estropeado, el firmware sí las imprimió y empezó un segundo ciclo, así que la generación de semillas funciona en el dispositivo, aunque su salida no se analizó. La validación reproduce por tanto la construcción en Python (`analysis/esp32_rng/harvester_from_sources.py`) sobre datos capturados: un patrón de SRAM; la secuencia de la condición D como aportación del generador, la peor condición, a propósito; y 32 bytes de la misma secuencia en lugar del campo del ADC, del que no existe una captura aparte. De las 1000 semillas de 32 bytes así derivadas, todas son distintas y sus bytes son uniformes (χ^2 $p = 0{,}30$); la distancia de Hamming entre dos semillas, sobre 20 000 pares aleatorios, promedia 128,05 de 256 bits (50,02 %) con una desviación típica de 8,03, frente a 128 y 8 para semillas uniformes independientes; y las siete pruebas de SP 800-22 aplicables a los 256 000 bits del flujo de semillas pasan, con p de 0,148 a 0,956 (Figura 3.6).

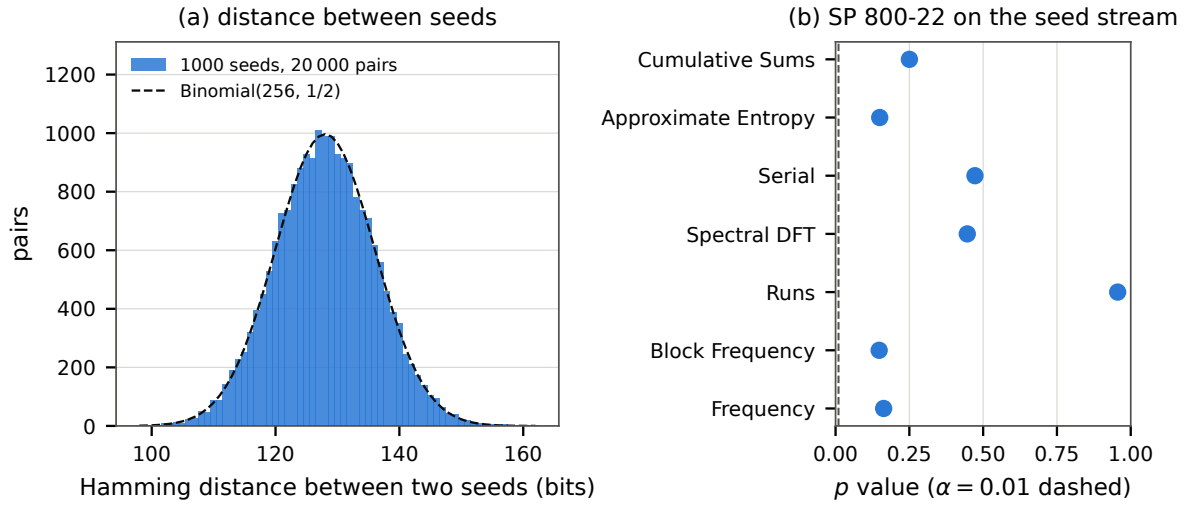


Figura 3.6: Validación fuera de línea de la construcción del recolector (calculado, sobre entradas medidas). (a) Distancia de Hamming entre dos de las 1000 semillas derivadas de un patrón de SRAM y de la secuencia de la condición D, sobre 20 000 pares aleatorios, frente a la distribución binomial de semillas uniformes independientes. (b) Valores p de las siete pruebas de SP 800-22 aplicables a los 256 000 bits del flujo de semillas. Ambas se pasan por construcción (véase el texto) y no dicen nada de la entropía de las semillas.

La fuente SRAM se caracterizó por separado. Solo un corte de alimentación vuelve a aleatorizar las celdas, ya que un reinicio que mantiene la alimentación deja en el búfer lo que escribiera la ejecución anterior, así que se capturó una serie limpia de cuatro patrones con un ciclo de alimentación antes de cada uno. Los nueve patrones capturados tienen un peso de Hamming del 49,6 al 49,9 %, y las seis distancias de la serie limpia van del 6,45 al 6,81 %, con una media del 6,63 %. La fracción de celdas estables, las que leen el mismo valor en los cuatro encendidos, se midió directamente: 87,90 % en la serie limpia y 87,64 % en los cuatro encendidos de la primera serie, la mixta, así que unas 4000 de las 32 768 celdas cambiaron al menos una vez. El primer análisis había inferido un 93 % aproximadamente

a partir de la distancia media, que cuenta con qué frecuencia difieren dos encendidos, no cuántas celdas difieren alguna vez; la cifra que se usa es la contada (Sección 1.5). La distancia entre arranques del 23,9 % calculada inicialmente para la fuente no es una cifra limpia: promedia diez pares de una serie que mezclaba ciclos de alimentación con reinicios que mantenían la alimentación, en la que un patrón difiere de los otros cuatro en el 49,6 al 49,7 % de sus bits, como lo harían datos no relacionados, y los otros seis pares difieren en el 6,5 al 7,4 %, como lo hacen los encendidos de las mismas celdas. La media limpia del 6,63 % queda dentro del 3 al 8 % publicado para el ruido de arranque de las celdas de SRAM [65, 67]; con una placa, cuatro encendidos y una temperatura, eso solo dice que esta SRAM se comporta como las de la bibliografía, no cuánto varía.

La validación muestra que la construcción se implementó tal como se diseñó, no que las semillas lleven entropía. Todas las comprobaciones se pasan por construcción: el contador hace que cada salida sea distinta aunque todas las demás entradas fueran constantes, y SHA-256 convierte entradas distintas cualesquiera en salidas que parecen uniformes y distan media palabra entre sí. Las entradas de esta validación lo ilustran. El campo de SRAM fue un mismo patrón para las 1000 semillas — el primero de la serie mixta, que no es un estado de encendido — y la única entrada que cambiaba entre semillas era la salida de la condición D, que la prueba de reinicios muestra que es una secuencia fija una vez eliminado el ruido de arranque. La entropía de una semilla es como mucho mín$(256, H_\infty)$ bits, siendo H_∞ la min-entropía de las entradas, y esta no se midió para ninguna de ellas. Un peso del 50 % no cambia esto: el 88 % de las celdas de la SRAM son una huella fija, conocida por cualquiera que haya leído el patrón una vez, y acotar lo que aporta el 12 % restante requeriría muchos más encendidos y un modelo de las celdas. El recolector es una arquitectura cuya salida es al menos tan fuerte como su mejor fuente, no una fuente de entropía medida.

El uso previsto se demostró fuera de línea (`analysis/esp32_rng/mqtt_tls_demo.py`) para un nodo conectado genérico, un termostato que publica telemetría. Una semilla de 32 bytes se expande en una clave de sesión de 256 bits con HKDF-SHA256 [72], que primero condensa la semilla en una clave pseudoaleatoria y después la expande,

$$\mathrm{PRK} = \mathrm{HMAC}_H(\mathrm{salt}, \mathrm{seed}), \qquad K = \mathrm{HMAC}_H(\mathrm{PRK}, \mathrm{info} \,\|\, \texttt{0x01})_{1..L}, \tag{3.5}$$

con H = SHA-256, sin sal (HKDF usa entonces una cadena de ceros de la longitud del hash), info = `"tfm-mqtt-session"` y $L = 32$ bytes, un solo bloque porque L coincide con la longitud del hash; cambiando info se derivan de la misma semilla claves independientes para otros fines. La clave alimenta AES-256-GCM con un nonce de 96 bits y el tema `esp32/hvac/telemetry` como datos asociados, de modo que la etiqueta de 128 bits cubre texto cifrado y tema. Con las dos primeras semillas del conjunto de validación, la misma semilla da la misma clave, las dos claves difieren en 130 de 256 bits, un mensaje de 61 bytes

hace el viaje de ida y vuelta, y el descifrado rechaza el mensaje presentado bajo otro tema y el mensaje con un bit del texto cifrado cambiado (calculado, en el PC). Ese resultado tiene dos límites. GCM por sí solo no rechaza un mensaje reenviado bajo el mismo tema, para lo que hace falta un contador en los datos autenticados, y la demostración no lo tiene. Y toma su nonce del sistema operativo del PC, no del recolector; un par de clave y nonce repetido rompería a la vez la confidencialidad y la autenticidad de GCM, así que en un dispositivo el generador de nonces necesita el mismo cuidado que la clave. El caso para el que se construyó el recolector es un nodo que aprovisiona su clave en la condición D poco después de un arranque en frío, donde la Sección 3.6 muestra que, sin el ruido de arranque, el generador devuelve una de unos pocos miles de salidas, la misma en dos chips; derivar las claves con una función de derivación a partir de una reserva acondicionada, y no del registro en bruto, es la práctica que recomienda la RFC 4086 [48]. La demostración no cierra ese caso: ejercita la criptografía, y la clave hereda la entropía de la semilla sin añadir ninguna. Un segundo modo del script, una sesión MQTT sobre TLS mediante el cliente paho que tomaría una semilla viva del tema `esp32/harvester/seed`, se implementó y nunca se ejecutó. Acepta cualquier certificado del broker y recibe la semilla por la red, así que muestra la fontanería, no un diseño de establecimiento de claves.

3.6. La referencia del ESP32 frente a SP 800-90B

O8 pide una comparación numérica con el ESP32, y la mitad puede cerrarse sin la placa: el lado de la referencia. Las Secciones 3.3 y 3.4 aplicaron SP 800-22 y la entropía de Shannon a la primera campaña; los estimadores de SP 800-90B se aplican aquí a las tres primeras y a la primera tanda de la cuarta, que en conjunto dan nueve repeticiones de 10^6 bits por condición, tres en cada uno de los tres chips del mismo modelo (ESP32-D0WD-V3). Otras dos capturas con la radio apagada, de $3{,}52 \times 10^6$ y $8{,}38 \times 10^6$ bits, se tomaron con el firmware verificador del Capítulo 6. Las treinta y ocho se pasaron por la implementación de referencia del National Institute of Standards and Technology (NIST) de los estimadores de SP 800-90B [37, 6] (versión 1.1.8), por la vía IID y por la vía no IID, en el formato binario que la norma prescribe para fuentes de bits. La Tabla 3.3 recoge el resultado.

La resolución va primero, como en todo este trabajo. El estimador MCV devuelve $-\log_2 p_u$ con $p_u = \hat{p} + 2{,}576\sqrt{\hat{p}(1-\hat{p})/N}$, de modo que ni siquiera una fuente perfecta puede puntuar por encima de 0,9963 con 10^6 bits, de 0,9980 con $3{,}5 \times 10^6$ ni de 0,9987 con $8{,}4 \times 10^6$. Cada una de las treinta y ocho capturas queda a menos de 0,0037 de ese techo, y ninguna implica un desequilibrio entre unos y ceros superior a $2{,}6\sigma$, que es lo que se observa en treinta y ocho extracciones de una fuente sin sesgo.

La vía IID necesita una calibración antes de que puedan leerse sus veredictos. De las setenta y seis ejecuciones (treinta y ocho capturas en los formatos de bits y de bytes),

Tabla 3.3: El generador del ESP32 frente a los estimadores de SP 800-90B [MEDIDO]. Condiciones A–D: nueve repeticiones de 10^6 bits cada una, tres en cada uno de los tres chips, media y rango sobre las repeticiones; E y F: capturas únicas de $3{,}5 \times 10^6$ y $8{,}4 \times 10^6$ bits. El techo es el valor más alto que el estimador del valor más común (MCV) puede devolver para una fuente perfectamente uniforme de esa longitud con un 99 % de confianza; la cifra no IID es el mínimo de los diez estimadores de la Sección 6.3 de la norma, y la última columna nombra el estimador que la fijó (veces de nueve). Los veredictos de la vía IID se discuten en el texto.

Captura	Condición	H (IID)	Techo	H (no IID)	Limitante
A	Wi-Fi + ADC	0,9950	0,9963	0,843 (0,812–0,893)	Compr. 7, Lag 1, Colis. 1
B	Wi-Fi	0,9952	0,9963	0,857 (0,809–0,912)	Compresión 8, LRS 1
C	ADC	0,9948	0,9963	0,847 (0,812–0,918)	Compresión 9
D	ninguna	0,9955	0,9963	0,853 (0,832–0,916)	Compresión 8, Colisión 1
E (verificador)	ninguna	0,9977	0,9980	0,867	Compresión
F (verificador)	ninguna	0,9982	0,9987	0,911	Compresión

cinco fallan una sola prueba, nunca el mismo fichero en ambos formatos: una prueba de permutación en cada formato y tres pruebas χ^2 en el formato de bytes. La frecuencia con que datos buenos fallan estas pruebas a esa misma longitud no es el 10^{-3} nominal, así que se midió, ejecutando la misma herramienta sobre 1000 ficheros de 125 000 bytes y 200 ficheros de 10^6 bits del generador del sistema operativo. En el formato de bytes, la batería de permutación falla en el 1,9 % de los ficheros buenos y el par χ^2 en el 2,4 %; en el formato de bits, la batería de permutación falla en el 2,5 % y el par χ^2 en ninguno de los 200. La cifra de χ^2 se rastreó hasta su causa reimplementando ambas pruebas, lo que reproduce los cincuenta y dos valores p de la herramienta en las veintiséis primeras capturas hasta la sexta cifra decimal, y ejecutándolas sobre 5000 ficheros buenos. La prueba de independencia del formato de bytes reparte 62 500 pares sin solape entre 65 536 celdas y agrupa celdas hasta que cada grupo espera cinco cuentas, pero a esta longitud su p no es uniforme: la mediana es 0,175 en lugar de 0,5, y cae por debajo de 10^{-3} en el 1,7 % de los ficheros, diecisiete veces la tasa nominal. Las otras tres pruebas χ^2 están bien calibradas, y el defecto desaparece a partir de 10^6 bytes. Frente a estas tasas, los fallos del ESP32 no tienen nada de particular: un fallo de permutación en treinta y ocho ejecuciones en formato de bytes donde se esperan 0,72 ($P = 0{,}52$), tres fallos χ^2 donde se esperan 0,91 ($P = 0{,}063$), y un fallo de permutación en el formato de bits donde se esperan 0,95 ($P = 0{,}62$). En un chip, la prueba de independencia de bytes dio $p < 0{,}05$ en seis capturas de doce, frente a 2,9 esperadas; agrupar las capturas de cada chip, donde la prueba está calibrada y es más potente, da valores p empíricos de 0,84 y 0,94 en ese chip para pares de bytes alineados con la palabra de 32 bits y desplazados un byte, y de 0,53 y 0,66, y de 0,42 y 0,22, en los otros dos, así que no hay dependencia entre bytes vecinos. En el tercer chip los valores p de bondad de ajuste en bytes salieron bajos sin que ninguno fallara (mediana 0,21 frente a 0,51 en ficheros buenos; Mann-Whitney unilateral

$p = 0{,}0031$, en el límite de una corrección de Bonferroni para dieciséis comparaciones); la segunda tanda de doce capturas de ese chip dio una mediana de 0,34 y $p = 0{,}13$, así que se lee como azar. El suceso más raro es el fallo de bondad de ajuste de la primera captura de la condición D ($p = 2{,}4 \times 10^{-4}$): esa prueba falla en el 0,04 % de los ficheros buenos, así que un fallo así en treinta y ocho tiene una probabilidad cercana al 1,5 %. No se reprodujo en las ocho capturas posteriores de la misma condición ($p = 0{,}14$ a 0,96) ni en E y F, y se registra como azar; la advertencia de Espressif de que la aleatoriedad del generador no está garantizada sin la radio o el ADC sigue en pie como una advertencia que estas pruebas no ven. Formalmente, un único fallo retira la hipótesis IID para esa captura; como la cifra que se retiene es la no IID, nada de lo que sigue depende de ello. Leída con su calibración, la vía IID encuentra la salida del ESP32 indistinguible de una fuente binaria de entropía completa con este tamaño de muestra. Es el mismo veredicto que dio SP 800-22, alcanzado por otro camino, y dice lo mismo: no se ve nada mal.

La vía no IID dice otra cosa, y leerla bien importa más que el número. En treinta y siete de las treinta y ocho capturas, su mínimo lo fija uno de tres estimadores — compresión, colisión o subcadena repetida más larga — con 0,81 a 0,92 bit por bit; en la restante, el predictor Lag, con 0,88; los estimadores MCV y de Markov nunca bajan de 0,99. Se sabe que esos tres estimadores son conservadores con datos casi uniformes [121]; los predictores de Kelsey, McKay y Sönmez Turan [122] se introdujeron precisamente porque los estimadores anteriores de la norma juzgan mal fuentes de ese tipo. Las repeticiones muestran que esto es un comportamiento de los estimadores y no una propiedad de las condiciones. La estimación por compresión tiene una desviación típica de 0,03 a 0,04 *dentro* de una condición, mayor que el 0,013 que separa las medias de las cuatro condiciones, y un análisis de la varianza de un factor sobre cada uno de los once estimadores no encuentra diferencia entre condiciones: el p más pequeño es 0,045, para el predictor MultiMCW, cuyas medias por condición no siguen ningún orden físico, y un valor por debajo de 0,05 entre once pruebas es lo que da el azar; queda muy por encima del 0,0045 que exigen once pruebas. Lo mismo vale entre chips. Agrupando las condiciones, que no difieren, ningún estimador separa las doce capturas de cada chip al nivel corregido; el p más pequeño, 0,032 para MCV, corresponde a una diferencia de unos 0,001 bit por bit junto al techo de resolución. El mínimo no IID es 0,852, 0,847 y 0,851 en los tres chips ($p = 0{,}92$): con tres dispositivos, la referencia no depende de cuál se mida, a la resolución de estos estimadores. La captura más grande añade una segunda muestra del efecto de la resolución: en el formato de bytes, su estimador limitante es la prueba de t-tupla con $t = 2$, que devuelve 7,39 bits por byte porque 10^6 bytes contienen demasiado pocos de los 65 536 pares posibles para que el más frecuente sea representativo, exactamente como le ocurre al techo del MCV con $t = 1$. Ningún estimador resuelve con nueve repeticiones diferencia alguna entre Wi-Fi encendido, ADC encendido y ninguno de los dos, y no se afirma ninguna.

La prueba de reinicios de la Sección 3.1.4 de la norma responde a lo que la estadística

anterior no puede: si un generador no tiene ruido propio, su salida tras un reinicio en frío debe repetirse. Un firmware dedicado leyó 1024 bytes con `esp_fill_random` como primera acción de la aplicación, con la radio y el ADC apagados (condición D), y el chip de MAC a0:dd:6c:0e:fb:4c se reinició 1000 veces por su patilla de habilitación; la ROM de arranque notificó un reinicio por encendido todas las veces. Dos variantes comparten la aplicación y difieren solo en el bootloader de segunda etapa: el de serie, que inyecta ruido del SAR ADC en el generador durante unos 126 ms mientras se ejecuta, y el mismo bootloader con esa llamada desviada a una función vacía en tiempo de enlazado, lo que confirma el desensamblado. Otras tres variantes hacen una segunda lectura tras arrancar el Wi-Fi como en las condiciones A y B, sin asociarse a ninguna red, y esperar 200 ms, con cada uno de los dos bootloaders; la tercera es un control que espera el mismo tiempo sin tocar la radio, de modo que la lectura cae en el mismo tick del planificador. La Tabla 3.4 recoge el resultado.

Tabla 3.4: Prueba de reinicios del generador del ESP32 [MEDIDO]: 1000 reinicios en frío por variante, un chip. H_r y H_c son el mínimo de los estimadores no IID sobre el conjunto de filas y el conjunto de columnas de la norma (1000 reinicios × 1000 bits); la prueba falla cuando cualquiera de los dos queda por debajo de la mitad de la estimación inicial H_I, la cifra no IID más baja de este chip en la misma condición (0,838 bit por bit en D, 0,859 en B). El formato de bytes da los mismos veredictos. Los solapes son reinicios cuya salida aparece desplazada dentro de la de otro, todos idénticos palabra por palabra. Las lecturas tardías caen entre 352 y 412 ms después de que arranque la aplicación, la mayoría a 352 ms.

Bootloader	Lectura	H_r / H_c	Distintas	Solapes	Veredicto
De serie	en el arranque, radio apagada	0,898 / 0,854	1000	0	pasa
Sin ruido de arranque	en el arranque, radio apagada	0,024 / 0,813	759	5424	falla
Sin ruido de arranque	tardía, radio apagada (control)	0,024 / 0,808	760	5632	falla
Sin ruido de arranque	tardía, Wi-Fi encendido	0,831 / 0,845	1000	0	pasa
De serie	tardía, Wi-Fi encendido	0,844 / 0,807	1000	0	pasa

Con el bootloader de serie el generador pasa en ambos formatos, y las 1000 salidas difieren entre sí exactamente como lo harían datos aleatorios: la distancia de Hamming media entre dos reinicios es de 4096,0 bits de 8192, frente a 4096 ± 45 esperados. Sin el ruido de arranque falla de forma tajante, y los datos muestran por qué. Cada reinicio es una ventana sobre una única secuencia fija, leída desde un punto de partida ligeramente distinto: dondequiera que el comienzo de la salida de un reinicio aparece dentro de la de otro, desplazado unas pocas palabras de 32 bits, todo el solape coincide palabra por palabra, 5424 veces sin una sola diferencia. Lo único que cambia entre reinicios es el instante en que la aplicación empieza a leer, que varía en unos pocos miles de ciclos de reloj de un arranque al siguiente. Eso vale entre 6,5 bits por reinicio, la cota de la norma a partir de la salida más repetida, y 10,8 bits si todos los puntos de partida fueran igual de probables, de 8192 bits leídos. Con la radio y el ADC apagados el generador

no tiene ruido propio, y la afirmación de Espressif de que entonces su salida es solo pseudoaleatoria [43] queda confirmada directamente, cosa que ninguna prueba estadística sobre un flujo continuo podría hacer. De ello se siguen dos consecuencias. Las capturas de la condición D de la Tabla 3.3 son la salida de un generador determinista sembrado durante el arranque: los estimadores les acreditan 0,85 bit por bit, tanto como con la radio encendida, mientras que su entropía física es como mucho la que inyectó el bootloader, que esos estimadores no miden. Y una aplicación que deriva una clave justo después del arranque con la radio apagada depende por completo del ruido del bootloader de serie; sin él, la clave podría adivinarse en unos pocos miles de intentos, el problema del arranque en frío descrito por Blanco-Romero et al. [3].

Las filas con radio responden a la otra mitad de la afirmación de Espressif. Sin el ruido de arranque, arrancar el Wi-Fi basta para que la prueba pase: las 1000 salidas son todas distintas, no se comparte ninguna ventana, y los 289 pares de reinicios que leían valores idénticos antes de arrancar la radio leen 289 valores diferentes después. La temporización por sí sola no lo explica. El control espera el mismo tiempo y lee en el mismo tick del planificador, con un instante de lectura que varía otro tanto (3824 frente a 5260 ciclos de reloj entre los reinicios que comparten el tick), y se repite exactamente como en el arranque: 760 salidas distintas, 5632 solapes idénticos, unos 1700 estados. Con tan pocos estados, los 797 reinicios que comparten un tick en la variante con radio mostrarían unas 180 repeticiones solo por temporización; no muestran ninguna. Sin la radio, el generador no recibe ruido en sus primeros 350 ms de funcionamiento; con la radio, sí. Lo que la prueba no da es cuánta entropía por bit aporta la radio: los 0,83 bit por bit validados son la cifra de los estimadores, con su sesgo habitual, no una medida del ruido de la radio.

El tercer chip, f8:b3:b7:2a:09:28, repite el cuadro con los mismos dos binarios: el bootloader de serie pasa (H_r / H_c 0,892 / 0,848, 1000 salidas distintas), y sin el ruido de arranque falla ($H_r = 0{,}016$, 746 salidas distintas, 5444 solapes idénticos, otra vez 87 grupos). Más aún, los dos chips se repiten *entre sí*. 256 salidas distintas de 1024 bytes aparecen en los dos, y cubren 352 y 347 de sus 1000 reinicios; 988 salidas del tercer chip empiezan, a un número entero de palabras de 32 bits, dentro de una salida del otro, en 8369 solapes sin una sola palabra distinta; y las 3000 salidas sin ruido de ambos chips y de otra compilación del firmware caen en los mismos 87 grupos, cada uno con salidas de las tres. Con el bootloader de serie ninguna salida se alinea con otra, ni dentro de un chip ni entre chips. Sin el ruido de arranque, la salida de los dos chips probados es por tanto una función determinista de un punto de partida pequeño, y la misma función en ambos: quien tenga un chip y el firmware puede precalcular lo que devuelve otro. Dos chips de una misma revisión no bastan para afirmarlo de todos los ESP32.

Un firmware dedicado mostró qué elige ese punto de partida. Sin el ruido de arranque y con las interrupciones apagadas, lee 1024 bytes, espera un número fijo de instrucciones y vuelve a leer, con la espera elegida en cada reinicio entre cuatro valores. En las 204

parejas de reinicios cuya primera lectura fue idéntica, las segundas lecturas fueron idénticas cuando la espera era la misma, desplazadas exactamente una palabra de 32 bits cuando era 174 ciclos de CPU más larga (lo que tarda `esp_fill_random` por palabra), exactamente 100 palabras cuando era 17 400 ciclos más larga, y sin relación cuando era 2 ciclos más larga, un ciclo de APB, en todas las parejas. Sin ruido, el generador avanza con el reloj APB desde un estado de reset fijo, se lea o no, y su salida solo depende del instante en que empieza la lectura: los 87 grupos son las 87 fases de un ciclo de APB en una palabra de 174 ciclos, una propiedad de este firmware y de este reloj, no del chip.

Tabla 3.5: Qué fuente inyecta ruido [MEDIDO], tercer chip, sin el ruido de arranque: una primera lectura al arrancar, una fuente y una segunda lectura en un instante fijo de 800 ms. Una fuente inyecta ruido si las segundas lecturas dejan de ser ventanas de la secuencia fija (solapes) y no se repiten nunca. 300 reinicios por variante, 1000 para el ADC y la fuente de arranque, que además pasan la prueba de reinicios de la norma (H_r / H_c 0,864 / 0,877 y 0,840 / 0,842).

Fuente entre las dos lecturas	Distintas	Solapes	Ruido
Ninguna (control)	284 de 300	475	no
Wi-Fi iniciado, radio sin arrancar	284 de 300	461	no
Radio arrancada y parada (85 a 262 ms encendida)	300 de 300	0	sí
Unidad del ADC creada y borrada, sin conversiones	274 de 300	0	no
Conversiones del ADC durante 1 ms (17)	300 de 300	0	sí
Conversiones del ADC durante 200 ms (unas 4400)	1000 de 1000	0	sí
Fuente de arranque encendida por la aplicación, 200 ms	1000 de 1000	0	sí

La Tabla 3.5 responde a la pregunta que quedó abierta en la Sección 3.2. Iniciar el controlador del Wi-Fi, que alimenta el dominio del módem pero no la radio, no inyecta nada; arrancar la radio sí, incluso con la exposición más corta posible, porque la calibración de la PHY dentro de `esp_wifi_start` ya la mantiene encendida unos 85 ms. La fuente de arranque funciona cuando la enciende la aplicación. Y las conversiones ordinarias del SAR ADC, que ni la guía de software ni las fuentes del controlador conectan con el generador, también inyectan ruido: bastan diecisiete, mientras que crear y borrar el conversor sin convertir no. Las repeticiones exactas de esta última variante muestran que sigue sin ruido aunque, en esa compilación, sus salidas no compartan ventanas desplazadas. No se ha encontrado en la bibliografía consultada una prueba de reinicios del generador del ESP32, ni una comparación de sus salidas entre chips.

3.7. Lo que una caja negra puede decir y lo que no

Todos los instrumentos estadísticos aplicados al generador del ESP32 dan la misma respuesta. SP 800-22 pasa las quince pruebas en las cuatro condiciones; las métricas de Shannon, de autocorrelación, espectrales y de compresión quedan donde quedaría una fuente perfecta con esta longitud; la vía IID de SP 800-90B, leída con su calibración,

encuentra la salida indistinguible de una fuente binaria de entropía completa; y la vía no IID acredita de 0,81 a 0,92 bit por bit, sin ninguna diferencia entre condiciones o entre chips que resuelva algún estimador. Ninguna de estas cifras distingue la condición D, en la que Espressif dice que el generador es pseudoaleatorio, de la condición A, en la que están activas las dos fuentes documentadas.

La prueba de reinicios es la única medida que lo hace, porque plantea una pregunta física y no estadística: £se repite la salida cuando el generador vuelve a arrancar desde el mismo estado? Sin el ruido de arranque y con la radio apagada se repite, palabra por palabra, y repite la misma secuencia en dos chips distintos; con la radio, la fuente del arranque o el ADC convirtiendo, no. El generador no tiene ruido propio con la radio y el ADC apagados, y la radio sí le inyecta ruido. Las cifras de SP 800-90B de la condición D son, por tanto, el veredicto de los estimadores sobre la continuación determinista de una semilla de arranque: una cifra estadística, que no puede acreditarse como entropía física. Lo mismo vale, de forma menos visible, para las demás condiciones. La prueba de reinicios muestra que la radio inyecta ruido, no cuánto; para las condiciones A y C muestra que las conversiones del ADC sí llegan al generador, pero según las propias tasas del manual el camino SAR no podría aportar un bit de ruido nuevo por bit de salida a la tasa a la que se leyó el generador.

Algunas medidas afinarían aún la imagen del propio ESP32; se dejaron sin hacer porque ninguna puede cambiar esa conclusión. La fuente SRAM necesita decenas de encendidos, a varias temperaturas, para trazar la probabilidad de cambio de cada celda: tanto para acotar lo que aportan las celdas inestables como para dimensionar la corrección de errores de un extractor difuso que convertiría las celdas estables en una clave del dispositivo, una función físicamente no clonable (PUF) de SRAM que ligaría cada semilla a su chip [65]. Un estudio de temperatura necesita un sensor calibrado y una cámara climática, de -20 a $+85$ °C. Las semillas producidas en el dispositivo deberían capturarse y analizarse, y no solo reproducirse fuera de línea. Y la prueba de reinicios, la única medida que separó las condiciones, debería repetirse en un chip posterior como el ESP32-S3, cuyo generador documenta la guía de software con otras tasas de lectura (Sección 3.1).

Lo que falta no es una prueba mejor sino un tipo distinto de conocimiento: un modelo de la fuente — qué proceso físico produce la aleatoriedad, y con qué parámetros — y una medida de esos parámetros que acote la min-entropía por debajo. Ninguna de las dos cosas puede obtenerse desde fuera del ESP32: sus fuentes de ruido, su estado interno y su posprocesado no están documentados, y los estimadores de SP 800-90B se diseñaron para muestras de ruido en bruto, no para la salida de un bloque que puede estar ya acondicionado. Por eso la segunda parte de este trabajo construye el generador en lugar de medir el de otro. El Capítulo 4 parte de la física de un oscilador en anillo, extrae su jitter y lo convierte, mediante un modelo validado, en una cota inferior de la min-entropía; los capítulos siguientes construyen y verifican el motor en torno a esa fuente. El ESP32

sigue en el trabajo con otros dos papeles: su cifra no IID es la referencia de la comparación de la Sección 8.7, y su biblioteca criptográfica es la implementación independiente con la que se verifica el motor.

Capítulo 4

Fuente de entropía: del transistor a la min-entropía

Este capítulo construye la cadena sobre la que descansa el resto del TFM: del jitter físico de un oscilador en anillo a una cifra defendible de min-entropía por bit de salida, y de esa cifra a los parámetros de diseño del muestreador. Cada paso se valida antes de usarse. Los estimadores estadísticos se validan con datos sintéticos de jitter conocido; las cotas analíticas se validan frente a un cálculo numérico exacto; y el banco a nivel de transistor se valida midiendo su propia resolución antes de medir nada con él. Todas las cifras de este capítulo son [SIMULADO] salvo que se indique lo contrario; la campaña de hardware se presenta en el Capítulo 8.

4.1. El oscilador en anillo elemental

La fuente de entropía es un oscilador en anillo elemental (ERO): un oscilador en anillo libre RO1 cuya salida se muestrea con un flip-flop que recibe como reloj un segundo oscilador en anillo RO2 dividido por un entero K_D [10, 13]. Entre dos muestras, la fase de RO1 acumula jitter durante K_D periodos de RO2; cuando el jitter acumulado es comparable al periodo de RO1, el bit muestreado se vuelve impredecible. Se eligió el ERO frente a alternativas de mayor tasa de bits (XOR de varios anillos, muestreo coherente, anillos de efecto de transición) por una sola razón: es la arquitectura para la que existe un modelo estocástico con una cota de entropía demostrable y para la que se ha publicado y validado un método embebido de medida del jitter [8]. La tasa de bits no es un requisito aquí —el motor solo consume entropía para dar semilla a un generador determinista—, mientras que una cota demostrable sí lo es.

Una decisión de diseño que se deriva de la plataforma es que *el oscilador de la placa nunca se usa como referencia*. La fuente de reloj de la Basys 3 es un oscilador MEMS con un lazo de enganche de fase (PLL) fraccional cuya hoja de características especifica un

jitter ciclo a ciclo de hasta 95 ps [ESTIMADO] [123], entre uno y dos órdenes de magnitud por encima del jitter por periodo de un oscilador en anillo en un proceso de 28 nm (2–4 ps [ESTIMADO] [8]). Muestrear RO1 con ese reloj mediría el ruido de fase del PLL, una componente global y potencialmente inducible, en lugar del ruido térmico del anillo. Por eso RO2 muestrea a RO1, y el reloj de sistema de 100 MHz solo gobierna la lógica síncrona.

4.2. Modelo estocástico y factor de calidad

Siguiendo a Baudet *et al.* [10], la fase $\varphi(t)$ de RO1, medida en periodos, se modela como un proceso de Wiener con deriva μ y volatilidad σ^2. Entre dos muestras separadas Δt, la fase avanza en promedio $\nu = \mu\,\Delta t$ periodos y acumula una varianza

$$Q = \sigma^2\,\Delta t, \tag{4.1}$$

el *factor de calidad*. El bit de salida es $s = 1$ cuando la fase fraccionaria está en $[1/2, 1)$. Condicionado a la fase anterior φ_0, el bit sigue una distribución de Bernoulli con

$$P(s = 1 \mid \varphi_0) = \sum_{k \in \mathbb{Z}} \left[\Phi\!\left(\tfrac{k + 1 - \varphi_0 - \nu}{\sqrt{Q}} \right) - \Phi\!\left(\tfrac{k + \frac{1}{2} - \varphi_0 - \nu}{\sqrt{Q}} \right) \right], \tag{4.2}$$

donde Φ es la función de distribución normal estándar; se trata de la gaussiana envuelta evaluada de forma exacta. Su primer armónico da el sesgo máximo $\varepsilon_{\max} \approx (2/\pi)\, e^{-2\pi^2 Q}$, a partir del cual el Corolario 1 de Baudet proporciona la conocida expresión

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2}\, e^{-4\pi^2 Q} + O\!\left(e^{-6\pi^2 Q}\right). \tag{4.3}$$

4.3. Entropía condicional exacta frente a la cota asintótica

La ecuación (4.3) se cita habitualmente como una *cota inferior* de la entropía por bit. Leída con atención, la desigualdad del corolario solo afirma que la tasa de entropía de la secuencia es al menos la entropía de un bit condicionada a la fase anterior exacta; el miembro derecho es un desarrollo asintótico *de* esa entropía condicional, no una cota de ella. Desarrollando la entropía binaria en función del sesgo, $h_2(\tfrac{1}{2} + \varepsilon) = 1 - \frac{1}{\ln 2} \sum_{n \geq 1} \frac{(2\varepsilon)^{2n}}{2n(2n-1)}$, y promediando sobre una φ_0 uniforme con $\varepsilon = A \sin(2\pi\varphi)$, $A = (2/\pi) e^{-2\pi^2 Q}$, se obtiene

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2}\, e^{-4\pi^2 Q} - \frac{8}{\pi^4 \ln 2}\, e^{-8\pi^2 Q} - \cdots \tag{4.4}$$

Todos los términos omitidos son negativos, así que *cualquier* truncamiento sobrestima la entropía; el término siguiente decae como $e^{-8\pi^2 Q}$, no como $e^{-6\pi^2 Q}$. La Tabla 4.1 compara el valor exacto, calculado a partir de (4.2) sobre una malla de 8192 fases, con los desarrollos de primer y segundo orden. Añadir el término de segundo orden reduce el error de (4.3) en un factor de 95 para $Q = 0{,}10$ y de 4970 para $Q = 0{,}20$, lo que confirma la deducción.

Tabla 4.1: Entropía de Shannon condicional por bit: cálculo exacto con la gaussiana envuelta frente a desarrollos truncados [SIMULADO].

Q	Exacta	Orden 2	Orden 1 (Ec. 4.3)	Error del orden 1
0,02	0,7013194	0,7100952	0,7345213	$+3{,}3 \times 10^{-2}$
0,05	0,9163001	0,9164920	0,9187783	$+2{,}5 \times 10^{-3}$
0,10	0,9886728	0,9886733	0,9887174	$+4{,}5 \times 10^{-5}$
0,15	0,9984319	0,9984319	0,9984327	$+8{,}5 \times 10^{-7}$
0,20	0,9997823	0,9997823	0,9997823	$+1{,}6 \times 10^{-8}$

En la región de funcionamiento ($Q \approx 0{,}2$) la discrepancia es de 10^{-8} e irrelevante; para un generador infradimensionado ($Q \approx 0{,}02$) la fórmula truncada atribuye un 3,3 % de entropía que no existe. Por ello, este TFM usa el cálculo exacto como valor conservador y cita (4.3) como lo que es: una aproximación asintótica excelente para $Q \gtrsim 0{,}1$. La misma herramienta numérica muestra que la min-entropía obtenida a partir del sesgo del primer armónico, $H_\infty = -\log_2(\frac{1}{2} + \varepsilon_{\text{máx}})$, coincide con el valor exacto del peor caso con un error inferior a 10^{-4} bit en $Q \in [0{,}05;\ 0{,}30]$.

4.3.1. Objetivo de diseño

La versión 3.0 de AIS 20/31 [5] admite dos criterios alternativos para la clase PTG.2: entropía de Shannon $\geq 0{,}9998$ o min-entropía $\geq 0{,}98$ por bit interno. Resolver ambos por bisección sobre las entropías exactas da $Q \geq 0{,}2022$ y $Q \geq 0{,}2286$, respectivamente. El criterio de min-entropía es el más exigente y es también la magnitud con la que trabaja NIST SP 800-90B, por lo que es el que se adopta aquí: **el objetivo de diseño es $Q \geq 0{,}23$, con un punto de consigna de $Q = 0{,}30$** ($H_\infty = 0{,}9951$), con un margen que cubre la reducción del jitter térmico a baja temperatura, que es la prueba adversaria estándar de Fischer y Lubicz [13].

4.4. Estimador embebido del jitter y sus dos correcciones

Para elegir K_D hay que conocer σ^2, y el método de Fischer y Lubicz [13] la mide a partir de los propios bits: con $K_D = 1$, la fracción c de pares (b_j, b_{j+M}) que difieren dentro de un bloque de N pares estima la fase acumulada a lo largo de M muestras, y su varianza

entre K bloques crece como $4MQ_{\text{bruto}}$. Implementado al pie de la letra, con un ajuste a una recta, el estimador *subestima* Q_{bruto} entre un 13 % y un 38 % según el nivel de jitter. Se identificaron dos causas sobre datos sintéticos con Q conocido, y se corrigieron:

1. **Efecto de vértice.** c no es lineal en la fase acumulada; es una onda *triangular* de periodo uno y pendiente ± 2. Cuando la fase determinista $M\nu$ mód 1 cae cerca de un vértice, la distribución queda a ambos lados de él y la varianza se desploma —hasta 0,38 de su valor nominal en un caso medido—. El vértice se detecta a partir de la propia media medida ($\bar{c} \approx 0$ o 1 es un vértice; $\bar{c} \approx \frac{1}{2}$, el centro de la rampa), de modo que las distancias M con $\bar{c}$ fuera de [0,15; 0,85], o para las que $3\sqrt{MQ}$ alcanza el vértice, se descartan de forma iterativa.

2. **Ruido de estimación no constante.** Estimar una fracción a partir de N pares añade una varianza proporcional a $\bar{c}(1-\bar{c})$, que cambia con M. Ajustarla como una ordenada en el origen constante sesga la pendiente; en su lugar se ajusta como un segundo regresor, $\mathrm{Var}(c) = Q\,(4M) + \kappa\,\bar{c}(1-\bar{c})/N$.

Tras ambas correcciones, el estimador recupera el Q_{bruto} inyectado dentro de un ± 2 % en un rango de 150× (Tabla 4.2), con un sesgo de $-0{,}6$ % y una dispersión de 4,0 % sobre ocho semillas independientes para $K = 20\,000$ bloques. El exponente de acumulación ajustado es 1,009 para un simulador sin ruido de parpadeo; medido en hardware, el mismo exponente separa la componente térmica ($b = 1$, que cuenta como entropía) del ruido de parpadeo y el jitter global ($b = 2$, que no cuenta).

Tabla 4.2: Recuperación de un Q_{bruto} conocido mediante el estimador embebido corregido ($K = 20\,000$ bloques de $N = 100$ pares) [SIMULADO].

Q_{bruto} inyectado	Estimado	Error	Valores de M usados
$2{,}0 \times 10^{-7}$	$1{,}979 \times 10^{-7}$	$-1{,}07$ %	19
$5{,}0 \times 10^{-7}$	$5{,}060 \times 10^{-7}$	$+1{,}21$ %	18
$2{,}0 \times 10^{-6}$	$2{,}037 \times 10^{-6}$	$+1{,}86$ %	16
$8{,}0 \times 10^{-6}$	$8{,}039 \times 10^{-6}$	$+0{,}48$ %	9
$3{,}0 \times 10^{-5}$	$2{,}977 \times 10^{-5}$	$-0{,}78$ %	5

Una consecuencia práctica para el hardware es que los bloques no tienen por qué ser contiguos: cada uno requiere solo $N + M$ bits consecutivos, y los bloques independientes dan una estimación algo mejor (error de $-0{,}30$ % con 20 000 fragmentos independientes). Por eso el búfer de captura de la FPGA contiene un único fragmento de unos pocos kilobits, que se llena a la velocidad del anillo y se lee despacio, en lugar de megabits de muestras contiguas.

4.5. Banco a nivel de transistor y su resolución

La fuente de entropía es el único bloque del motor ligado a la tecnología, y en un circuito integrado se diseña a nivel de transistor. Se construyó un banco SPICE en torno a un anillo de cinco etapas con una puerta NAND de habilitación, con todos los parámetros dependientes de la tecnología (modelos de dispositivo, anchuras, longitud mínima, alimentación) confinados en una cabecera, de modo que pasar de los modelos genéricos provisionales usados aquí al kit de diseño del proceso (PDK) de una fundición cambia un único bloque de la netlist. El jitter se extrae de los instantes de cruce por el umbral de la salida mediante la segunda diferencia $D(M) = t_{k+2M} - 2t_{k+M} + t_k$, es decir, la varianza de Allan aplicada a datos de fase. Su esperanza, $2M\sigma^2$, está libre del sesgo que arrastra una diferencia simple sobre ventanas solapadas (medido en un 5 %), y es el método que usan Benea *et al.* para caracterizar anillos tanto en Artix-7 como en silicio de 28 nm [32]. Las estimaciones a distintos M se combinan con pesos de varianza inversa proporcionales al número de ventanas independientes; sin esa ponderación, la dispersión se quedaba en el 15 % con independencia de la longitud simulada.

El banco mide primero su propia resolución: con todas las fuentes de ruido desactivadas, el jitter residual es el ruido numérico del simulador. El primer intento dio 0,33 ps, del mismo orden que el jitter que se quiere medir; se debía por completo al transitorio de arranque del oscilador, y descartar los primeros veinte ciclos lleva el suelo de resolución a **1,38 fs**, tres órdenes de magnitud por debajo de la señal. Solo entonces se inyectó ruido. El jitter es proporcional a la raíz cuadrada de la densidad espectral inyectada ($\times$2,01 medido para $\times$4 en densidad, frente a $\times$2,00 esperado) hasta el punto en que la corriente de ruido eficaz alcanza aproximadamente el 80 % de la corriente de conducción de la etapa; ahí deja de valer el régimen de pequeña señal, el periodo medio se desplaza y la ley de escala falla por un factor de cinco. De ello se deduce un criterio de validez para toda campaña, incluidas las que se hagan sobre un PDK real: mantener el ruido inyectado por debajo de $\sim$ 40 % de la corriente de la etapa y comprobar que el periodo medio no se mueve al activar el ruido. Con los modelos provisionales el anillo oscila a 4,3 GHz, lo que no es físico; las cifras que entran en este TFM proceden de modelos de dispositivo reales.

Los primeros modelos de ese tipo son los del PDK abierto IHP SG13G2 (130 nm, núcleo de 1,2 V, modelo compacto PSP 103.6, licencia Apache-2.0) [115], que pueden publicarse junto con todo lo demás. Hay dos diferencias respecto al banco con modelos provisionales. Primero, los modelos están escritos en Verilog-A y hay que compilarlos a la interfaz OSDI; esto obligó a compilar ngspice 47 y el compilador OpenVAF-Reloaded en espacio de usuario, ya que ningún canal de paquetes ofrecía versiones lo bastante recientes. Segundo, y más importante, el ruido ya no se elige: ngspice no inyecta el ruido de los dispositivos durante un análisis transitorio, así que la densidad de las fuentes de corriente se toma de un análisis de ruido en pequeña señal de los mismos transistores, un inversor

polarizado en su punto de conmutación con la entrada unida a la salida, donde ambos dispositivos conducen y el ruido desplaza el instante de cruce. El ruido referido a la entrada de ese nodo es exactamente la corriente que debe reproducir la fuente del transitorio. Su espectro es un término $1/f$ puro más un suelo de ruido blanco de $4{,}13 \times 10^{-24}$ A^2/Hz, plano de 100 kHz a 10 GHz, con la frecuencia de esquina del $1/f$ en 19,8 MHz y un factor de ruido efectivo $\gamma = 1{,}04$, por encima del 2/3 de canal largo, como se espera en un dispositivo corto. Antes de usarla, la parte $1/f$ de la fuente de ruido de ngspice se calibró frente a una resistencia conocida: su densidad es $\mathrm{NAMP}^2/(\pi f)$ con un margen del 0,4 %. Los resultados están en la Sección 8.3.

4.6. Objetivo de diseño y selección del divisor

Con σ/T medido por periodo, $Q_{\mathrm{bruto}} = (\sigma/T)^2$ y el divisor se obtiene de $K_D = \lceil Q_{\mathrm{objetivo}}/Q_{\mathrm{bruto}} \rceil$. Para el rango de jitter esperado en un proceso de 28 nm, el divisor queda entre 3×10^4 y 5×10^5, y la tasa de bits por anillo entre 1 y 20 kbit/s. El modelo reproduce el punto de diseño publicado por Fischer y Lubicz ($K_D \approx 430\,000$ para $\sigma = 5{,}01$ ps sobre $T = 7{,}81$ ns) [13]. Estas tasas bastan aquí: dar semilla al generador determinista con 256 bits de entropía completa requiere unos 512 bits brutos con $H_\infty \approx 0{,}98$, es decir, entre 0,03 y 0,5 s por semilla, y el motor solo emite una semilla en el arranque y en los intervalos de resembrado. Si alguna vez hiciera falta más tasa de bits, el camino limpio son varios EROs independientes, cada uno con su propia cota, en lugar de un XOR de varios anillos cuya independencia no puede demostrarse [16].

Capítulo 5

Motor digital

El motor digital convierte los bits en bruto de la fuente de entropía en claves y las usa, y está escrito de forma que el mismo nivel de transferencia de registros (RTL) funcione en la FPGA usada para la verificación y, más adelante, en celdas estándar. Nada en él depende de un fabricante: ni primitivas, ni macros de memoria, ni bloques de gestión de reloj. El único bloque dependiente de la tecnología en todo el diseño es la pareja de osciladores en anillo del Capítulo 4.

5.1. Arquitectura y la decisión de una única primitiva

La Figura 5.1 resume la ruta de datos. Los bits en bruto salen del ERO en el dominio de reloj del anillo de muestreo, cruzan al reloj del sistema de bit en bit y alimentan a tres consumidores: los tests de salud, el colector de semilla del generador determinista y un búfer de captura que solo se usa para la caracterización. Las claves salen de un generador determinista de bits aleatorios en modo contador (CTR_DRBG); la clave actual se carga en un bloque AES-CMAC que responde a retos; y un esclavo bus entre circuitos integrados (I2C) lo expone todo como un mapa de registros.

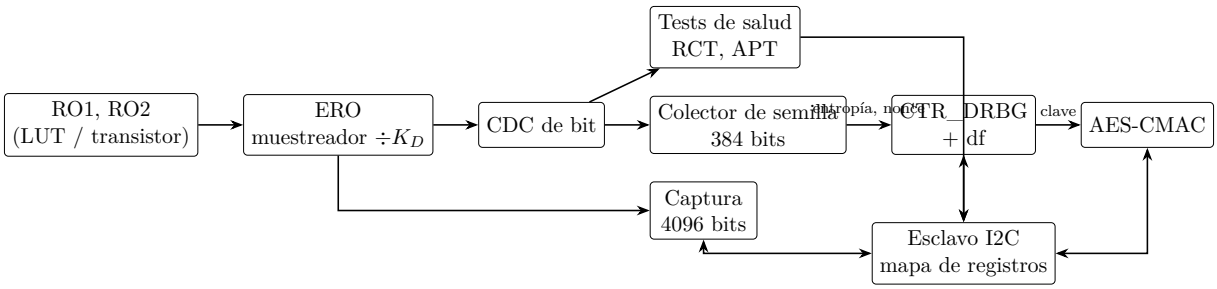


Figura 5.1: Ruta de datos del motor. Solo los osciladores en anillo dependen de la tecnología; todos los demás bloques son RTL portable.

La decisión que da forma a todo el motor es usar **una sola primitiva criptográfica,**

AES-128, para las tres tareas: acondicionar la entropía, generar claves y autenticar. La primera arquitectura tenía un núcleo SHA-256 como acondicionador; las estimaciones de síntesis del Capítulo 7 mostraron que ocuparía casi la mitad del área digital, y SP 800-90B incluye construcciones basadas en AES — CMAC, CBC-MAC y la función de derivación basada en cifrado de bloque de SP 800-90A — entre sus componentes de acondicionamiento *aprobados* [6]. Prescindir del hash lleva el motor de un valor estimado de ∼25 a ∼13 puerta equivalentes (GEs) (miles), a costa de pasar de un CTR_DRBG *sin* función de derivación, que necesita una entrada con entropía completa, a uno *con* ella, que es la configuración habitual en la práctica y admite directamente entradas sin entropía completa. El núcleo AES solo cifra: el modo contador y CMAC nunca descifran, y ofrecer el descifrado como servicio al ESP32 no tendría sentido, ya que el ESP32 tiene su propio acelerador. La verificación cruzada no se ve afectada: el motor cifra y el ESP32 descifra con mbedtls.

5.2. Núcleo de cifrado AES-128

El núcleo es iterativo, una ronda por ciclo de reloj, once ciclos por bloque (suma inicial de clave más diez rondas), con las claves de ronda expandidas al vuelo a partir de la anterior — el cifrado solo necesita la expansión hacia delante, y así se evita almacenar once claves de ronda. Un segundo núcleo con el mismo interfaz y ruta de datos de 32 bits (cuatro S-boxes compartidas con la expansión de clave, una columna por ciclo, 51 ciclos por bloque) se escribió después para el capítulo del circuito integrado y es el que el motor instancia ahora por defecto; el de una ronda sigue siendo la referencia contra la que el banco lo compara (Capítulo 7). A 100 MHz eso supone 1,16 Gbit/s de tasa de bits del núcleo, muy por encima de lo que puede transportar cualquier enlace I2C, así que segmentar el núcleo (pipelining) sería área desperdiciada; en su lugar se elige la forma iterativa porque una ronda es una función combinacional pura que puede comprobarse directamente contra la norma [78]. La S-box es una tabla de 256 entradas sintetizada como lógica, MixColumns se escribe como la red `xtime` habitual, y el estado sigue la convención por columnas de FIPS 197 con el byte 0 a la izquierda, de modo que las cadenas hexadecimales de la norma se corresponden una a una con los vectores VHDL.

El núcleo se verificó con los cinco vectores oficiales disponibles para el cifrado AES-128 (apéndices B y C.1 de FIPS 197, y los tres bloques ECB de SP 800-38A F.1.1) y con una cadena de mil cifrados para poner de manifiesto cualquier estado residual entre bloques. Una comprobación que no estaba prevista resultó ser la más convincente: el cifrado del bloque todo ceros con la clave de ejemplo de SP 800-38A dio `7df76b0c...546f`, que es exactamente el valor intermedio L publicado por el NIST en sus ejemplos de CMAC [124].

5.3. AES-CMAC

AES-CMAC [92, 93] es la primitiva del reto-respuesta: el ESP32 envía un nonce y el motor devuelve su etiqueta con la clave actual. El bloque calcula las subclaves K_1, K_2 una sola vez al cargar una clave (un cifrado del bloque cero y dos desplazamientos condicionales a la izquierda con la constante polinómica `0x87`) y después ejecuta CBC-MAC sobre el mensaje, aplicando K_1 si el último bloque está completo o rellenando con `0x80` y aplicando K_2 en caso contrario. Se eligió frente a HMAC-SHA-256 porque reutiliza la única primitiva y cuesta un cifrado por bloque frente a dos compresiones del hash. El bloque supera los cuatro vectores de la RFC 4493 (mensaje vacío, un bloque, 40 y 64 bytes), que ejercitan las dos ramas de relleno.

5.4. CTR_DRBG con función de derivación

El generador implementa CTR_DRBG con AES-128 y la función de derivación basada en cifrado de bloque de SP 800-90A [89]. Su formato de entrada es fijo a propósito: 256 bits de entropía en bruto y 128 bits de nonce, de modo que $S = L \,\|\, N \,\|\, E \,\|\, \mathit{nonce} \,\|\, \texttt{0x80} \,\|\, 0\ldots$ ocupa exactamente cuatro bloques y la función de derivación se convierte en una secuencia cerrada de catorce cifrados; una entrada de longitud variable costaría una ruta de datos orientada a bytes sin ningún beneficio. La instanciación (o el resembrado) cuesta 14 operaciones AES, 183 ciclos; generar 256 bits cuesta cuatro, 53 ciclos. El estado (Key, V) nunca se expone; el contador de resembrados es legible para que el anfitrión pueda imponer una política de resembrado muy por debajo del límite de 2^{48} de la norma.

La verificación se hizo en dos pasos. Primero, se escribió en Python un modelo de referencia exacto al bit — con su propio AES, independiente del VHDL — y se ejecutó contra los vectores del CAVP: 960 casos AES-128 con y sin función de derivación, con y sin resembrado, todos superados. Segundo, ese modelo generó las salidas esperadas para el formato de entrada fijo del hardware, y el VHDL las reprodujo: dos generaciones consecutivas, un resembrado seguido de una generación, el contador de resembrados y el hecho de que una nueva instanciación borra el estado anterior.

5.5. Tests de salud

Los dos tests aprobados por SP 800-90B se ejecutan de forma continua sobre los bits en bruto [6]. El test de recuento de repeticiones (*Repetition Count Test*) falla cuando llegan seguidas $C_{\mathrm{RCT}} = 1 + \lceil -\log_2 \alpha / H \rceil$ muestras idénticas; el test de proporción adaptativa (*Adaptive Proportion Test*) cuenta, en una ventana de $W = 1024$ muestras (el valor para fuentes binarias; 512 es para fuentes no binarias), cuántas repiten la primera, y falla al llegar a $C_{\mathrm{APT}} = 1 + \mathrm{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$. Para el objetivo de diseño $H = 0{,}98$ y

$\alpha = 2^{-20}$ resultan $C_{\mathrm{RCT}} = 22$ y $C_{\mathrm{APT}} = 596$; ambos son parámetros genéricos (generics), porque H se fija a partir de la min-entropía *medida*, no se supone. El test de proporción comprueba también el complemento de la cuenta, algo que la norma permite para fuentes binarias: de lo contrario, una fuente atascada en el valor opuesto a la primera muestra de la ventana puntuaría cero y pasaría. Las alarmas quedan enclavadas hasta que se borran explícitamente, y mientras haya alguna alarma activa no sale nada del motor: el secuenciador se niega a cargar la semilla, a generar clave y a producir salida aleatoria. Dos reglas más cierran el hueco entre un test continuo y una semilla de 384 bits. Primero tiene que pasar un test de arranque (SP 800-90B §4.3): una ventana completa del test de proporción, 1024 muestras, cerrada sin alarma tras el reset o tras borrar las alarmas, y borrarlas reinicia la ventana. Y una semilla solo se acepta cuando se ha cerrado sin alarma la ventana que contiene su último bit; la semilla es más corta que la ventana, así que de otro modo entraría en el generador antes de que el test de proporción la hubiera juzgado.

Cuatro reglas más salieron de revisar el secuenciador contra el mapa de registros y no contra la norma. El buffer de captura de bits en bruto registra los mismos bits de los que se recoge la semilla, de modo que una captura lanzada durante una siembra dejaba la semilla legible en modo de prueba; ahora capturar es una orden de modo de prueba que se rechaza con el motor ocupado, y la siembra se rechaza mientras hay una captura en curso. El contador de peticiones del generador determinista de bits aleatorios (DRBG) ya no da la vuelta: en su tope de 32 bits el motor se niega a generar hasta que se resiembra y lo indica en un bit de diagnóstico, que es el límite por hardware de la política de resembrado que el anfitrión sigue teniendo que aplicar mucho antes. Una orden de borrado total pone a cero la clave, el estado y el contador del generador, el reto, el registro aleatorio y la semilla recogida, y carga la clave nula en el autenticador; se atiende incluso con alarma, y después hay que volver a sembrar. Y el divisor de muestreo se reinicia a su máximo y no a un valor pequeño, de modo que un anfitrión que olvide configurarlo siembra a la tasa más lenta y con más entropía por bit, nunca por debajo del ajuste caracterizado.

El banco de pruebas inyecta los modos de fallo para los que existen los tests: una fuente sana supera veinte ventanas (peor racha 13, peor proporción 540); una fuente atascada dispara el test de repeticiones exactamente en la muestra 22; una fuente sesgada al 70 % dispara el test de proporción en 713 sin disparar el test de repeticiones; y una fuente atascada en el complemento se detecta. La primera fuente "sana" del banco — un registro de desplazamiento inicializado con un único uno — fue detectada por los propios tests, con una racha inicial de 27 ceros y una primera ventana al 76 %. El detector tenía razón y el banco estaba mal; ahora la fuente parte de una semilla densa y se calienta antes de usarse.

5.6. Ruta de captura y cruces de dominios de reloj

Los bits en bruto se producen en el dominio de reloj de RO2 y se consumen en el dominio del sistema. La ruta serie, bit a bit, hacia los tests de salud y el colector de semilla usa un sincronizador por conmutación (toggle): el origen registra el bit e invierte una bandera; el destino sincroniza la bandera a través de tres flip-flops y lee el bit en el flanco de la bandera. Requiere que cada bit dure al menos ocho ciclos del sistema, es decir, $K_D \geq 64$ para un anillo de muestreo de ∼700 MHz a 100 MHz, lo que el modo generador cumple por órdenes de magnitud. La ruta de caracterización no cruza bit a bit: el búfer de captura se escribe en el dominio del anillo a plena velocidad en una memoria de 4096 bits y se lee después en el dominio del sistema, con las dos direcciones del protocolo de acuerdo (handshake) transportadas por banderas de conmutación. La escritura y la lectura nunca se solapan, así que no hace falta una FIFO asíncrona, y el tamaño del búfer se deduce de la Sección 4.4: un fragmento de $N + M_{\text{máx}}$ bits es todo lo que necesita el estimador.

5.7. Interfaz de registros I2C y secuenciador de órdenes

La interfaz del producto es I2C, como en los elementos seguros comerciales: dos pines, direccionable y el bus que el ESP32 usa para sus periféricos. El esclavo implementa el protocolo de puntero de registro con autoincremento, un filtro de pulsos espurios (glitches) de cinco muestras en ambas líneas, como exige la especificación, y salidas en drenador abierto. No alarga el reloj (clock stretching): a 400 kHz hay 250 ciclos del sistema por bit, de sobra para cualquier acceso a registro, y las operaciones largas (carga de semilla, generación) se lanzan como órdenes y se consultan por sondeo a través de un registro de estado, que es como se comportan los dispositivos comerciales. El esclavo se probó con un maestro funcional de bus (bus-functional); aparecieron dos defectos reales — el puntero no se autoincrementaba en las escrituras de varios bytes, y el bit más significativo de cada byte leído se generaba un flanco de reloj tarde, de modo que todas las lecturas habrían salido desplazadas — y se corrigieron.

El mapa de registros (Apéndice A) expone la identificación, el estado y el control, la configuración del divisor y de la página de captura, los diagnósticos de los tests de salud, el contador de resembrados, el reto y su etiqueta, un registro de salida aleatoria de 256 bits y, *solo mientras está activo un pin de prueba*, la clave actual y el búfer de captura. El registro de salida aleatoria se llena mediante su propia orden con una generación separada del DRBG, nunca con la que produjo la clave: una versión anterior del diseño dejaba en ese registro los 256 bits de la llamada que generaba la clave, legibles sin el pin de prueba, y así filtraba la clave. La revisión que lo detectó se describe en el Capítulo 6. En la FPGA el pin de prueba es un interruptor; en un circuito integrado sería un fusible que se funde

en producción, porque exponer la clave o la entropía en bruto es una vulnerabilidad. Un pequeño secuenciador serializa las órdenes: la carga de semilla recoge 384 bits en bruto, espera a que los tests de salud los hayan evaluado todos, aborta si mientras tanto salta una alarma de salud, e instancia o resiembra el generador; la generación toma 256 bits, guarda los 128 primeros como clave, descarta el resto y recarga las subclaves de CMAC; una orden aparte llena el registro de salida aleatoria a partir de una generación independiente; la autenticación ejecuta el CMAC sobre el bloque de reto.

El motor completo se ejercitó tal como lo hará el ESP32: un maestro I2C funcional de bus lee la identificación, configura el divisor, carga la semilla, genera, lee la clave en modo de prueba, escribe dos retos y lee sus etiquetas, vuelve a generar, captura bits en bruto y verifica que la clave no puede leerse sin el pin de prueba — dieciséis comprobaciones. Lo que un banco VHDL no puede saber es si las etiquetas son *correctas*, porque la clave procede del ruido de los osciladores en anillo; el banco vuelca la clave, los retos y las etiquetas a un fichero y un script de Python recalcula los CMAC con el AES independiente. Coinciden. Es la misma comprobación cruzada que el ESP32 realiza en hardware con mbedtls.

Capítulo 6

Verificación

6.1. Estrategia: vectores, simulación, verificador independiente

Un motor criptográfico que solo se verifica contra sí mismo no demuestra nada: un error en la S-box se reproduciría idéntico en el modelo usado para comprobarlo. Por eso, la verificación de este trabajo se apoya en tres pilares que no comparten implementación.

1. **Vectores oficiales.** Cada primitiva se comprueba frente a los vectores de prueba publicados con su norma: FIPS 197 y SP 800-38A para AES, RFC 4493 y el fichero de ejemplos del NIST para CMAC, y los ficheros de respuesta del CAVP para CTR_DRBG (con y sin función de derivación, con y sin resembrado). Son pruebas de respuesta conocida y zanjan la corrección del algoritmo.

2. **Modelos de referencia independientes.** Se escribió en Python un modelo exacto al bit del generador, con su propio AES, derivado de la norma sin mirar el VHDL. Primero se valida frente a los vectores del CAVP, y solo entonces se usa para generar los valores esperados para la disposición fija de entradas del hardware y para recalcular las etiquetas del banco de pruebas de sistema. Dos implementaciones que coinciden valen más que una que pasa las pruebas.

3. **Un verificador independiente en el lazo.** En hardware, el ESP32 recalcula con mbedtls, y con su propio acelerador AES, cada resultado que produce el motor, de modo que la comprobación final la realiza una tercera implementación, sin relación con las otras, sobre un bus real.

Dos reglas de trabajo se aplican en todo momento, ambas aprendidas en el estudio de caja negra del ESP32 con el que empieza este trabajo: la salida de una compilación, simulación o medida nunca se filtra antes de haberla leído entera — un filtro oculta justo el aviso que importaba — y los fallos se registran completos, nunca se descartan. Dos

veces en este trabajo un filtro sobre el log del simulador ocultó un resultado, y las dos veces la solución fue quitar el filtro.

6.2. Modelos de referencia en Python

`ctr_drbg_ref.py` implementa el cifrado AES-128, la función de derivación basada en cifrado por bloques, CBC-MAC y CTR_DRBG con y sin función de derivación, en Python puro y a partir del texto de SP 800-90A. El AES comprueba con aserciones los dos vectores de FIPS 197 al importarse; el DRBG se prueba frente a los ficheros de respuesta del CAVP siguiendo el procedimiento CAVS (instanciar, resembrado opcional, dos llamadas de generación, comparar la segunda): **960 casos AES-128, todos superados**. Se encontró un defecto en el analizador de ficheros y no en el modelo — una cabecera de sección debe cerrar el caso de prueba pendiente antes de que cambien los parámetros; si no, el último caso de cada bloque de parámetros se evalúa con los parámetros del bloque siguiente — y merece la pena dejarlo anotado porque su síntoma, un único fallo en count 14 de cada bloque, parecía exactamente un error del algoritmo.

Un segundo script, `gen_drbg_vectores.py`, hace pasar al modelo validado por la secuencia que admite el hardware (instanciar con 32 bytes de entropía y 16 de nonce, generar dos veces, resembrar, generar) y emite un paquete VHDL con los valores esperados; si alguna vez cambia la disposición de las entradas del hardware, ese script es el único sitio donde hay que cambiarla. Un tercero, `verifica_top.py`, recalcula AES-CMAC a partir del mismo AES y se valida a sí mismo frente a RFC 4493 antes de comprobar nada.

6.3. Bancos de pruebas y resultados

Todo el VHDL se simula con GHDL 6 mediante un único script de ejecución que analiza las fuentes en orden de dependencias y ejecuta cada banco, imprimiendo la salida del simulador sin filtrar. La Tabla 6.1 resume los siete bancos y su estado en el momento de la redacción.

Parte de lo que encontraron los bancos merece contarse, porque un capítulo de verificación que solo informa de aciertos no ha verificado gran cosa.

Modelo del anillo. Las primitivas LUT usadas para simular el anillo se modelaron al principio con una regla estricta de propagación de valores desconocidos: cualquier entrada desconocida daba una salida desconocida. Al comienzo de una simulación todas las señales son desconocidas, así que el anillo nunca salía de ese estado y nunca arrancaba. Una NAND real con su entrada de habilitación a cero da un uno a la salida sea cual sea la otra entrada; el modelo ahora evalúa todas las combinaciones compatibles con las entradas conocidas y

Tabla 6.1: Bancos de pruebas, qué comprueban y su resultado (GHDL 6.0, VHDL-2008).

Banco	Comprobaciones	Resultado
Oscilador en anillo	arranque/parada, $f = 1/(2Nt_{pd})$, suelo nulo, $\sigma_T = \sqrt{2N}\,\sigma_{\text{stage}}$	6/6
Esclavo I2C	direccionamiento, dirección ajena, escritura multibyte, lectura con inicio repetido; bus bajo abuso: espurios bajo el filtro, parada e inicio repetido a mitad de byte, bytes tras dirección ajena, puntero a través de la parada y vuelta, maestro perdido con SDA retenido, parada tras ACK de lectura, 1 MHz y 100 kHz; 1 propiedad	49/49
Núcleos AES-128 (los dos)	FIPS 197 B, C.1; SP 800-38A F.1.1 (3 bloques); cadena de 1000 bloques; zeroización a mitad de bloque; el de una ronda por ciclo y el de 32 bits coinciden	18/18
AES-CMAC	RFC 4493 ejemplos 1–4; repetibilidad	5/5
CTR_DRBG	instanciación, 2×generación, resembrado, contador, reinstanciación, rechazo sin semilla, zeroización en reposo y a mitad, contador de peticiones que satura	18/18
Tests de salud	fuente sana, atascada, sesgada al 70 %, atascada en el complemento, retención/borrado	15/15
Motor completo	secuencia del ESP32 por I2C: id, semilla (divisor por debajo del mínimo rechazado, divisor congelado mientras está ocupado), clave, registro aleatorio libre de la clave, 2 retos, 2.ł clave, captura (la semilla nunca en el buffer), clave oculta, agotamiento, zeroización y residuos; 11 propiedades, 6 coberturas	64/64

resuelve la salida cuando todas coinciden. Sin esto, todo el banco de pruebas de sistema habría sido imposible.

Esclavo I2C. Dos errores de diseño reales: el puntero de registro no avanzaba en las escrituras multibyte, de modo que los bytes consecutivos sobrescribían la misma dirección; y en las lecturas el bit más significativo se presentaba en el bus un flanco de reloj demasiado tarde, lo que desplazaba todos los bytes. El primero se corrigió incrementando el puntero un estado después del pulso de escritura, de forma que la dirección sigue siendo válida durante el pulso; el segundo, cargando el registro de desplazamiento y presentando su primer bit en el mismo flanco de bajada que libera el bit de reconocimiento. Un tercer defecto estaba en el propio banco: inicializar la memoria de registros desde el proceso de estímulos creaba un segundo driver sobre la misma señal, y todo bit en conflicto se resolvía a desconocido. El esclavo es la única puerta del chip, así que el banco se amplió después con el bus bajo abuso: pulsos espurios más cortos que el filtro de entrada en mitad de un byte, en SDA con SCL alto (un inicio seguido de una parada, de no filtrarse) y en SCL en su fase baja; una parada y un inicio repetido a mitad de byte; bytes enviados tras una dirección ajena; el puntero que sobrevive a la parada y da la vuelta al final del mapa; un maestro que desaparece con el reloj parado mientras el esclavo mantiene SDA a nivel bajo en su reconocimiento; un maestro que emite una parada tras reconocer un byte leído, cosa que el propio I2C hace imposible cuando el byte siguiente empieza por cero porque el esclavo retiene la línea; y el bus a 1 MHz y a 100 kHz con reloj de sistema de 20 MHz. El banco cuenta los pulsos de escritura al banco de registros, de modo que ninguna transacción abortada puede escribir, y una propiedad comprobada en cada ciclo establece que el esclavo nunca tira de SDA estando en reposo. La guarda de bus colgado suelta la línea en el número de ciclos configurado en los dos casos de bloqueo, y nada anormal escribe.

Tests de salud. La fuente "sana" del primer banco, un registro de desplazamiento inicializado con un solo uno, empezaba con una racha de 27 ceros y una primera ventana al 76 %; el detector saltó. El detector tenía razón.

Fuga de la clave por el registro aleatorio. El hallazgo más grave no salió de un banco, sino de releer el diseño antes de entregarlo. La orden que generaba la clave escribía la salida completa de 256 bits del DRBG en el registro de salida aleatoria, que se puede leer sin el pin de prueba; los primeros 128 de esos bits *eran* la clave. El banco de sistema incluso imponía esa igualdad mediante una aserción, como "comprobación". La corrección separa ambas cosas: la generación de la clave no deja nada legible, y una orden nueva llena el registro aleatorio a partir de una generación independiente, después de que el DRBG haya actualizado su estado. Ahora el banco comprueba con una aserción lo contrario, que

el registro no contiene la clave en ninguna de sus mitades, y la comprobación cruzada en Python hace lo mismo.

Motor completo. Con los anillos a su velocidad física, el banco de sistema, que genera casi un milisegundo de tráfico I2C, no terminaba en veinte minutos de CPU: cada transición del anillo cuesta dos sorteos aleatorios en el modelo de jitter. Solo en ese banco el retardo de etapa se fija diez veces mayor, con el mismo jitter relativo, y así se indica en el propio banco. Las dieciocho comprobaciones se superan, y la clave, los dos retos y sus etiquetas se escriben en un fichero que comprueba `verifica_top.py`: las dos etiquetas son iguales al CMAC recalculado por el AES independiente en Python, el registro de salida aleatoria legible no contiene la clave en ninguna de sus mitades, y dos generaciones dan claves distintas. Este es el primer punto en que una clave nacida del ruido de osciladores en anillo, acondicionada, generada y usada para autenticar, queda confirmada por una implementación que no comparte nada con el VHDL. Dos de las dieciocho comprobaciones se añadieron cuando dos obligaciones que el mapa de registros imponía al anfitrión se trasladaron al hardware: una orden de siembra emitida con el divisor por debajo de su mínimo ahora se rechaza, y una escritura en el registro del divisor mientras se ejecuta una orden se ignora. Una regla que solo impone la documentación es una regla que un anfitrión descuidado incumple; el banco ahora prueba ambas. Las reglas de seguridad del secuenciador están escritas además como propiedades PSL dentro del banco, comprobadas en cada ciclo de toda la simulación y no solo en los pasos que las provocan: ni generación ni siembra con una alarma activa, ninguna recogida de semilla antes del test de arranque, ninguna generación con el contador de peticiones agotado, captura y siembra mutuamente excluyentes y captura solo en modo de prueba, ninguna autenticación sin clave válida, el borrado total deja la clave a cero en el mismo ciclo, y el motor nunca se queda ocupado para siempre; cuatro directivas de cobertura confirman que el banco llega a los estados que esas propiedades vigilan. Un mutante del secuenciador que omite la comprobación de alarma al generar la clave lo detectan tanto el paso que espera el rechazo como la propiedad. Además, hubo que indicarle al banco que se detuviera: con los anillos funcionando tras el reset, una simulación cuyos estímulos han terminado nunca se queda sin eventos, y la primera repetición consumió cuarenta y cinco minutos de CPU después de haber superado su última comprobación.

6.4. Firmware verificador del ESP32

El firmware es una aplicación ESP-IDF 5 para el ESP32 clásico que se comunica con el motor a través del mapa de registros del Apéndice A y ofrece una consola por la UART. Sus órdenes se corresponden con el plan de verificación: `id` lee la identificación y el estado; `sembrar` y `generar` lanzan las órdenes largas y consultan el indicador de ocupado; en modo

de prueba, `generar` además relee la clave (aprovisionamiento) y comprueba que es igual a los dieciséis primeros bytes del bloque generado; `reto N` realiza N rondas de reto-respuesta con nonces de `esp_random()`, recalculando cada etiqueta con `mbedtls_cipher_cmac` y contando coincidencias y discrepancias, y las discrepancias se imprimen completas; `crudo N` vuelca N fragmentos de bits en bruto y `esp N` vuelca bloques del propio generador del ESP32, ambos en el formato en tramas protegido con CRC32 de las campañas de captura del ESP32 (Sección 3.6), para que la misma cadena de procesado estadístico trate las dos fuentes. CMAC no viene habilitado por defecto en la compilación de mbedtls de ESP-IDF y se activa en la configuración por defecto del proyecto; la consola funciona a 115 200 baudios, la única velocidad que resultó fiable con el puente USB de la placa en esas campañas, y el nivel de log se fija en solo errores para que las líneas de log no puedan corromper las tramas. La capa de acceso a registros (mapa, emisión de órdenes, sondeo en tiempo real, comprobación de rechazo) es un componente compartido con la aplicación de la Sección 6.5. El firmware compila con ESP-IDF 5.3.5 en una imagen de 231 kB; ejecutarlo queda a la espera del hardware.

Los dos firmwares se ejercitan además en el PC, sin placa: el banco incluye el código fuente del firmware sin cambios, lo enlaza con un modelo en software del motor que implementa el mapa de registros como una segunda lectura independiente de la especificación, y recorre la secuencia con el mismo árbol de mbedtls 3.6.7 que distribuye ESP-IDF. El AES y el CMAC del modelo se validan a sí mismos contra FIPS 197 y RFC 4493 antes de cualquier comprobación, para que un fallo del modelo no pase por un fallo del firmware. El banco comprueba cuentas, no solo resultados: que una orden rechazada no se confunde con una que acabó enseguida, que un fallo del bus se cuenta como fallo del bus y no como discrepancia criptográfica, que nada llega al PC con una alarma levantada y que los plazos duran lo que dicen.

6.5. El motor en un producto: lecturas firmadas

El verificador juzga al motor; un segundo firmware lo *usa*, como lo haría un producto. El producto es un medidor de energía genérico sobre un ESP32-C3, la parte RISC-V de los equipos pequeños con Wi-Fi, con el motor en el mismo bus I2C que los periféricos del medidor. Dos usos, los dos alimentados por la entropía del motor. Primero, el equipo genera su propio par de claves ECDSA P-256: mbedtls obtiene el escalar privado a través de una función que emite una orden `aleatorio` por cada bloque de 32 bytes, dos bloques por clave, y la clave privada nunca sale del chip; lo que sale es la clave pública, o una petición de certificado PKCS#10 firmada con ella, que es lo que un equipo enviaría a la CA del fabricante para que nadie, ni el fabricante, tenga nunca la clave privada. Segundo, cada lectura (identificador del equipo, número de secuencia, instante, energía acumulada, potencia; 24 bytes) va firmada, de modo que quien la recibe comprueba integridad y origen

sin fiarse del transporte ni del almacenamiento intermedio sin conexión.

Tres decisiones sostienen el diseño. La fuerza de la clave la acota lo que la generó: el DRBG del motor es CTR_DRBG con AES-128, 128 bits de seguridad, que es lo que ofrece P-256, así que una curva mayor no ganaría nada con esta semilla. Las firmas usan ECDSA determinista (RFC 6979): el nonce de cada firma se deriva de la clave y del mensaje, dos firmas de la misma lectura son idénticas (el banco lo comprueba) y un generador pobre en el momento de firmar no puede filtrar la clave, que es el fallo clásico de ECDSA; el motor solo hace falta, por tanto, para *crear* la clave, y un medidor cuyo motor está en alarma sigue firmando con la clave que tiene, mientras que no puede hacer una nueva. Y la función de entropía falla cerrada: si el motor rechaza la orden (sin sembrar, alarma, test de arranque no superado) o el bus falla, mbedtls aborta y no se sustituye ninguna otra fuente; una generación fallida deja en servicio la clave anterior y nunca una a medias. Un número de secuencia monótono permite al receptor rechazar lecturas reinyectadas, cosa que una firma válida por sí sola no hace.

El banco en el PC ejecuta el firmware del medidor contra el modelo del motor y entrega su salida de consola a un script del PC que verifica las tramas con otra biblioteca (`cryptography`, no mbedtls). El banco planta una lectura manipulada (un bit del campo de energía) y una repetida entre ocho válidas, y el script debe encontrar exactamente esas dos; la petición de certificado debe parsearse, llevar una autofirma válida y la clave pública del equipo. Todas las comprobaciones pasan. El firmware compila con ESP-IDF 5.3.5 para el ESP32-C3, y corre además en la emulación QEMU de Espressif de ese chip: sin motor, toda orden falla cerrada, y con el modelo del motor enlazado dentro del firmware en lugar del driver I2C (una opción de compilación, nunca para una placa) la secuencia completa corre en el núcleo RISC-V emulado con el mbedtls compilado para el objetivo, y el script del PC acepta sus tramas; la clave pública que produce el chip emulado es la misma que produce el banco en el PC a partir de los mismos bloques del motor.

Esa misma compilación se grabó después en un módulo ESP32-C3 real por USB-serie y se manejó con un script que registra cada línea y cronometra cada orden. En el silicio, sembrar el modelo lleva decenas de milisegundos, generar la clave P-256 y guardarla unos 0,17 s, la petición de certificado 0,2 s y una lectura firmada unos 0,2 s con la ida y vuelta serie incluida; la clave privada sobrevive a reinicios físicos (el pin de habilitación pulsado desde el adaptador serie), el contador de arranques guardado en flash avanza en cada uno, la pública leída tras el reinicio es la misma y la primera lectura del arranque siguiente supera en secuencia a la última del anterior, así que la regla contra la repetición se cumple a través de cortes de alimentación reales. Dos cosas que el emulador no podía enseñar aparecieron a la primera. La consola leía las órdenes con una llamada de línea completa sobre una entrada no bloqueante, y por un enlace USB-serie real los bytes de una orden llegan con huecos: una sola orden llegó al firmware en cuatro trozos, cada uno rechazado como desconocido; la consola acumula ahora caracteres hasta el fin de línea. Y

la clave nueva generada tras olvidar la anterior resultó idéntica a ella, porque el modelo del motor saca su "aleatorio" de una función pseudoaleatoria sin semilla que repite la misma secuencia en cada arranque; no es un defecto del medidor, sino la demostración en vivo de para qué sirve el motor. El verificador del PC acepta todas las tramas del log de la placa. Lo que sigue sin verificar es el propio motor en placa: solo existe en simulación hasta la puesta en marcha de la FPGA, y la entropía del modelo es un AES con semilla fija, no una medida.

6.6. Puesta en marcha en la FPGA

Pendiente del hardware. Las restricciones que necesitan los anillos se conocen por la bibliografía y por la documentación del fabricante — `DONT_TOUCH` en las celdas y redes de cada anillo (a diferencia de `KEEP`, sobrevive a la implementación), `ALLOW_COMBINATORIAL_LOOPS` en una red de cada lazo, `set_disable_timing` para romper el lazo de cara al análisis estático de tiempos, y un bloque de emplazamiento por anillo para que anillos idénticos ni se fusionen ni se coloquen uno junto a otro — y el primer bitstream tiene un único propósito: leer los contadores de frecuencia y confirmar que todos los anillos oscilan. Antes de eso no se afirma nada sobre la entropía.

Capítulo 7

Camino hacia un circuito integrado

El motor está diseñado para acabar siendo un circuito integrado dedicado que sirve claves a un microcontrolador, y la FPGA es su vehículo de verificación, no su destino. Este capítulo toma el mismo VHDL que pasa los bancos de pruebas del Capítulo 6, lo sintetiza sobre una biblioteca de celdas estándar abierta y expone cuánto costaría una tirada de fabricación y qué tendría que cambiar en la fuente de entropía. Un envío a fabricación (tape-out) queda explícitamente fuera del alcance de este TFM — no por coste ni por área, como se verá, sino por plazos — y todo lo que aquí aparece se etiqueta como lo que es: una cota inferior del área obtenida de la síntesis lógica, con los factores de crecimiento hasta el área del núcleo tomados de datos publicados.

7.1. Flujo de síntesis con celdas estándar

El flujo es reproducible a propósito con herramientas libres. Yosys lee el VHDL a través del front end de GHDL, sintetiza cada bloque aplanado, mapea los flip-flops y la lógica sobre el fichero liberty de la biblioteca objetivo e informa del número de celdas y del área. La biblioteca es el PDK abierto de IHP, SG13G2, un proceso BiCMOS de 130 nm con una biblioteca de celdas estándar mantenida, macros de SRAM compiladas, celdas de E/S y soporte de OpenROAD. Se ha elegido porque es el único PDK abierto que está vivo, es europeo y es accesible a través de la misma afiliación a Europractice que la universidad ya tiene. La puerta equivalente se lee del propio fichero liberty y no de memoria: 1 GE = área de `sg13g2_nand2_1` = 7,2576 μm^2. No se aplica ninguna restricción de reloj: Yosys no la usa para el área, así que las cifras no dependen de una frecuencia objetivo. Los osciladores en anillo no se sintetizan; son el único bloque diseñado a nivel de transistor (Capítulo 4).

7.2. Área por bloque en puertas equivalentes

La Tabla 7.1 da el resultado de cada bloque. El motor digital — todo salvo el búfer de captura — ocupa unos **46 kGE**. Sintetizados por separado, el autenticador y el generador llevan cada uno su propio núcleo de cifrado de una ronda por ciclo, de 16 kGE, y así están medidas sus filas de la tabla; en el motor completo un solo núcleo de 32 bits sirve a los dos, porque el secuenciador nunca los deja funcionar a la vez (Sección 7.3), de modo que las filas no suman la cifra del motor. Aproximadamente: el AES compartido de 32 bits 6,7 kGE, el autenticador sin su núcleo 7, el generador sin él 22, los tests de salud 1,3, muestreador, cruce y resets 0,4, I2C 1,1, y unos 7 kGE de secuenciador, registros y reparto del núcleo en el nivel superior. El búfer de captura se incluye para que la tabla esté completa, pero no debe leerse como parte del producto: sus 46,8 kGE son una memoria de 4096 bits mapeada sobre flip-flops porque el flujo no tiene macros de SRAM. En la FPGA ocupa una block RAM sin coste de lógica; en silicio sería una SRAM compilada o, con más sentido, un búfer mucho más pequeño, porque es una ruta de prueba que se desactiva en producción.

Tabla 7.1: Síntesis lógica sobre IHP SG13G2, aplanada, sin rutado. 1 GE = 7,2576 μm^2. Cota inferior del área [SIMULADO].

Bloque	Celdas	Flip-flops	Área (μm^2)	kGE
Núcleo AES-128, una ronda por ciclo (20 S-boxes, entrada de borrado)	10 571	262	116 550	16,06
Núcleo AES-128, ruta de 32 bits (4 S-boxes, 51 ciclos/bloque)	3 813	266	48 434	6,67
AES-CMAC (con su AES de una ronda)	13 289	912	169 382	23,34
CTR_DRBG con df (con su AES de una ronda)	19 895	2 357	277 293	38,21
Tests de salud (RCT, APT)	588	86	9 137	1,26
Muestreador ERO y divisor de 24 bits	127	28	2 371	0,33
Cruce de dominio de reloj del bit	11	7	408	0,06
Sincronizador de reset (uno por dominio)	3	3	147	0,02
Esclavo I2C (con guarda de bus bloqueado)	435	89	7 729	1,07
Búfer de captura (4096 bits en flip-flops)	12 526	4 140	339 386	46,76
Motor completo, anillos como cables, un solo AES de una ronda	39 879	8 175	739 061	101,83
Motor completo, un solo AES de 32 bits	32 954	8 179	671 576	**92,53**

Los números son coherentes entre sí, y esa coherencia es la que permitió detectar un error en el flujo: antes de aplanar, el informe devolvía para el autenticador y para el generador solo el área de su submódulo AES. Un autenticador que pese exactamente lo mismo que un AES desnudo es imposible, así que el que fallaba era el informe, no el diseño. Tras aplanar, el autenticador añade al AES 650 flip-flops (subclaves, estado de encadenamiento, clave) y 7,6 kGE, y el generador añade 2095 flip-flops y 22 kGE, de los que cerca de la mitad son sus registros. La tabla es la síntesis del motor con la versión 3 del mapa de registros, que añadió la orden de borrado total, un contador de peticiones que satura en vez de dar la vuelta, las reglas de exclusión de la captura y los sincronizadores de reset; el generador creció 1,0 kGE por el contador saturante y la ruta de borrado, y el motor completo 0,8 kGE, hasta 115,7 kGE con dos núcleos; compartir el núcleo lo dejó en 101,8, y el núcleo de 32 bits en los 92,5 kGE de la tabla. El flujo de esta tanda leyó el lenguaje de descripción de hardware VHSIC (VHDL) a través de la propia salida Verilog de GHDL y de la versión de Yosys instalable con `pip`, en lugar del conector de GHDL; en el núcleo AES, que no cambió, los dos caminos coinciden dentro del 0,9 %.

7.3. Optimizaciones y sus ahorros esperados

La síntesis es ingenua a propósito — el RTL se escribió para ser verificable, no pequeño — y apunta a tres optimizaciones, por orden de rentabilidad del esfuerzo:

1. **Un único núcleo AES compartido — hecho.** El secuenciador del nivel superior ya serializa las órdenes: el generador y el autenticador nunca funcionan a la vez. El núcleo vive ahora en el nivel superior y se entrega al bloque que levanta su pulso de arranque; una propiedad del banco del sistema comprueba que los dos nunca lo hacen en el mismo ciclo. Ahorro medido: **14,1 kGE** (de 115,7 a 101,6), algo por debajo de los 15,8 estimados, porque el reparto cuesta cerca de medio kGE y la entrada de borrado que la revisión exigió al núcleo y al generador cuesta otro. Un genérico conserva la disposición de dos núcleos.

2. **Ruta de datos compacta para la S-box — hecho.** Veinte tablas S-box independientes se sintetizan en 16 kGE para el núcleo de una ronda por ciclo; los diseños optimizados en área de la literatura van desde 2,4 kGE para una ruta de datos de 8 bits a 226 ciclos por bloque [81] hasta unos 5 kGE para rutas de datos de 32 bits. Aquí la velocidad es irrelevante, así que se escribió un segundo núcleo con ruta de 32 bits: cuatro S-boxes compartidas entre SubBytes y el SubWord de la expansión de clave, un solo MixColumns, ShiftRows aplicado como permutación de cableado del registro de estado al empezar cada ronda, una columna por ciclo, 51 ciclos por bloque en vez de 11. Pasa los mismos vectores oficiales, la cadena de mil bloques y la prueba de zeroización, y el banco exige que su salida coincida con la del núcleo de

una ronda en cada bloque. Medido: **6,67 kGE frente a 16,06** en el núcleo (ahorro de 9,4, frente a los 10 estimados; la diferencia son los multiplexores de columna y el registro de fase) y **92,5 frente a 101,8 kGE** en el motor completo, en la misma tanda de síntesis. El núcleo de una ronda sigue disponible mediante un genérico.

3. **Menos registros en el generador.** De sus 2355 flip-flops, la entropía capturada y el nonce (384), el material de semilla (256), la salida (256) y los temporales de la función de derivación pueden reducirse consumiendo la semilla como un flujo y compartiendo temporales; unos 800 flip-flops, cerca de 4 kGE.

Las dos optimizaciones medidas llevaron el motor digital de 69 kGE (dos núcleos de una ronda) a 46; la tercera lo dejaría en **≈42 kGE**. Es más que los 13 a 25 kGE que anticipaba el estado del arte para motores de este tipo, y la tabla dice por qué: el generador sin su núcleo son 22 kGE, la mitad registros, y el secuenciador y los registros del nivel superior otros 7; llegar al rango publicado exigiría repensar el almacenamiento del generador y no su cifrador, lo que este trabajo deja planteado y no hecho. Con el factor de crecimiento publicado de ≈2,35 del área de celdas al área del núcleo para un AES en esta misma biblioteca, 42 kGE son unos 0,72 mm^2 de núcleo, que siguen cabiendo varias veces en el bloque académico más pequeño (Sección 7.5). Los dos primeros ahorros son medidas; el tercero es una estimación y se recoge como trabajo futuro, no se da por hecho.

7.4. Lo que necesita el oscilador en anillo en silicio

Los bloques digitales se llevan a silicio con el flujo estándar. La fuente de entropía no, y la literatura que ha dado ese salto — en particular un estudio de 2024 que caracteriza el mismo modelo de anillo en una Artix-7 y en un circuito FD-SOI de 28 nm [32] — plantea tres puntos que el diseño del chip debe asumir.

En primer lugar, el anillo se reconstruye con celdas inversoras en lugar de tablas de consulta, y su jitter cambia en valor y en naturaleza: el ruido de parpadeo gana peso en los nodos más recientes, y el estudio encuentra, en contra de lo esperado, que contarlo como fuente legítima puede aumentar el tasa de bits para la misma entropía. El modelo estocástico y el estimador del Capítulo 4 se trasladan sin cambios; la σ medida no. En segundo lugar, la variación de proceso hace que cada dado sea distinto, así que la frecuencia y el jitter de cada anillo deben calibrarse chip a chip; los diseños totalmente digitales publicados que superan barridos de tensión y temperatura lo hacen con un anillo configurable y un lazo de ajuste [104]. En tercer lugar, y lo más importante para un dispositivo de seguridad, inyectar una frecuencia en la alimentación engancha los anillos y elimina el jitter: una señal de 500 mV en el raíl redujo el espacio de claves de un microcontrolador seguro comercial de 2^{64} a unas 3300 [1], y unos pocos cientos de microvatios de campo

radiado destruyeron la estadística de un generador de cincuenta anillos [105]. El mismo lazo de calibración, junto con el filtrado de la alimentación, es la defensa publicada, y los tests de salud de la Sección 5.5 son el detector de último recurso. Nada de esto está en el prototipo en FPGA; es el trabajo que añade una implementación en silicio.

7.5. Coste y plazos de una lanzadera académica

La fabricación es más accesible de lo que sugiere el peso de la palabra. La Universidad de Sevilla y el IMSE-CNM son miembros de Europractice, cuya lista de precios académicos es pública: un bloque mini@sic en UMC 180 nm cuesta 3 430 €, uno en TSMC 65 nm 3 691 €, e IHP SG13G2 se ofrece a 5 110 €/mm^2 con una tirada multiproyecto gratuita para diseños de código abierto de menos de 2 mm^2 [107]. Un motor de 0,3–0,4 mm^2 cabe muchas veces en el bloque más pequeño de cualquiera de ellos; el chip estaría limitado por sus pads, no por su lógica. El propio grupo de seguridad hardware del instituto ya ha llevado primitivas de oscilador en anillo desde FPGAs Xilinx 7-series hasta silicio de 65 nm caracterizado, lo que hace que continuar este trabajo sea cuestión de sumarse a una línea existente más que de abrir una.

Lo que no cabe en un trabajo fin de máster son los plazos: desde el envío a fabricación (tape-out) hasta tener muestras encapsuladas pasan de meses a más de un año, el precio no incluye el encapsulado y la caracterización de los dados devueltos es un proyecto en sí misma. Como comparación, un elemento seguro comercial como el ATECC608B cuesta 0,90 $ por unidad suelta y se vende con una validación NIST de la fuente de entropía [7]; dos de sus competidores directos no mencionan SP 800-90 en ninguna parte de sus hojas de datos. Por tanto, la posición honesta de este trabajo no es competir en precio sino en *transparencia de la entropía*: cada bit de una clave producida por este motor puede trazarse hasta un jitter medido y una cota declarada, que es precisamente lo que una pieza de caja negra no puede ofrecer.

Capítulo 8

Resultados y comparación

Este capítulo reúne lo que el trabajo establece y lo separa, sin suavizarlo, de lo que no establece. El motor se ha modelado, escrito, verificado en simulación frente a vectores oficiales y sintetizado sobre una biblioteca de celdas estándar real; todavía no ha funcionado en la matriz de puertas programable en campo (FPGA). Por eso cada cifra que sigue lleva su marca de procedencia, y la campaña de entropía — el único resultado que necesita silicio en el banco — se presenta como un protocolo por ejecutar, no como datos.

8.1. Estado frente a los objetivos

La Tabla 8.1 relaciona los objetivos de la Sección 1.3 con las evidencias.

El resultado mínimo defendible declarado en el Capítulo 1 — O1–O3 junto con O5 — se alcanza en su parte de modelado y diseño. Lo que aún le falta a O5 es la puesta en marcha, no el diseño.

8.2. Ruido del anillo a nivel de transistor

El anillo se simuló con una fuente de ruido de corriente de densidad espectral controlada inyectada en el nodo de conmutación, barriendo la densidad y midiendo la desviación típica de los cruces de flanco de subida a lo largo de 500 periodos. La Tabla 8.2 da el resultado para un anillo de cinco etapas con una carga de 10 fF.

Tres lecturas, y la tercera es la que importa.

El suelo numérico queda lo bastante por debajo de la señal. Con la fuente de ruido apagada, la medida devuelve 1,38 fs, que es la resolución de la propia simulación, no física. Está unas 870 veces por debajo del menor jitter que hay que medir, así que el suelo no contamina el barrido. Llegar a él exigió cuidado: el primer intento devolvió 0,33 ps, del mismo orden que la magnitud estudiada, y la causa era que el transitorio de arranque del

Tabla 8.1: Objetivos y evidencia que respalda cada uno.

Obj.	Estado	Evidencia
O1	cumplido	Anillo a nivel de transistor en el PDK abierto IHP SG13G2, con el ruido de sus propios modelos de dispositivo (Sección 8.3); pendiente el PDK de fundición
O2	cumplido	Jitter extraído de las formas de onda; térmico y $1/f$ separados (banco y función de sensibilidad al impulso); estimador validado sobre datos sintéticos con Q conocido
O3	cumplido	Modelo estocástico cerrado: cálculo exacto con la gaussiana envuelta, punto de diseño $Q \geq 0{,}2286$ (Sección 8.4)
O4	cumplido	Motor escrito en VHDL-2008 portable; siete bancos de pruebas pasan con los vectores de FIPS-197, SP 800-38B, RFC 4493 y Cryptographic Algorithm Validation Program del NIST (CAVP)
O5	parcial	El firmware del verificador compila y el protocolo está cerrado; falta la ejecución en placa
O6	pendiente	Requiere la FPGA en el banco (Sección 8.9)
O7	cumplido	Síntesis en IHP SG13G2, área por bloque y vía de optimización (Capítulo 7)
O8	parcial	Comparación estructural cerrada (Sección 8.6); el lado ESP32 de la comparación numérica está medido según SP 800-90B (Sección 3.6); el lado del motor necesita O6

Tabla 8.2: Jitter por periodo del anillo de cinco etapas frente a la densidad de ruido inyectada [SIMULADO]. $\sigma_{\text{rel}} = \sigma/T$.

Densidad de ruido (A^2/Hz)	Cruces	T (ps)	σ (fs)	σ_{rel}
ninguno (suelo numérico)	500	230,30	1,38	$6{,}0 \times 10^{-6}$
1×10^{-20}	501	230,37	1200	$5{,}2 \times 10^{-3}$
4×10^{-20}	507	229,98	2416	$1{,}1 \times 10^{-2}$
$1{,}6 \times 10^{-19}$	509	227,77	25588	$1{,}1 \times 10^{-1}$

anillo entraba en la estadística. Descartar los veinte primeros ciclos lo eliminó. Un suelo de medida que coincide con la señal no se distingue de un resultado, y precisamente por eso el suelo se caracterizó antes del barrido y no después.

La ley de escala se cumple donde debe. El jitter térmico se acumula como la raíz cuadrada de la densidad de ruido, así que cuadruplicar la densidad debe duplicar σ. De 1×10^{-20} a 4×10^{-20} la medida da un factor de 2,01 frente a los 2,00 previstos. Esa coincidencia es la prueba de que lo que se mide es el ruido inyectado y no un artefacto del resolvedor.

Y se rompe, a la vista, en lo alto del barrido. Con $1{,}6 \times 10^{-19}$, dieciséis veces la densidad de referencia, σ crece en un factor de 21,3 donde la ley predice 4,0, y el periodo medio baja de 230,4 a 227,8 ps. Una fuente de ruido de media nula no puede desplazar un periodo medio. Por tanto, el punto queda fuera del régimen que describe el modelo — la perturbación es lo bastante grande como para cambiar el punto de funcionamiento de los inversores — y se presenta en lugar de descartarse, porque el límite de validez de un modelo es en sí mismo un resultado. Solo los dos puntos centrales se usan en lo que sigue.

8.3. El anillo con los transistores de un PDK abierto

El mismo anillo de cinco etapas se simuló con los transistores de baja tensión del PDK abierto IHP SG13G2 ($W_n = 0{,}5$ μm, $W_p = 1{,}2$ μm, $L = 0{,}13$ μm, 1,2 V, 27 °C, esquina típica, 1 fF por nodo como estimación del cableado previa al layout), con la densidad de ruido medida en los mismos dispositivos (Sección 4.5) inyectada en cada nodo. La Tabla 8.3 da el resultado sobre 400 ns, unos 1070 periodos por punto. El simulador saca una realización de ruido nueva en cada ejecución y no se le puede hacer repetir una (en ngspice 47 el generador gaussiano de la fuente de ruido transitorio se siembra con el identificador del proceso, y ninguna de sus opciones de semilla llega a él), así que la incertidumbre del jitter se obtuvo repitiendo: el punto térmico del PDK se simuló ocho veces.

Primero, la resolución: 0,68 fs, 262 veces por debajo del jitter que produce el ruido de los dispositivos, y con un exponente de 0,42 que la identifica como ruido numérico y no como un paseo aleatorio. Las ocho realizaciones térmicas se dispersan un 3,3 % en σ (de 169,0 a 186,5 fs), así que una ejecución suelta lleva esa incertidumbre y la media, un 1,2 %. Cuadruplicar la densidad multiplica el jitter por $1{,}99 \pm 0{,}06$ frente al 2 esperado, y el periodo medio no se mueve, así que el banco se mantiene en el régimen lineal que el banco provisional abandonaba en su punto más alto. El exponente de acumulación es 0,99 de media: ruido blanco, como supone el modelo estocástico. Como comprobación independiente, la expresión cerrada de Abidi para osciladores en anillo [125], evaluada

Tabla 8.3: Anillo de cinco etapas en IHP SG13G2 [SIMULADO]. La densidad de ruido no se supone: es el suelo blanco del propio modelo de ruido de los transistores en el punto de conmutación. El punto térmico es la media de ocho realizaciones independientes, ± una desviación típica de la media; las demás filas son realizaciones únicas. La potencia es la media sobre periodos completos.

Ruido inyectado	T (ps)	σ (fs)	σ/T	Exponente	P (μW)
ninguno (suelo numérico)	366,04	0,68	$1{,}8 \times 10^{-6}$	0,42	143,0
térmico del PDK, $4{,}13 \times 10^{-24}$ A^2/Hz	365,97	$176{,}7 \pm 2{,}0$	$4{,}83 \times 10^{-4}$	0,99	143,0
térmico $\times 4$	365,98	351,0	$9{,}59 \times 10^{-4}$	0,92	143,0
térmico + $1/f$ del PDK	365,97	183,3	$5{,}01 \times 10^{-4}$	1,01	143,0

con magnitudes medidas en el mismo punto de polarización, predice $\sigma/T = 1{,}2 \times 10^{-3}$; la simulación da 0,41 veces ese valor, el mismo orden. La forma de canal largo de esa fórmula, que pasa por la tensión umbral, no se aplica aquí: en el punto de conmutación los dispositivos están solo 0,07 V por encima del umbral.

Con $\sigma/T = 4{,}83 \times 10^{-4}$, el divisor para $Q \geq 0{,}2286$ es $K_D = 981\,095$ (de 958 895 a 1 004 347 dentro de una desviación típica), un intervalo de muestreo de 359 μs con un anillo de muestreo idéntico, y 2,79 kbit/s por ERO; si además se cuenta el jitter independiente del anillo de muestreo, K_D se reduce a la mitad y la tasa se duplica, pero eso exige que los dos anillos no compartan ruido a través de la alimentación o del sustrato, algo que solo el silicio puede mostrar. Una consecuencia de diseño se aplicó de inmediato: el contador del divisor tenía 20 bits, así que el mayor K_D era $2^{20} = 1\,048\,576$, un margen del 7 % (del 4 al 9 %) sobre lo requerido en la esquina típica a temperatura ambiente. Ahora tiene 24 bits (cuatro flip-flops, 0,04 kGE), y el banco del motor completo y su verificación cruzada se volvieron a ejecutar con él.

Esquinas y temperatura. El punto térmico se repitió en las esquinas lenta, típica y rápida a -40, 27 y 125 °C, recalculando el ruido de los dispositivos en cada punto y con cuatro realizaciones por punto [SIMULADO]. El peor caso es el frío, no el calor: σ/T crece con la temperatura en las tres esquinas, porque el ruido térmico va con kT y el periodo cambia poco, así que el divisor necesario baja en un factor de 2,5 a 3,5 de -40 a 125 °C. La esquina lenta es peor que la rápida, y las dos cosas se combinan peor que su producto: la esquina lenta a -40 °C ($\sigma/T = 3{,}15 \times 10^{-4}$, periodo 438,6 ps) pide $K_D = 2\,306\,215$. Ahí 2^{20} da $H_\infty = 0{,}78$ y 2^{21} solo 0,970, así que un divisor fijo válido en todo el rango a la tensión nominal de 1,2 V es 2^{22}, con $H_\infty = 0{,}9995$ en el peor punto y de 0,51 a 0,82 kbit/s según la esquina. La alimentación es el tercer factor, y el más fuerte. A -40 °C en la esquina lenta, bajar la tensión un 10 %, a 1,08 V, alarga el periodo un 34 % mientras σ apenas se mueve, así que el divisor necesario crece en un factor 1,65, hasta $K_D = 3\,808\,445$ (de 3,71 a $3{,}91 \times 10^6$ a $\pm 1\sigma$, ocho realizaciones); a 1,32 V baja en un factor 0,79. Las esquinas asimétricas a -40 °C (sf, fs) piden 1,32 y $1{,}55 \times 10^6$, cerca de tt (1,47) y por

debajo de ss; a 1,08 V, tt, sf y fs piden 1,93, 1,57 y $1{,}74 \times 10^6$, la mitad que ss. En el peor punto, 2^{22} todavía da $H_\infty = 0{,}9873$, pero con un 10 % de margen sobre K_D, cerca del 7 % que se juzgó escaso para 2^{20} en el punto típico; 2^{23} da 0,9999 y de 0,20 a 0,41 kbit/s, unos 1,9 s para los 384 bits de una siembra en el peor caso. El desapareamiento, sorteado por transistor con los modelos estadísticos del PDK y con una realización de ruido nueva en cada ejecución, dispersa el periodo un 1,3 % en 48 chips en el punto típico y un 1,6 % en 15 en el peor punto, pero deja σ/T donde estaba: su dispersión entre chips (2,6 y 3,2 %) no supera la que da el ruido solo entre ejecuciones. Una decimosexta ejecución en el peor punto se descartó porque su número de cruces no cabe en el tiempo simulado, probablemente por dobles cruces de un flanco lento; la detección de cruces tiene ahora histéresis y deja constancia de cualquier caso así junto con su forma de onda. El peor de los 15 chips pide $K_D = 4{,}49 \times 10^6$, por encima de 2^{22} ($H_\infty = 0{,}973$), y 2^{23} lo sigue cubriendo ($H_\infty = 0{,}9996$). El registro del divisor arranca en 2^{24}, así que un maestro que no lo escriba ya está del lado seguro; un valor menor tiene que salir de la esquina más fría y la tensión más baja a las que deba funcionar el producto.

Ruido de parpadeo. Añadir el ruido $1/f$ del PDK no cambia nada en 400 ns: σ sube un 3,8 %, dentro de la dispersión entre realizaciones, y la separación $E[D^2] = a_1 M + a_2 M^2$, validada antes sobre fases sintéticas (componente térmica recuperada al 0,1 % con y sin $1/f$), da un a_2 compatible con cero. Amplificar la fuente $1/f$ para hacerla visible no funciona, y el motivo es instructivo: la fuente es de banda ancha y, amplificada mil veces, su densidad cerca de la frecuencia de oscilación ya es siete veces el suelo térmico, así que el anillo la convierte como ruido blanco y el término lineal crece en el factor esperado mientras que el término cuadrático no escala. La respuesta llega, en cambio, de la función de sensibilidad al impulso de Hajimiri y Lee [126], medida inyectando el 1 % de la carga del nodo en 40 fases (lineal al 0,1 % entre 0,5 y 2 %). Es casi simétrica, $|\Gamma_{dc}/\Gamma_{rms}| \approx 0{,}03$–$0{,}05$, así que el ruido de baja frecuencia inyectado de forma uniforme a lo largo del periodo apenas se convierte. Pero los transistores no son simétricos: medidos por separado, el dispositivo de canal p tiene tres veces el ruido $1/f$ del de canal n, y cada uno inyecta ruido sobre todo mientras conduce, durante uno de los dos lóbulos opuestos. Los dos límites acotan la realidad: con ruido estacionario, como en el banco, la frecuencia de esquina $1/f^3$ es 36 kHz y el parpadeo iguala al jitter térmico tras 27 000 periodos; con cada dispositivo confinado a su lóbulo, la frecuencia de esquina es 5,3 MHz y el cruce, 185 periodos. En ambos casos, con el divisor de funcionamiento el parpadeo domina la varianza acumulada, por un factor de entre 36 y 5 300. De ahí se siguen dos consecuencias. El jitter que fija la cota de entropía debe medirse con acumulación corta, por debajo de unos cien periodos para este anillo, y separarse del término cuadrático — deducirlo del jitter acumulado total en K_D atribuiría una entropía que no existe. Y el banco estacionario puede sobrestimar la varianza térmica hasta en un factor dos, lo que duplicaría K_D; el

contador de 24 bits lo cubre, y la herramienta que lo resuelve es un análisis de ruido transitorio cicloestacionario, un uso concreto de un simulador comercial.

8.4. Punto de funcionamiento: divisor y tasa de bits

El modelo estocástico fija el objetivo. Para los umbrales PTG.2 de AIS 20/31 v3.0, el criterio de min-entropía ($H_\infty \geq 0{,}98$) exige $Q \geq 0{,}2286$ y es más exigente que el criterio de Shannon ($H \geq 0{,}9998$, $Q \geq 0{,}2022$); por eso el diseño apunta al primero, con $Q = 0{,}30$ como valor de consigna de trabajo, que da $H_\infty = 0{,}9951$. Dos propiedades del modelo se demostraron en lugar de suponerse: la serie truncada de la ecuación 14 de [10] *sobrestima* la entropía y, por tanto, no es una cota inferior, así que se usa en su lugar el cálculo exacto con la gaussiana envuelta; y la deducción de la min-entropía con el primer armónico coincide con la exacta en seis cifras en todo el rango de interés.

Como $Q_{\text{gen}} = K_D \cdot Q_{\text{bruto}}$, el divisor se deduce del jitter que mida la campaña. La Tabla 8.4 da los puntos de funcionamiento para el rango de jitter esperable en esta tecnología [8], suponiendo RO2 a 500 MHz.

Tabla 8.4: Divisor y tasa de bits resultante por anillo para $Q \geq 0{,}2286$ [ESTIMADO].

σ/T por periodo	Q_{bruto}	K_D	Tasa de bits
$7{,}1 \times 10^{-4}$	5×10^{-7}	457 200	1,09 kbit/s
$1{,}4 \times 10^{-3}$	2×10^{-6}	114 300	4,37 kbit/s
$2{,}8 \times 10^{-3}$	8×10^{-6}	28 575	17,5 kbit/s

Estos valores encierran el punto de diseño publicado por Fischer y Lubicz [13] ($K_D \approx 430\,000$ para $\sigma = 5{,}01$ ps sobre $T = 7{,}81$ ns), y esa es la comprobación de coherencia de que el modelo se aplica como pretendían sus autores.

La consecuencia para la arquitectura se dice sin rodeos: un solo ERO entrega kbit/s, no Mbit/s. Ese es el precio de una entropía demostrable, y aquí se puede pagar porque el producto del motor es una semilla de 256 bits para el DRBG, no un flujo de bits. Más tasa de bits significa más anillos independientes, cada uno con su propia cota, no un anillo más rápido.

8.5. Verificación funcional consolidada

El Capítulo 6 describe la estrategia; esta sección recoge los totales. Cada cifra de la Tabla 8.5 se obtuvo ejecutando la batería de pruebas, no leyendo un registro anterior.

Dos de ellas merecen una frase. Los 960 casos CAVP son la prueba individual más sólida del trabajo: el generador no solo es coherente consigo mismo, sino que reproduce las salidas que el NIST publica para CTR_DRBG con AES-128, tanto en la configuración

Tabla 8.5: Totales de la verificación [SIMULADO].

Batería	Qué comprueba	Resultado
Bancos de pruebas VHDL (GHDL)	anillos, ERO, captura, tests de salud, I2C, Advanced Encryption Standard, estándar de cifrado avanzado (AES), código de autenticación de mensajes basado en cifrado (CMAC), CTR_DRBG, nivel superior	7/7 pasan
`ctr_drbg_ref.py`	vectores CAVP, con y sin función de derivación	960/960
`ero_model.py`	modelo estocástico, exactitud y cotas	24/24
`jitter_estimator.py`	recuperación del jitter sobre datos sintéticos de Q conocido	20/20
`cortes_salud.py`	umbrales de los tests de salud frente a las fórmulas de SP 800-90B	6/6
`verifica_top.py`	verificación cruzada del banco de nivel superior con un AES independiente	5/5

con función de derivación como en la de entropía en bruto. Y los umbrales de los tests de salud, $C_{RCT} = 22$ y $C_{APT} = 596$, se habían calculado a mano a partir de la norma y nunca se habían comprobado; volver a obtenerlos con las fórmulas de SP 800-90B [6] confirma ambos. La misma comprobación cuantifica el coste de la prueba del complemento añadida al Adaptive Proportion Test, prueba de proporción adaptativa (APT), que es una aportación de este trabajo y no de la norma: eleva la tasa de falsas alarmas por ventana de $2^{-20{,}15}$ a $2^{-20{,}14}$, un aumento del 0,8 %, porque la cola complementaria está a 5,7 desviaciones típicas. Por tanto, el umbral sigue siendo válido tal como está y la adición sale gratis en la práctica.

8.6. Comparación con elementos seguros comerciales

El motor ocupa el mismo hueco funcional que un elemento seguro comercial: un pequeño dispositivo acompañante que genera y guarda claves para un microcontrolador anfitrión a través de un bus de dos hilos. La Tabla 8.6 lo sitúa junto a los tres componentes que dominan el mercado de elementos seguros I2C.

La lectura honesta de esa tabla tiene dos mitades.

En prestaciones y nivel de garantía ganan los componentes comerciales, y con mucha diferencia. Dos de ellos tienen evaluaciones CC EAL6+, que suponen un grado de escrutinio por terceros — incluida la resistencia a ataques físicos — que ningún proyecto académico reproduce. Ofrecen criptografía asimétrica, flujos de aprovisionamiento certificados y soporte para la fabricación. Este trabajo ofrece generación de claves simétricas y autenticación, y no tiene ninguna certificación.

En lo que de verdad trata este TFM, la comparación tiene más matices que “abierto

Tabla 8.6: El motor frente a elementos seguros comerciales. Datos de certificación tomados de las hojas de datos de los fabricantes y de los registros de validación del NIST [7, 102, 98, 99].

	ATECC608B	SE050	OPTIGA Trust M	Este trabajo
Bus	I2C	I2C	I2C	I2C
Certificación	JIL High; NIST ESV #E46 para la fuente de entropía	CC EAL6+ (hasta el SO)	CC EAL6+ (hardware)	ninguna
Fuente de entropía	osciladores en anillo, según el registro ESV	"TRNG, SP 800-90B" en la hoja de datos	no indicada	ERO, publicada
Entropía declarada	0,5071 bit/muestra, validada por el NIST	no publicada	no publicada	cota obtenida del jitter medido
Modelo estocástico	no publicado	no publicado	no publicado	publicado
Exportación de claves	no es posible	no es posible	no es posible	solo en modo de prueba
Inspeccionable	no	no	no	sí, RTL publicado

frente a cerrado", y conviene ser exacto. De los tres, solo el ATECC608B tiene su fuente de entropía validada según SP 800-90B: el registro del NIST cita osciladores en anillo y declara 0,5071 bit de entropía por muestra [102]. Es una cifra real, de un tercero, y hay que reconocerla. Lo que ningún fabricante publica — tampoco Microchip — es el modelo estocástico que hay detrás de la cifra, la medida de jitter que la respalda ni el propio circuito; la hoja de datos de un competidor afirma cumplir la norma sin un registro de validación y la del otro no menciona SP 800-90 en absoluto. Se confía en su aleatoriedad porque se confía en el fabricante y en el esquema de evaluación. El motor descrito aquí hace el intercambio contrario: no tiene certificado, y puede mostrar de dónde viene cada bit de entropía, qué medida física lo acota y en qué se convierte la cota cuando cambian la temperatura o el divisor. Es la misma distinción con la que tropieza, desde el otro lado, el estudio de caja negra del generador del ESP32 de este trabajo (Sección 3.6) — una caja negra se puede probar estadísticamente, pero no se puede acotar — y es la razón por la que este motor se construyó como una caja blanca.

Cuál de los dos es preferible depende de la pregunta que se haga. Para sacar un producto al mercado, el componente certificado. Para saber cuánto vale la aleatoriedad de ese producto, solo responde el abierto.

8.7. La referencia del ESP32

El Capítulo 3 caracteriza el generador del ESP32 en cuatro condiciones de funcionamiento y en tres chips; la comparación de O8 toma de allí sus cifras.

De ello se siguen tres consecuencias para O8. Primera, la cifra que este TFM retiene como referencia del ESP32 es la no IID, de 0,81 a 0,92 bit por bit con una media de 0,85 sobre las treinta y seis capturas de las condiciones A–D, porque es lo que un evaluador acreditaría sin conceder la hipótesis IID, con la prueba de reinicios como recordatorio de que para la condición D es una cifra estadística y no entropía física; los bits en bruto del motor, cuando la campaña de la Sección 8.9 los produzca, deben pasarse por la misma batería con el mismo N, y compararse estimador por estimador, ya que el mismo sesgo conservador actuará sobre ellos; la calibración de las pruebas IID a esa longitud se les aplica sin cambios. Segunda, la cifra de Shannon de la primera campaña, 0,99999 bit por bit, y las cifras de min-entropía de más arriba no se contradicen — $H_\infty \leq H$ siempre — pero son monedas distintas: acreditar 256 bits de entropía cuesta 256 bits de salida en la primera y hasta 316 en la segunda, y solo la segunda es la que puede usar la generación de claves. Tercera, e independiente de cualquier número, la medida es de una caja negra: la fuente y el posprocesado que hay tras `esp_fill_random` no están documentados, y los estimadores de SP 800-90B están diseñados para ruido en bruto, no para una salida que puede estar ya acondicionada; la prueba de reinicios muestra hasta dónde puede llegar eso, un flujo de salida pseudoaleatoria que todos los estimadores puntúan como ruido. Lo que el ESP32 no puede aportar, devuelvan lo que devuelvan los estimadores, es la cota que la Sección 8.4 deriva para el motor a partir de una medida física. Esa asimetría es la comparación.

8.8. Balance de incertidumbre

Cinco magnitudes sostienen las conclusiones, y cada una tiene un tipo de incertidumbre distinto y declarado.

Jitter simulado. Suelo de resolución 1,38 fs [SIMULADO], unas 870 veces por debajo de la menor magnitud medida. La comprobación de escalado frente a la ley de raíz cuadrada esperada concuerda dentro del 0,5 % en el régimen válido, y el límite del régimen está documentado en la Sección 8.2. En el PDK abierto el suelo es de 0,68 fs, 262 veces por debajo del jitter, cuya dispersión entre ejecuciones es del 3,3 % (1,2 % para la media de ocho), y la densidad de ruido ya no se supone, sino que se toma de los modelos de dispositivo. Lo que sigue siendo aproximado es cómo se aplica ese ruido: la inyección estacionaria puede sobrestimar la varianza térmica hasta en un factor dos frente a un modelo cicloestacionario, y el cableado previo al layout es una estimación de 1 fF. Se cubrieron las esquinas de proceso y la temperatura (ss, tt y

ff de −40 a 125 °C), y a −40 °C también las esquinas asimétricas, la alimentación a ±10 % y las esquinas tt, sf y fs con la tensión baja, y el desapareamiento en el punto típico y en el peor; no ff con la tensión baja ni la tensión baja en caliente. La cifra es, por tanto, un valor simulado con una banda de un factor dos, no el jitter de un dispositivo fabricado.

Q **recuperado.** El estimador reproduce un Q conocido con un sesgo inferior al 2 % y una dispersión del 4,0 % sobre ocho semillas con 20 000 bloques [SIMULADO]. Sin las correcciones aplicadas aquí, un simple ajuste por recta sesga el resultado en un −18 %, así que la corrección no es cosmética.

Área. Una cota inferior [SIMULADO]: solo celdas, con el diseño aplanado, sin rutado, relleno ni árbol de reloj. Los factores de crecimiento publicados de área de celdas a área de núcleo en esta biblioteca son $\approx$ 2,35 para un AES, y los ahorros de optimización del Capítulo 7 son [ESTIMADO] y explícitamente no se presentan como medidos.

Min-entropía de la referencia del ESP32. Medida [MEDIDO] en treinta y ocho capturas de 10^6 a $8{,}4 \times 10^6$ bits, nueve por condición, tres en cada uno de tres chips del mismo modelo identificados por dirección MAC. La cifra de la vía IID está en el techo de resolución del estimador MCV para ese N (0,9963 con 10^6 bits), así que es un suelo situado en el techo más que una medida de la fuente, y los veredictos IID se leen frente a la tasa de falsos fallos medida de cada prueba a esa longitud, no frente a la nominal. La cifra no IID la fijan estimadores cuyo sesgo conservador con datos casi uniformes supera la dispersión entre condiciones; un análisis de la varianza no encuentra efecto de la condición ($p \geq 0{,}045$ en todos los estimadores) ni efecto del chip ($p \geq 0{,}032$, frente a 0,0045 tras la corrección). Tres dispositivos muestran que la referencia no depende de uno de ellos; no acotan la dispersión en una población de producción. La prueba de reinicios se ejecutó en dos chips, en la condición D, con el Wi-Fi arrancado pero no asociado y, en el segundo chip, con cada fuente encendida y apagada por separado; no se probaron el Bluetooth ni una radio asociada. Acota la entropía que inyecta el bootloader de serie solo por abajo: que no haya ninguna repetición en 1000 reinicios significa más de unos $1{,}7 \times 10^5$ estados de partida igual de probables, unos 17 bits, con un 95 % de confianza. El tamaño del estado interno del generador no está documentado.

Min-entropía del dispositivo construido. Aún no acotada. El modelo, el estimador y los umbrales de corte están listos; el número requiere la campaña de la Sección 8.9. Aquí no se ofrece ningún valor, y no debe inferirse ninguno del jitter simulado.

8.9. La campaña pendiente

El único resultado que debe aportar el silicio es la min-entropía de los anillos tal como se construyan, y el procedimiento se fija de antemano para que no pueda ajustarse después de ver los datos.

1. **Confirmar que los anillos oscilan.** Leer la frecuencia de cada anillo con el contador integrado antes de fiarse de cualquier medida de entropía. Un anillo eliminado por el sintetizador, o dos anillos que se han enganchado entre sí, serían de otro modo indistinguibles de una fuente que funciona.

2. **Medir el jitter** con $K_D = 1$ por el método de Fischer y Lubicz [13], barriendo la distancia entre pares M y ajustando la región lineal, con al menos tres anillos para que la componente global se separe de la local [31]. Ajustar $\sigma^2(t) = at + bt^2$ y quedarse solo con el término térmico.

3. **Fijar** K_D a partir del jitter medido para $Q \geq 0{,}2286$ con margen, y recalcular los umbrales de corte de los tests de salud para la H_∞ medida en lugar del 0,98 supuesto, con `cortes_salud.py`.

4. **Recoger bits en bruto** en ese punto de funcionamiento y ejecutar los estimadores de NIST SP 800-90B, contrastando la estimación empírica con la cota del modelo. El acuerdo respalda el modelo; el desacuerdo es un resultado sobre las hipótesis del modelo, y en ambos casos se informa de ello.

5. **Repetir con distintas ubicaciones** — anillos en regiones separadas frente a anillos adyacentes — para cuantificar el acoplamiento sobre el que advierte la bibliografía, que es la medida que la arquitectura de este motor se eligió para hacer posible.

Hasta que se cierre el paso 4, la afirmación del motor es precisa y limitada: la cadena desde el ruido físico hasta el material de clave está diseñada, implementada y verificada frente a las normas, y la cota que producirá es calculable a partir de una medida que aún no se ha tomado.

Capítulo 9

Conclusiones y trabajo futuro

9.1. Conclusiones

9.2. Aportaciones

9.3. Trabajo futuro

Bibliografía

[1] A. T. Markettos and S. W. Moore, "The frequency injection attack on ring-oscillator-based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2009*, ser. LNCS, vol. 5747. Springer, 2009, pp. 317–331. [Online]. Available: https://www.iacr.org/archive/ches2009/57470316/57470316.pdf

[2] D. Hurley-Smith and J. Hernandez-Castro, "Certifiably biased: An in-depth analysis of a common criteria EAL4+ certified TRNG," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 4, pp. 1031–1041, 2018.

[3] J. Blanco-Romero, F. Almenares Mendoza, D. Díaz-Sánchez, and A. Marín-López, "Entropy bootstrapping for wireless embedded systems," arXiv:2607.08865, Sep. 2026. [Online]. Available: https://arxiv.org/abs/2607.08865

[4] National Institute of Standards and Technology, "Decision to revise NIST SP 800-22 Rev. 1a," Apr. 2022. [Online]. Available: https://www.nist.gov/news-events/news/2022/04/decision-revise-nist-sp-800-22-rev-1a

[5] M. Peter and W. Schindler, "A proposal for functionality classes for random number generators, version 3.0," Bundesamt für Sicherheit in der Informationstechnik (BSI), Mathematical-technical reference of AIS 20/31, Sep. 2024. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_31_Functionality_classes_for_random_number_generators_e_2024.pdf

[6] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, "Recommendation for the entropy sources used for random bit generation," NIST, Special Publication SP 800-90B, Jan. 2018. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/b/final

[7] *ATECC608B CryptoAuthentication Device Summary Data Sheet*, Microchip Technology Inc., 2021. [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/SCBU/ProductDocuments/DataSheets/ATECC608B-CryptoAuthentication-Device-Summary-Data-Sheet-DS40002239B.pdf

[8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *26th International Conference on Field Programmable Logic and Applications (FPL 2016)*, 2016, pp. 1–10. [Online]. Available: https://hal.science/hal-01382955/document

[9] D. Lubicz and V. Fischer, "Recommendations for the design and validation of a physical true random number generator integrated in an electronic device, version 1.0," IACR Cryptology ePrint Archive, Report 2024/301, Feb. 2024. [Online]. Available: https://eprint.iacr.org/2024/301.pdf

[10] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011. [Online]. Available: https://eprint.iacr.org/2009/299.pdf

[11] W. Killmann and W. Schindler, "A design for a physical RNG with robust entropy estimators," in *Cryptographic Hardware and Embedded Systems – CHES 2008*, ser. LNCS, vol. 5154. Springer, 2008, pp. 146–163.

[12] Y. Ma, J. Lin, T. Chen, C. Xu, Z. Liu, and J. Jing, "Entropy evaluation for oscillator-based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 544–561.

[13] V. Fischer and D. Lubicz, "Embedded evaluation of randomness in oscillator based elementary TRNG," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 527–543. [Online]. Available: https://www.iacr.org/archive/ches2014/87310109/87310109.pdf

[14] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Transactions on Computers*, vol. 56, no. 1, pp. 109–119, 2007.

[15] K. Wold and C. H. Tan, "Analysis and enhancement of random number generator in FPGA based on oscillator rings," *International Journal of Reconfigurable Computing*, vol. 2009, p. Article ID 501672, 2009.

[16] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, "True-randomness and pseudo-randomness in ring oscillator-based true random number generators," *International Journal of Reconfigurable Computing*, vol. 2010, pp. Article ID 879 281, 1–13, 2010.

[17] P. Kohlbrenner and K. Gaj, "An embedded true random number generator for FPGAs," in *Proc. ACM/SIGDA 12th International Symposium on Field Programmable Gate Arrays (FPGA 2004)*, 2004, pp. 71–78. [Online]. Available: https://people-ece.vse.gmu.edu/~kgaj/publications/conferences/GMU_FPGA_2004.pdf

[18] B. Valtchanov, V. Fischer, and A. Aubert, "Enhanced TRNG based on the coherent sampling," in *3rd International Conference on Signals, Circuits and Systems (SCS 2009)*, 2009, pp. 1–6.

[19] A. Peetermans, V. Rožić, and I. Verbauwhede, "Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, no. 2, pp. 7:1–7:20, 2021. [Online]. Available: https://cosicdatabase.esat.kuleuven.be/backend/publications/files/journal/3280

[20] ——, "A highly-portable true random number generator based on coherent sampling," in *29th International Conference on Field Programmable Logic and Applications (FPL 2019)*, 2019, pp. 218–224. [Online]. Available: https://lirias.kuleuven.be/retrieve/0ee069b3-afb3-43e3-9fe3-9e6d0b90ae46

[21] M. Varchola and M. Drutarovský, "New high entropy element for FPGA based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2010*, ser. LNCS, vol. 6225. Springer, 2010, pp. 351–365. [Online]. Available: https://www.iacr.org/archive/ches2010/62250341/62250341.pdf

[22] F. Bernard, P. Haddad, V. Fischer, and J. Nicolai, "From physical to stochastic modeling of a TERO-based TRNG," *Journal of Cryptology*, vol. 32, no. 2, pp. 435–458, 2019.

[23] A. Cherkaoui, V. Fischer, L. Fesquet, and A. Aubert, "A very high speed true random number generator with entropy assessment," in *Cryptographic Hardware and Embedded Systems – CHES 2013*, ser. LNCS, vol. 8086. Springer, 2013, pp. 179–196. [Online]. Available: https://www.iacr.org/archive/ches2013/80860154/80860154.pdf

[24] B. Yang, V. Rožić, M. Grujić, N. Mentens, and I. Verbauwhede, "ES-TRNG: A high-throughput, low-area true random number generator based on edge sampling," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 3, pp. 267–292, 2018. [Online]. Available: https://tches.iacr.org/index.php/TCHES/article/view/7276

[25] V. Rožić, B. Yang, W. Dehaene, and I. Verbauwhede, "Highly efficient entropy extraction for true random number generators on FPGAs," in *Proc. 52nd Annual Design Automation Conference (DAC 2015)*, 2015, pp. 116:1–116:6.

[26] CEA-Leti, "Opentrng: open-source reference ptrngs based on ring oscillators (ero, muro, coso) — hardware documentation and repository,"

https://opentrng.org/docs/hardware.html ; https://github.com/opentrng/ptrng, 2025.

[27] W. Schindler and W. Killmann, "Evaluation criteria for true (physical) random number generators used in cryptographic applications," in *Cryptographic Hardware and Embedded Systems – CHES 2002*, ser. LNCS, vol. 2523. Springer, 2003, pp. 431–449.

[28] P. Haddad, Y. Teglia, F. Bernard, and V. Fischer, "On the assumption of mutual independence of jitter realizations in P-TRNG stochastic models," in *Design, Automation & Test in Europe (DATE 2014)*, 2014, pp. 1–6.

[29] M.-J. O. Saarinen, "On entropy and bit patterns of ring oscillator jitter," arXiv:2102.02196 [cs.CR], v4, Oct. 2021. [Online]. Available: https://arxiv.org/abs/2102.02196

[30] V. Fischer, F. Bernard, N. Bochard, and M. Varchola, "Enhancing security of ring oscillator-based TRNG implemented in FPGA," in *International Conference on Field Programmable Logic and Applications (FPL 2008)*, 2008, pp. 245–250.

[31] D. Lubicz and M. Skórski, "Improved phase noise measurement for multi-ring oscillator based TRNG (arxiv title: Quantifying jitter transfer for differential measurement: Enhancing security of oscillator-based TRNGs)," arXiv:2410.08259 [cs.CR], Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.08259

[32] L. Benea, M. Carmona, V. Fischer, F. Pebay-Peyroula, and R. Wacquez, "Impact of the flicker noise on the ring oscillator-based TRNGs," in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024.

[33] D. Lubicz and N. Bochard, "Towards an oscillator based TRNG with a certified entropy rate," *IEEE Transactions on Computers*, vol. 64, no. 4, pp. 1191–1200, 2015.

[34] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators, version 2.0," Bundesamt für Sicherheit in der Informationstechnik (BSI), Tech. Rep., Sep. 2011. [Online]. Available: https://cosec.bit.uni-bonn.de/fileadmin/user_upload/teaching/15ss/15ss-taoc/01_AIS31_Functionality_classes_for_random_number_generators.pdf

[35] Bundesamt für Sicherheit in der Informationstechnik, "Transition policy to version 3.0 of the mathematical/technical reference, part of AIS 20/31, version 1.0," BSI, Tech. Rep., Apr. 2025. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_20_31_Transition_Policy_Version_3_0.pdf

[36] W. Schindler, "The new AIS 20/31," Presentación, European Cyber Week 2024, Rennes, Nov. 2024. [Online]. Available: https://www.creachlabs.fr/sites/default/files/public/media/document/2024-12/ecw2024-rng-puf-schindlerwerner-20241120.pdf

[37] National Institute of Standards and Technology, "Sp800-90b_entropyassessment: C++ implementation of the min-entropy assessment methods of sp 800-90b," GitHub repository, 2018. [Online]. Available: https://github.com/usnistgov/SP800-90B_EntropyAssessment

[38] L. E. Bassham, A. L. Rukhin, J. Soto, J. R. Nechvatal, M. E. Smid, E. B. Barker, S. D. Leigh, M. Levenson, M. Vangel, D. L. Banks, N. A. Heckert, J. F. Dray, and S. Vo, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," National Institute of Standards and Technology, NIST Special Publication 800-22 Rev. 1a, Apr. 2010.

[39] M.-J. O. Saarinen, "SP 800-22 and GM/T 0005-2012 tests: Clearly obsolete, possibly harmful," IACR Cryptology ePrint Archive, Report 2022/169, 2022. [Online]. Available: https://eprint.iacr.org/2022/169

[40] A. Iwasaki, "Analysis of NIST SP800-22 focusing on randomness of each sequence," arXiv:1710.01441, 2017. [Online]. Available: https://arxiv.org/abs/1710.01441

[41] "Public comments on SP 800-22 rev. 1a (crypto publication review project)," National Institute of Standards and Technology, 2021. [Online]. Available: https://csrc.nist.gov/csrc/media/Projects/crypto-publication-review-project/documents/initial-comments/sp800-22r1a-initial-public-comments-2021.pdf

[42] D. Hurley-Smith and J. Hernandez-Castro, "Great expectations: A critique of current approaches to random number generation testing & certification," in *Security Standardisation Research (SSR 2018)*, ser. LNCS, vol. 11322. Springer, 2018, pp. 143–163.

[43] "ESP-IDF programming guide: Random number generation (ESP32)," Espressif Systems, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/random.html

[44] *ESP32 Technical Reference Manual, Version 5.8, Chapter 18: Random Number Generator (RNG)*, Espressif Systems, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[45] J. Castillo, P. Aran Vila, F. Palacio, B. Garrido, S. Hernández, and A. Cirera, "Automated framework for testing random number generators for IoT security applications using NIST SP 800-22," *IoT*, vol. 7, no. 1, p. 26, 2026.

[46] P. Hidayatulloh, N. S. Qolbiyah, A. Nugroho, H. Mukhtar, I. Purnama, and F. Simanjuntak, "Performance evaluation of ESP32 random number generator for LoRa communication security," in *2024 International Conference on Radar, Antenna, Microwave, Electronics, and Telecommunications (ICRAMET)*, 2024, pp. 44–49.

[47] N. Heninger, Z. Durumeric, E. Wustrow, and J. A. Halderman, "Mining your Ps and Qs: Detection of widespread weak keys in network devices," in *Proceedings of the 21st USENIX Security Symposium*. USENIX Association, 2012, pp. 205–220. [Online]. Available: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/heninger

[48] D. E. Eastlake 3rd, J. I. Schiller, and S. Crocker, "Randomness requirements for security," RFC 4086, BCP 106, IETF, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4086

[49] J. Tillmanns, J. Classen, F. Rohrbach, and M. Hollick, "Firmware insider: Bluetooth randomness is mostly random," in *14th USENIX Workshop on Offensive Technologies (WOOT '20)*, 2020. [Online]. Available: https://arxiv.org/abs/2006.16921

[50] "ESP-IDF source: components/bootloader_support/src/bootloader_random.c," Espressif Systems, 2026. [Online]. Available: https://github.com/espressif/esp-idf/blob/master/components/bootloader_support/src/bootloader_random.c

[51] "ESP-IDF source: components/mbedtls/port/esp_hardware.c (mbedtls_hardware_poll)," Espressif Systems, 2026. [Online]. Available: https://github.com/espressif/esp-idf/blob/master/components/mbedtls/port/esp_hardware.c

[52] Espressif Systems, "Mbed TLS — ESP-IDF programming guide v5.4 (ESP32)," https://docs.espressif.com/projects/esp-idf/en/v5.4/esp32/api-reference/protocols/mbedtls.html, 2025.

[53] R. Salles and R. Farias, "TLS protocol analysis using IoTST — an IoT benchmark based on scheduler traces," *Sensors*, vol. 23, no. 5, p. 2538, 2023.

[54] KaeLL and Espressif Systems, "Clarification on the HW RNG (IDFGH-13813), espressif/esp-idf issue #14665," 2024. [Online]. Available: https://github.com/espressif/esp-idf/issues/14665

[55] Espressif esp-idf community, "boot: Disabling RNG early entropy source? (IDFGH-5906), espressif/esp-idf issue #7598," 2021. [Online]. Available: https://github.com/espressif/esp-idf/issues/7598

[56] "ESP-IDF programming guide: Random number generation (ESP32-C3)," Espressif Systems, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-reference/system/random.html

[57] M. Cheich, "Just how random is the ESP32 random number generator?" 2025. [Online]. Available: https://www.programmingelectronics.com/esp32-random-number-generator/

[58] AndroidCrypto, "ESP32: How to generate true random numbers ("true RNG")," 2024. [Online]. Available: https://medium.com/@androidcrypto/esp32-how-to-generate-true-random-numbers-true-rng-951fcda58e9b

[59] STMicroelectronics, "AN4230: Introduction to random number generation validation using the NIST statistical test suite for STM32 MCUs and MPUs." [Online]. Available: https://www.st.com/resource/en/application_note/an4230-introduction-to-random-number-generation-validation-using-the-nist-statistical-test-suite-for-stm32-mcus-and-mpus-stmicroelectronics.pdf

[60] K. Ngo and E. Dubrova, "Side-channel analysis of the random number generator in STM32 MCUs," in *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22)*. ACM, 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3526241.3530324

[61] "nRF52840 product specification: RNG — random number generator," Nordic Semiconductor. [Online]. Available: https://infocenter.nordicsemi.com/topic/ps_nrf52840/rng.html

[62] "RP2040 datasheet," Raspberry Pi Ltd. [Online]. Available: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[63] DMO, "RP2040 randomness and ring oscillator," 2022. [Online]. Available: https://dmo.page/2022/10/02/rp2040-randomness-and-ring-oscillator/

[64] S. C. Russell, ""well, that was unexpected...": The Raspberry Pi's hardware random number generator," 2013. [Online]. Available: https://scruss.com/blog/2013/06/07/well-that-was-unexpected-the-raspberry-pis-hardware-random-number-generator/

[65] D. E. Holcomb, W. P. Burleson, and K. Fu, "Power-up SRAM state as an identifying fingerprint and source of true random numbers," *IEEE Transactions*

on Computers, vol. 58, no. 9, pp. 1198–1210, 2009. [Online]. Available: https://ieeexplore.ieee.org/document/4674345/

[66] J. Guajardo, S. S. Kumar, G.-J. Schrijen, and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Cryptographic Hardware and Embedded Systems — CHES 2007*, ser. Lecture Notes in Computer Science, vol. 4727. Springer, 2007, pp. 63–80.

[67] V. van der Leest, E. van der Sluis, G.-J. Schrijen, P. Tuyls, and H. Handschuh, "Efficient implementation of true random number generator based on SRAM PUFs," in *Cryptography and Security: From Theory to Applications*, ser. Lecture Notes in Computer Science. Springer, 2012, vol. 6805.

[68] staniond, "esp32_puflib: SRAM-based physical unclonable function on the ESP32 microcontroller family." [Online]. Available: https://github.com/staniond/esp32_puflib

[69] ESP32 Forum users, "Wake-up-stub — uninitialized SRAM (SRAM PUF)." [Online]. Available: https://esp32.com/viewtopic.php?t=24384

[70] R. Impagliazzo, L. A. Levin, and M. Luby, "Pseudo-random generation from one-way functions," in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC '89)*. ACM, 1989, pp. 12–24.

[71] Y. Dodis, R. Gennaro, J. Håstad, H. Krawczyk, and T. Rabin, "Randomness extraction and key derivation using the CBC, cascade and HMAC modes," in *Advances in Cryptology — CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 494–510.

[72] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Advances in Cryptology — CRYPTO 2010*, ser. Lecture Notes in Computer Science, vol. 6223. Springer, 2010. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-14623-7_34

[73] M. P. Pawlowski, A. Jara, and M. Ogorzalek, "Harvesting entropy for random number generation for Internet of Things constrained devices using on-board sensors," *Sensors*, vol. 15, no. 10, pp. 26 838–26 865, 2015.

[74] J. Zhang and M. Wu, "Random number generation based on heterogeneous entropy sources fusion in multi-sensor networks," *Sensors*, vol. 23, no. 20, p. 8497, 2023.

[75] C. Foreman, R. Yeung, and F. J. Curchod, "Statistical testing of random number generators and their improvement using randomness extraction," *Entropy*, vol. 26, no. 12, p. 1053, 2024. [Online]. Available: https://arxiv.org/abs/2403.18716

[76] P. Kietzmann, T. C. Schmidt, and M. Wählisch, "A guideline on pseudorandom number generation (PRNG) in the IoT," *ACM Computing Surveys*, vol. 54, no. 6, pp. 112:1–112:38, 2021.

[77] Raspberry Pi Forums, "Cryptographically secure pseudorandom number generator (RP2040, pico_rand)." [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=362076

[78] National Institute of Standards and Technology, "Advanced encryption standard (AES)," NIST, Federal Information Processing Standards Publication FIPS 197-upd1, May 2023. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final

[79] T. Good and M. Benaissa, "AES on FPGA from the fastest to the smallest," in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 427–440.

[80] P. Chodowiec and K. Gaj, "Very compact FPGA implementation of the AES algorithm," in *Cryptographic Hardware and Embedded Systems — CHES 2003*, ser. Lecture Notes in Computer Science, vol. 2779. Springer, 2003, pp. 319–333.

[81] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, "Pushing the limits: A very compact and a threshold implementation of AES," in *Advances in Cryptology — EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, vol. 6632. Springer, 2011. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-20465-4_6

[82] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, "A compact Rijndael hardware architecture with S-Box optimization," in *Advances in Cryptology — ASIACRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2248. Springer, 2001, pp. 239–254.

[83] D. Canright, "A very compact S-box for AES," in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 441–455. [Online]. Available: https://www.iacr.org/archive/ches2005/032.pdf

[84] J. Boyar and R. Peralta, "A new combinational logic minimization technique with applications to cryptology," in *Experimental Algorithms (SEA 2010)*, ser. Lecture Notes in Computer Science, vol. 6049. Springer, 2010, pp. 178–189. [Online]. Available: https://www.nist.gov/publications/new-combinational-logic-minimization-technique-applications-cryptology

[85] H. Hadipour, "AES-VHDL: VHDL implementation of AES algorithm," https://github.com/hadipourh/AES-VHDL, 2026.

[86] J. Strömbergson, "secworks/aes: Verilog implementation of the symmetric block cipher AES as specified in NIST FIPS 197," https://github.com/secworks/aes, 2026.

[87] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, Federal Information Processing Standards Publication FIPS 180-4, Aug. 2015. [Online]. Available: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[88] J. Strömbergson, "secworks/sha256: Hardware implementation of the SHA-256 cryptographic hash function," https://github.com/secworks/sha256, 2026.

[89] E. Barker and J. Kelsey, "Recommendation for random number generation using deterministic random bit generators," NIST, Special Publication SP 800-90A Rev. 1, Jun. 2015. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/a/r1/final

[90] J. Woodage and D. Shumow, "An analysis of NIST SP 800-90A," in *Advances in Cryptology — EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, vol. 11476. Springer, 2019. [Online]. Available: https://eprint.iacr.org/2018/349

[91] E. Barker, J. Kelsey, K. McKay, A. Roginsky, and M. S. Turan, "Recommendation for random bit generator (RBG) constructions," NIST, Special Publication SP 800-90C, Sep. 2025. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/c/final

[92] M. Dworkin, "Recommendation for block cipher modes of operation: The CMAC mode for authentication," NIST, Special Publication SP 800-38B (incl. updates as of 10-06-2016), Oct. 2016. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final

[93] J. Song, R. Poovendran, J. Lee, and T. Iwata, "The AES-CMAC algorithm," IETF, RFC 4493, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4493

[94] National Institute of Standards and Technology, "CAVP testing: Block ciphers — AES test vectors (KAT_AES.zip, aesmmt.zip, aesmct.zip)," https://csrc.nist.gov/Projects/cryptographic-algorithm-validation-program/block-ciphers, 2026.

[95] ——, "CAVP testing: Block cipher modes — CMAC test vectors (cmactestvectors.zip)," https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/cavp-testing-block-cipher-modes, 2026.

[96] ——, "CAVP testing: Random number generators — DRBG test vectors (drbgtestvectors.zip)," https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/random-number-generators, 2026.

[97] ——, "ACVP-Server: NIST automated cryptographic validation protocol server and sample JSON test vectors," https://github.com/usnistgov/ACVP-Server, 2026.

[98] *SE050 Plug & Trust Secure Element — Product Data Sheet*, NXP Semiconductors, 2023. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/SE050-DATASHEET.pdf

[99] *OPTIGA Trust M Datasheet — SLS 32AIA010MH/S/K/L/M*, Infineon Technologies AG, 2024. [Online]. Available: https://www.infineon.com/assets/row/public/documents/30/49/infineon-optiga-trust-m-sls32aia-datasheet-en.pdf

[100] *STSAFE-A110: Authentication, State-of-the-Art Security for Peripherals and IoT Devices — Datasheet*, STMicroelectronics, 2019. [Online]. Available: https://www.st.com/resource/en/datasheet/stsafe-a110.pdf

[101] DigiKey Electronics, "Precios y disponibilidad de ATECC608B, SE050, OPTIGA Trust M y STSAFE-A110," https://www.digikey.com/, 2026.

[102] National Institute of Standards and Technology, "Entropy source validation certificate #e46: ECC608 NRBG entropy source," https://csrc.nist.gov/projects/cryptographic-module-validation-program/entropy-validations/certificate/46, 2023.

[103] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," in *Proc. ACM/SIGDA 14th Int. Symp. on Field Programmable Gate Arrays (FPGA '06)*, 2006, pp. 21–30.

[104] K. Yang, D. Blaauw, and D. Sylvester, "An all-digital edge racing true random number generator robust against PVT variations," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1022–1031, 2016.

[105] P. Bayon, L. Bossuet, A. Aubert, V. Fischer, F. Poucheret, B. Robisson, and P. Maurine, "Contactless electromagnetic active attack on ring oscillator based true random number generator," in *Constructive Side-Channel Analysis and Secure Design (COSADE 2012)*, ser. LNCS, vol. 7275. Springer, 2012, pp. 151–166.

[106] EUROPRACTICE, "Active EUROPRACTICE members," https://www.europractice.stfc.ac.uk/membership/membership_list.cfml, 2026.

[107] EUROPRACTICE IC Service, "MPW schedules and prices 2026," https://europractice-ic.com/schedules-prices-2026/, 2026.

[108] ——, "MPW fabrication and the mini@sic programme," https://europractice-ic.com/services/fabrication/, 2026.

[109] IHP – Leibniz-Institut für innovative Mikroelektronik, "Open source PDK for SG13G2," https://www.ihp-microelectronics.com/services/research-and-prototyping-service/fast-design-enablement/open-source-pdk, 2026.

[110] Instituto de Microelectrónica de Sevilla (IMSE-CNM, CSIC–Universidad de Sevilla), "Áreas de investigación: Hardware security," http://www.imse-cnm.csic.es/es/areas/hardware-sec.php, 2026.

[111] M. C. Martínez-Rodríguez, L. F. Rojas-Muñoz, E. Camacho-Ruiz, S. Sánchez-Solano, and P. Brox, "Efficient RO-PUF for generation of identifiers and keys in resource-constrained embedded systems," *Cryptography*, vol. 6, no. 4, p. 51, 2022. [Online]. Available: https://doi.org/10.3390/cryptography6040051

[112] P. Ortega-Castro, L. F. Rojas-Muñoz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, "VLSI integration of a RO-based PUF into a 65 nm technology," in *IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2024. [Online]. Available: https://doi.org/10.1109/NorCAS64408.2024.10752474

[113] P. Ortega-Castro, E. Camacho-Ruiz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, "Robust and scalable cell-based 65-nm CMOS RO-PUF implementation," *IEEE Open Journal of the Solid-State Circuits Society*, 2026. [Online]. Available: https://doi.org/10.1109/OJSSCS.2026.3689804

[114] J. M. Mora-Gutiérrez, C. J. Jiménez-Fernández, and M. Valencia-Barrero, "ASIC design and power characterization of standard and low power multi-radix Trivium," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 11, pp. 2682–2686, 2020. [Online]. Available: https://doi.org/10.1109/TCSII.2020.2969242

[115] IHP GmbH, "IHP-Open-PDK: Open source PDK for the IHP SG13G2 130 nm BiCMOS process," https://github.com/IHP-GmbH/IHP-Open-PDK, 2026.

[116] SkyWater Technology and Google, "SkyWater SKY130 PDK documentation: Foundry provided standard cell libraries," https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html, 2026.

[117] GlobalFoundries, Google, and FOSSi Foundation, "GF180MCU open PDK: standard cell libraries for the GlobalFoundries 180 nm MCU process," https://github.com/fossi-foundation/gf180mcu-pdk, 2026.

[118] Silicon Integration Initiative (Si2) and Silvaco (Nangate), "Open cell and free PDK libraries: Nangate 45 nm open cell library," https://si2.org/open-cell-and-free-pdk-libraries/, 2026.

[119] J.-P. Kaps, W. Diehl, M. Tempelmeier, E. Homsirikamol, K. Gaj *et al.*, "Hardware implementations of NIST lightweight cryptographic candidates: A first look," in *NIST Lightweight Cryptography Workshop 2019*, 2019. [Online]. Available: https://csrc.nist.gov/CSRC/media/Events/lightweight-cryptography-workshop-2019/documents/papers/hardware-implementations-of-nist-lwc-candidates-lwc2019.pdf

[120] J. von Neumann, “Various techniques used in connection with random digits,” in *Monte Carlo Method, National Bureau of Standards Applied Mathematics Series*. Washington, D.C.: U.S. Government Printing Office, 1951, vol. 12, pp. 36–38.

[121] S. Zhu, Y. Ma, T. Chen, J. Lin, and J. Jing, “Analysis and improvement of entropy estimators in NIST SP 800-90b for non-IID entropy sources,” *IACR Transactions on Symmetric Cryptology*, vol. 2017, no. 3, pp. 151–168, 2017.

[122] J. Kelsey, K. A. McKay, and M. Sönmez Turan, “Predictive models for min-entropy estimation,” in *Cryptographic Hardware and Embedded Systems – CHES 2015*, ser. LNCS, vol. 9293. Springer, 2015, pp. 372–393.

[123] Digilent Inc., *Basys3 FPGA Board Reference Manual (Revised August 12, 2014; applies to Basys3 rev. C)*, Digilent, 2014. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf

[124] National Institute of Standards and Technology, “Block cipher modes of operation — CMAC mode for authentication: Example values (CMAC-AES128/192/256),” https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Standards-and-Guidelines/documents/examples/AES_CMAC.pdf, 2016.

[125] A. A. Abidi, “Phase noise and jitter in CMOS ring oscillators,” *IEEE Journal of Solid-State Circuits*, vol. 41, no. 8, pp. 1803–1816, 2006.

[126] A. Hajimiri and T. H. Lee, “A general theory of phase noise in electrical oscillators,” *IEEE Journal of Solid-State Circuits*, vol. 33, no. 2, pp. 179–194, 1998.

[127] The ngspice team, “ngspice-47: Mixed-level/mixed-signal circuit simulator,” https://ngspice.sourceforge.io/, 2026.

[128] Á. Bűrmen and P. Kuthe, “OpenVAF-Reloaded: a Verilog-A compiler producing OSDI models,” https://github.com/OpenVAF-Reloaded/OpenVAF, 2026.

Apéndice A

Mapa de registros

Este apéndice es el contrato de interfaz entre el motor y su maestro, en la versión 0x03 del RTL. El mismo texto, con la historia de cada regla, está en el repositorio como `docs/mapa_registros.md`; lo siguen el RTL (`rtl/top/motor_top.vhd`), el modelo del PC (`firmware/host_check/motor_sim.c`) y la capa de acceso del firmware (`firmware/componentes/motor_i2c`).

El motor responde en la dirección I2C de 7 bits 0x30. Una escritura fija un puntero de registro de 8 bits que avanza con cada byte de datos, leído o escrito. El esclavo nunca estira el reloj: las órdenes largas se lanzan escribiendo `CONTROL` y el maestro sondea `STATUS` hasta que baja `ocupado`. Las direcciones no definidas se leen como 0x00 y las escrituras en registros de solo lectura se ignoran.

Reglas que impone el hardware. Una orden rechazada no llega a subir `ocupado` y levanta `DIAG.rechazada`.

- Con una alarma de salud levantada se rechazan `sembrar`, `resembrar`, `generar` y `aleatorio`; `autenticar` se sigue atendiendo, porque usa la clave ya generada y no consume entropía.
- No se siembra antes de pasar el test de arranque (SP 800-90B §4.3 [6]). La semilla solo entra en el DRBG cuando se cierra sin alarma la ventana del APT que contiene su último bit; una alarma o un `borrar_alarmas` durante la recogida aborta la siembra y deja el estado anterior intacto. Una siembra tarda como mucho $384+1024$ bits crudos.
- `sembrar` y `resembrar` se rechazan con `CFG_KD` por debajo del mínimo (genérico `G_KD_MIN`, 6 por defecto), por debajo del cual el cruce de dominio de reloj del bit pierde bits en silencio sin que los tests de salud puedan notarlo.
- Una captura y una siembra se excluyen, así que el búfer de captura nunca contiene bits de una semilla.

Tabla A.1: Registros. R: lectura, W: escritura.

Dirección	Nombre	Acceso	Contenido
0x00	ID	R	0xA5
0x01	VERSION	R	0x03
0x02	STATUS	R	Tabla A.2
0x03	CONTROL	W	órdenes, Tabla A.4; se autolimpian
0x04	CFG_KD	R/W	exponente del divisor, $K_D = 2^{\text{valor}}$, rango efectivo 0–24 (los valores mayores equivalen a 24). Tras el reset vale 24. Solo con el motor libre: una escritura con una orden en curso se ignora
0x05	CFG_PAG	R/W	página del búfer de captura, 0–3
0x06–0x07	RCT_MAX	R	racha más larga vista por el Repetition Count Test, prueba de recuento de repeticiones (RCT), byte bajo primero
0x08–0x09	APT_MAX	R	cuenta más alta vista por el APT, byte bajo primero
0x0A–0x0D	PETICIONES	R	contador de resembrado del DRBG, 32 bits, byte bajo primero. Se satura en $2^{32} - 1$
0x0E	DIAG	R	Tabla A.3
0x10–0x1F	CLAVE	R	clave actual, 16 bytes. **Solo en modo test**
0x20–0x2F	RETO	W	nonce de 16 bytes para el reto-respuesta
0x30–0x3F	ETIQUETA	R	$\text{CMAC}_{\text{CLAVE}}$(RETO), 16 bytes; se lee como ceros mientras `etiqueta_lista` vale 0
0x40–0x5F	ALEATORIO	R	32 bytes de la última orden `aleatorio`, una generación del DRBG independiente de la clave
0x80–0xFF	CAPTURA	R	128 bytes de bits crudos, página CFG_PAG. **Solo en modo test**

Tabla A.2: STATUS (0x02).

Bit	Nombre	Significado
0	sembrado	el DRBG tiene estado válido
1	ocupado	hay una orden en curso; no mandar otra
2	alarma_rct	test de repetición disparado (fuente pegada)
3	alarma_apt	test de proporción disparado (fuente sesgada)
4	clave_valida	hay clave cargada en el autenticador
5	captura_llena	el búfer de captura tiene un trozo listo
6	etiqueta_lista	ETIQUETA corresponde al RETO actual
7	modo_test	el pin de test está activo

Tabla A.3: DIAG (0x0E).

Bit	Nombre	Significado
0	arrancado	el test de arranque ha pasado: una ventana completa del APT (1024 muestras) sin alarma desde el reset o desde el último `borrar_alarmas`
1	rechazada	la última orden escrita en CONTROL se ignoró o, si era una siembra, se abortó. Se baja con la siguiente orden aceptada
2	agotado	el contador de peticiones está en $2^{32} - 1$; `generar` y `aleatorio` se rechazan hasta resembrar

Tabla A.4: CONTROL (0x03). Una orden por escritura.

Bit	Orden	Qué hace
0	sembrar	recoge 384 bits crudos (256 de entropía y 128 de nonce), espera a que los tests de salud los hayan evaluado enteros e instancia el DRBG desde cero
1	generar	pide 256 bits; los 128 primeros pasan a ser la CLAVE y se cargan en el autenticador, el resto se descarta. No deja nada legible
2	autenticar	ETIQUETA $\leftarrow \mathrm{CMAC}_{\mathrm{CLAVE}}$(RETO)
3	borrar_alarmas	baja las alarmas de salud; reinicia la ventana de arranque
4	capturar	llena el búfer de captura con bits crudos. Solo en modo test, con el motor libre y sin otra captura en marcha
5	resembrar	como `sembrar`, pero sobre el estado actual; exige un DRBG sembrado
6	aleatorio	pide 256 bits y los deja en ALEATORIO
7	borrar_todo	zeroización: CLAVE, todo el estado del DRBG y sus intermedios (también el núcleo AES), RETO, ALEATORIO, ETIQUETA y la semilla recogida a cero, y la clave nula cargada en el autenticador. Se atiende siempre que el motor esté libre, también con alarma

- El contador de peticiones no da la vuelta: en $2^{32} - 1$ el motor se niega a generar hasta resembrar (`DIAG.agotado`). La política de resembrado por debajo de ese tope es del maestro.

Prioridad. Si llegan varios bits de orden en una misma escritura, solo se atiende el de mayor prioridad: `borrar_todo`, después `sembrar`/`resembrar` (`resembrar` si van los dos), `generar`, `aleatorio`, `autenticar`, `capturar`. `borrar_alarmas` se atiende junto con cualquiera de ellas, pero no debe compartir escritura con una siembra: reinicia la ventana de arranque, así que la siembra se acepta y se aborta en el ciclo siguiente.

Secuencia típica del maestro.

1. Leer ID (0xA5) y VERSION (0x03).
2. Escribir `CFG_KD` con el divisor de la caracterización, o dejar el valor de reset.
3. Sondear DIAG hasta `arrancado`; escribir `sembrar`; sondear STATUS hasta que baje `ocupado`; comprobar `DIAG.rechazada` $= 0$, `sembrado` $= 1$ y sin alarmas.
4. Escribir `generar`, sondear, comprobar `DIAG.rechazada`. Ya hay clave.
5. Solo para provisionar en banco, en modo test: leer CLAVE.
6. Reto-respuesta: escribir RETO, escribir `autenticar`, sondear, leer ETIQUETA y compararla con el CMAC que calcula el maestro.
7. Aleatorio para el maestro: escribir `aleatorio`, sondear, leer ALEATORIO.

Modo test. El pin `modo_test`, sincronizado con dos biestables, habilita la lectura de CLAVE y CAPTURA y la orden `capturar`. En la FPGA es un interruptor de la placa; en un circuito integrado sería un fusible que se quema en producción.

Apéndice B

Vectores de prueba oficiales utilizados

La Tabla B.1 recoge de dónde sale cada valor de referencia que usan los bancos y cómo se cierra cada comprobación. Se usan vectores oficiales siempre que una norma los publica. Donde la disposición de entrada del hardware no coincide con ningún caso publicado, los valores esperados salen de un modelo de referencia en Python que se contrasta antes con los vectores oficiales y que está escrito de forma independiente del VHDL.

Los ficheros del CAVP se guardan en el repositorio (`tests/vectors/drbg/`), así que la validación no depende de que la web del NIST siga igual. Las estimaciones de entropía del Capítulo 8 no son vectores de prueba: salen de la herramienta SP 800-90B del NIST [37], ejecutada sin cambios.

Tabla B.1: Valores de referencia y su origen. Las cuentas son las comprobaciones superadas en la última ejecución (`results/tb_motor_top.log`, 177 en total, con 11 de 11 aserciones PSL y 6 de 6 coberturas).

Bloque	Origen de los valores esperados	Dónde se comprueba
AES-128 (los dos núcleos)	FIPS 197 [78], apéndices B y C.1; SP 800-38A, F.1.1 ECB-AES128, tres bloques; mil cifrados encadenados repetidos; de nuevo el apéndice C.1 tras `borrar`	`tb_aes_enc`, 18 comprobaciones, los dos núcleos con los mismos estímulos y obligados a coincidir
AES-CMAC	RFC 4493 [93] (los ejemplos de SP 800-38B [92]): los cuatro mensajes de 0, 16, 40 y 64 bytes con la clave `2b7e1516...`, y el de 16 bytes repetido; las subclaves K_1 y K_2 del mismo ejemplo	`tb_aes_cmac`, 7 comprobaciones
CTR_DRBG, modelo de referencia	`drbgtestvectors` del CAVP [96], `CTR_DRBG_no_reseed.rsp` y `CTR_DRBG_pr_false.rsp`, AES-128 con y sin función de derivación: $4 \times 240 = 960$ casos	`analysis/ctr_drbg_ref.py`, 960 de 960
CTR_DRBG, RTL	el modelo de referencia anterior, con la disposición de entrada del hardware (entropía de 256 bits, nonce de 128, sin personalización): `analysis/gen_drbg_vectores.py` escribe `drbg_vectores_pkg.vhd`	`tb_ctr_drbg`, 18 comprobaciones, incluidas la zeroización y la saturación del contador
Motor completo	la clave sale del ruido del anillo, así que el banco vuelca clave, reto y etiqueta y el CMAC se recalcula con el modelo en Python	`analysis/verifica_top.py`, 5 de 5
Firmware contra el modelo del motor	CMAC y ECDSA del mbedtls 3.6 del objetivo, compilado para el PC; las lecturas firmadas se verifican con la biblioteca `cryptography`	`run_host.py`, 0 fallos; `verifica_lecturas.py`
Cortes de los tests de salud	fórmulas de SP 800-90B §4.4 [6] para $H = 0{,}98$ y $\alpha = 2^{-20}$: RCT $C = 22$, APT $W = 1024$, $C = 596$	`analysis/cortes_salud.py`; comportamiento en `tb_health_tests`, 15 comprobaciones

Apéndice C

Guía de reproducción

Todo lo que en este trabajo no necesita las placas se vuelve a ejecutar en cada push mediante un flujo de integración continua (`.github/workflows/verifica.yml`) en una máquina Ubuntu limpia que no es la del autor: siete trabajos, sin credenciales. El repositorio es público: `https://github.com/mmggjj/TFM_ENCRY`. Las órdenes de abajo son las que ejecuta el flujo, así que una ejecución en verde las reproduce. Las salidas nunca se filtran: los scripts imprimen la salida completa de cada herramienta y fallan ante cualquier error.

Tabla C.1: Lo que reproduce el flujo de integración continua.

Trabajo	Orden	Resultado
Modelo y estimadores	`python analysis/ero_model.py`, `jitter_estimator.py`, `ctr_drbg_ref.py`, `cortes_salud.py`, `fuzz_verifica_lecturas.py`, `verifica_lecturas.py` sobre el log de la placa	autopruebas, 960 casos del CAVP, lecturas de la placa válidas
Bancos VHDL	`python rtl/run_tb.py` y después `python analysis/verifica_top.py`	7 bancos, informe PSL como artefacto
Síntesis	`python synth/sintetiza.py -verilog`	área por bloque en kGE (Capítulo 7)
Firmware en el PC	`python firmware/host_check/run_host.py -mbedtls <dir>`, y otra vez con `-sanear`	0 fallos, también con ASan y UBSan
Compilación con ESP-IDF	`idf.py set-target`, `build`, `size`, e `idf.py -B build_sim -DMOTOR_SIMULADO=1 build`, para el verificador (ESP32) y el medidor (ESP32-C3)	4 imágenes
Memoria	`tectonic tfm_main.tex` y `tfm_main_es.tex`	los dos PDF como artefactos

Herramientas. Python 3.12 con `numpy`, `scipy` y `cryptography`; GHDL de los paquetes de Ubuntu (5.0.1 con el motor mcode en la máquina del autor) para los bancos, que se ejecutan en VHDL-2008; Yosys mediante `yowasp-yosys`, o la OSS CAD Suite con el plugin de GHDL, y la biblioteca de celdas estándar IHP SG13G2 del PDK abierto [115]; mbedtls 3.6.2 del tarball de publicación (un clon de git no trae los fuentes generados); ESP-IDF 5.3.5, en la integración continua con el contenedor `espressif/idf:v5.3.5`; tectonic 0.15. La versión de desarrollo GHDL 7.0 con el motor LLVM se cae con las propiedades PSL de `tb_motor_top` que usan nombres externos, y no debe usarse.

Fuera del flujo. Tres partes necesitan software o hardware que el flujo no tiene:

- *Anillo a nivel de transistor.* ngspice 47 [127] con OpenVAF-Reloaded [128] para compilar los modelos PSP del PDK; `python analysis/anillo_ihp_repeticiones.py`, con `-esquina`, `-temp`, `-vdd` y `-mismatch` para esquinas, alimentación y desapareamiento. ngspice no deja fijar la realización del ruido, así que cada ejecución es una nueva y la incertidumbre sale de repetir. Una realización de 400 ns tarda unos 30 minutos.
- *Estimaciones SP 800-90B del ESP32.* La herramienta del NIST [37] compilada desde el fuente; `python analysis/entropia_90b.py` y después `resumen_90b.py`, sobre las capturas guardadas en `results/`; la prueba de reinicios con `reinicios_90b.py`. Las capturas nuevas necesitan un ESP32 y `analysis/campana_rng.py`.
- *FPGA.* Vivado (nivel gratuito, con el Artix-7 incluido) y `synth/basys3/build.tcl`; la instalación y la grabación se describen en `docs/instalacion_vivado_windows.md`.

Primera campaña del ESP32. El análisis de la primera campaña (Secciones 3.3 a 3.5) se vuelve a ejecutar sin la placa a partir de los ficheros guardados en `results/esp32_rng_agosto`, con `numpy`, `scipy`, `matplotlib` y `cryptography`:

- `python analysis/esp32_rng/nist_sp800_22.py` ejecuta la autocomprobación de la batería SP 800-22 (datos del sistema operativo, ceros, patrón alternado).
- `python analysis/esp32_rng/harvester_from_sources.py` escribe las 1000 semillas de la validación fuera de línea en `seeds_rep1.bin`, a partir de `puf_boot1.bin` y `condD_rep1.bin`.
- `python analysis/esp32_rng/run_campaign.py` ejecuta `nist_analysis.py`, `entropy_analysis.py`, `temperature_correlation.py`, `harvester_validation.py` y el modo fuera de línea de `mqtt_tls_demo.py`. Cada uno escribe sus CSV y figuras en el directorio de datos, y `run_campaign.py` copia

las figuras en `memoria/figures` con sus propios nombres; la memoria solo usa `nist_heatmap.pdf`, como `esp32_nist_heatmap.pdf`. Los cuatro scripts también se ejecutan uno a uno, con `-d <directorio de datos>` y `-figdir <directorio>`. Ejecutados de nuevo el 2026-10-31 sobre una copia de los datos, reproducen los CSV guardados, valor a valor.

- `python analysis/esp32_rng/make_result_tables.py` escribe las tablas de resultados en LaTeX en `tablas_generadas.tex`, y `python analysis/esp32_rng/figuras_memoria.py` dibuja las Figuras 3.3, 3.4 y 3.6 en `memoria/figures`.

Una captura nueva necesita un ESP32 con `firmware/esp32_rng` compilado y grabado (`idf.py set-target esp32`, `build`, `-p <puerto> flash`; ESP-IDF 5.3, consola a 115 200 baudios). Después, `python analysis/esp32_rng/capture_rng.py -port <puerto> -reps <n>` reinicia la placa por RTS y recoge n secuencias válidas por condición, con los patrones de SRAM (`puf_boot<k>.bin`), las semillas y `capture_log.csv`; por defecto escribe en `results/esp32_rng_agosto`, así que `-o <directorio>` mantiene aparte los datos nuevos. Un reinicio por RTS mantiene alimentada la SRAM y no da un patrón de encendido nuevo: para la serie limpia, se desenchufa la placa, se vuelve a enchufar y se ejecuta `python analysis/esp32_rng/capture_puf.py -port <puerto>` una vez por encendido, lo que añade un `puf_pc<N>.bin`. Las campañas posteriores usan el mismo firmware con `analysis/campana_rng.py`, que guarda el flujo serie íntegro y numera las repeticiones a partir de `-primera-rep`.